Distributed Systems & Cloud Computing (CIE-407T)

Mid-Sem Study Pack: Unit-I & Unit-II

BPIT · GGSIPU · B.Tech IT Semester VII

00 Syllabus Checklist and Guide
01 Unit1 Notes
02 Unit2 Notes
03 Faculty Emphasis and Quick Tables
04 Question Bank with Answers
05 Predicted Practice and Top30
06 Model Papers
07 Revision Viva and Writing

CIE-407T — Distributed Systems & Cloud Computing

MID-SEM Study Pack: UNIT-I + UNIT-II ONLY

BPIT (IT, Sem VII) · GGSIPU (USICT handbook, batch 2021-22 onwards)

---

0. How to use this pack

File	What is inside
`00_Syllabus_Checklist_and_Guide.md`	This file – checklist, source labels, exam pattern
`01_Unit1_Notes.md`	Complete Unit-I notes (Intro, models, IPC, RMI, RPC, events, Java RMI)
`02_Unit2_Notes.md`	Complete Unit-II notes (time, clocks, global state, mutual exclusion, election, multicast, consensus)
`03_Faculty_Emphasis_and_Quick_Tables.md`	Faculty-emphasis ranking, definitions, comparisons, diagrams, algorithms
`04_Question_Bank_with_Answers.md`	Question bank (questions first, model answers after)
`05_Predicted_Practice_and_Top30.md`	Extra "Claude Predicted Practice Questions" + Top-30 list
`06_Model_Papers.md`	3 model mid-sem papers + answer key
`07_Revision_Viva_and_Writing.md`	One-day revision sheet, 30 viva Qs, answer-writing format

Suggested order: Notes (files 01, 02) → Faculty emphasis (03) → attempt questions in 04 *without looking at answers* → 05 → model papers under time → 07 the night before.

---

1. Source labels used everywhere

Label	Meaning
📘 **[Official syllabus]**	The topic is written in the GGSIPU syllabus PDF you uploaded
📝 **[Assignment-emphasized]**	The topic/question appears in your BPIT Assignment-1 or Assignment-2
➕ **[Additional / exam-oriented]**	NOT named in the syllabus, but added because textbooks (Coulouris – Distributed Systems: Concepts and Design; Ghosh; Tanenbaum) cover it under the same topic and examiners often expect it
🎯 **[Claude Predicted Practice Question]**	A question I created for practice. **These are predicted/practice questions based on syllabus and emphasis, not leaked or guaranteed questions.**

> ⚠️ Nothing here is guaranteed to appear. The notes are built to make you *understand* the topics so you can answer any variation.

---

2. What the syllabus PDF actually says (verbatim summary)

Paper: CIE-407T, Distributed Systems and Cloud Computing, L-3, Credits 3. Theory: 25 marks internal (TCE) + 75 marks term-end.

UNIT-I – Introduction to Distributed Systems: Characteristics of distributed systems – Introduction; Examples of distributed systems (client-server, peer-to-peer, grid and cloud computing); Advantages of distributed systems; System models – Introduction, Architectural and Fundamental models; Networking and Internetworking; Interprocess Communication (message passing and shared memory); Distributed objects and Remote Method Invocation; RPC; Events and notifications; Case study – Java RMI.

UNIT-II – Synchronization: Time and Global States – Introduction; Logical clocks; Synchronizing physical clocks; events and process states; logical time and logical clocks; global states; distributed debugging. Coordination and Agreement: Distributed mutual exclusion; Elections; Multicast communication; consensus and related problems.

Textbooks (from syllabus): (1) Coulouris, Dollimore, Kindberg – *Distributed Systems: Concepts and Design*, 4th ed., Pearson; (2) Buyya – *Cloud Computing Principles and Paradigms*; (3) S. Ghosh – *Distributed Systems*, CRC Press 2010. **References:** Tanenbaum & van Steen; Kshemakalyani & Singhal; Gerard Tel.

Term-end paper pattern (from the handbook) – useful to know how questions are framed: - 9 questions in total. Q1 compulsory, objective/short, covers the entire syllabus, **15 marks**. - 4 units, each unit has 2 questions (attempt any 1), each question can have up to 5 sub-parts, **each unit = 15 marks**. - Standard of questions = level of the prescribed textbook.

> Note: your **mid-sem** is an internal BPIT exam, so the exact pattern may differ. Ask your faculty for marks/duration. This pack prepares 1-mark to 10-mark answers so that any pattern is covered.

Out of scope for mid-sem – "Outside Mid-Sem Syllabus": Unit-III (Distributed File Systems, Hadoop, load balancing/sharing, task assignment) and Unit-IV (Cloud Computing layers, IaaS/PaaS/SaaS, migration). Only the *idea* of "cloud computing" as an **example of a distributed system** is in Unit-I, so it is covered briefly there.

---

3. MID-SEM SYLLABUS CHECKLIST (tick as you finish)

UNIT-I

#	Topic (as in your brief)	Where	Assignment link	Done
1	Introduction to Distributed Systems	01 §1	A1-1a	☐
2	Characteristics of Distributed Systems	01 §2	A1-1a	☐
3a	Example: Client–Server	01 §3.1	A1-1b	☐
3b	Example: Peer-to-Peer	01 §3.2	A1-1b	☐
3c	Example: Grid Computing	01 §3.3	A1-1b	☐
3d	Example: Cloud Computing (as an example only)	01 §3.4	A1-1b	☐
3e	Comparison CS vs P2P vs Grid vs Cloud	01 §3.5	A1-1b	☐
4	Advantages of Distributed Systems	01 §4	–	☐
5a	System Models – Introduction	01 §5.1	A1-2a	☐
5b	Architectural Models	01 §5.2	A1-2a	☐
5c	Fundamental Models (interaction, failure, security)	01 §5.3	A1-2a	☐
6	Networking and Internetworking	01 §6	A1-2b	☐
7a	IPC – Message Passing	01 §7.1	A1-3a	☐
7b	IPC – Shared Memory	01 §7.2	A1-3a	☐
7c	Message Passing vs Shared Memory	01 §7.3	A1-3a	☐
8	Distributed Objects	01 §8	A1-3b	☐
9	Remote Method Invocation (RMI)	01 §9	A1-3b, A1-1c	☐
10	RPC	01 §10	(indirect)	☐
10b	RMI vs RPC	01 §10.5	–	☐
11	Events and Notifications	01 §11	–	☐
12	Case Study: Java RMI	01 §12	A1-1c	☐

UNIT-II

#	Topic	Where	Assignment link	Done
1	Introduction to Synchronization	02 §1	A2-1a	☐
2	Time and Global States (intro)	02 §2	A2-1a	☐
3	Logical Clocks	02 §5	A2-1b	☐
4	Synchronizing Physical Clocks (Cristian, Berkeley, NTP)	02 §7	A2-1c	☐
5	Events and Process States	02 §3	A2-1a	☐
6	Logical Time and Logical Clocks (Lamport, Vector)	02 §4–6	A2-1b	☐
6b	Logical vs Physical time	02 §8	–	☐
7	Global States (cut, snapshot)	02 §9	A2-2a, 2b	☐

#	Topic	Where	Assignment link	Done
8	Distributed Debugging	02 §10	–	☐
9	Coordination and Agreement (intro)	02 §11	–	☐
10	Distributed Mutual Exclusion	02 §12	A2-3a	☐
11	Election Algorithms (Bully, Ring)	02 §13	A2-3b	☐
12	Multicast Communication	02 §14	–	☐
13	Consensus and Related Problems	02 §15	A2-4a, 4b	☐

Coverage statement: every one of the 21 Unit-I and 13 Unit-II items in your brief has a section in the notes.

---

4. Symbols used in diagrams

```
P1, P2, P3   = processes            ──►  message / arrow
[ ]          = a box (component)    - ►   reply / return
●            = event on timeline    ✗    crashed process
```

Hinglish tips are marked with 🗣️.

UNIT-I — INTRODUCTION TO DISTRIBUTED SYSTEMS (Complete Notes)

Labels: 📘 official syllabus · 📝 assignment-emphasized · ➕ additional/exam-oriented

---

1. WHAT IS A DISTRIBUTED SYSTEM? 📘📝

Definition

A **distributed system** is a system in which **hardware or software components located at networked computers communicate and coordinate their actions only by passing messages** (Coulouris et al.). Another common form: *"A collection of independent computers that appears to its users as a single coherent system"* (Tanenbaum).

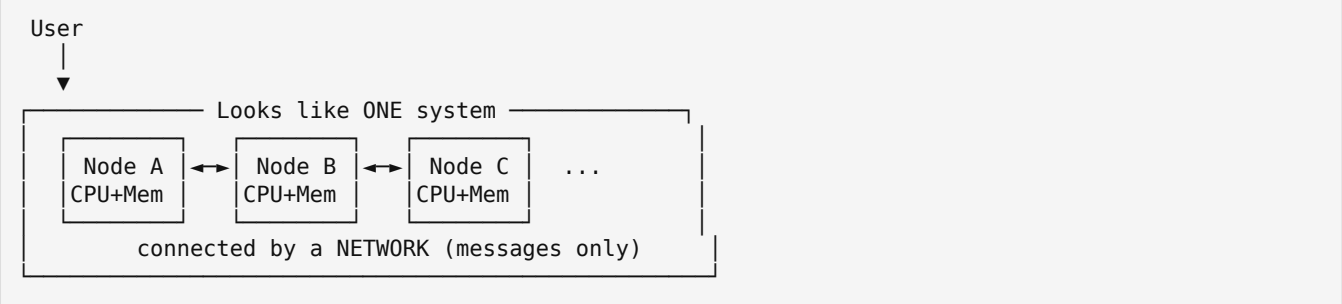
Simple explanation (🗣️ Hinglish)

Bahut saare alag-alag computers network se jude hain aur milkar ek kaam karte hain, lekin user ko lagta hai ki ek hi system hai. Jaise **Google Search** – aapko ek search box dikhta hai, peeche hazaaron computers kaam kar rahe hote hain.

Detailed explanation

- Each computer (node) has its **own CPU, memory, OS clock**. There is **no shared memory** and **no shared clock**.
- Nodes cooperate **only via messages** over a network (LAN/WAN/Internet).
- The system hides the complexity → the user sees "one computer" (this is called **transparency**).
- Examples: the Web, ATM/banking network, email, online multiplayer games, Google Search, Netflix, UPI payments, cloud platforms.

Diagram



Advantages / Disadvantages (short)

Advantages	Disadvantages
Resource sharing, scalability, reliability	Complex design, hard debugging

Advantages	Disadvantages
Better performance through parallelism	Network delays and failures
Geographic distribution	Security risks; no global clock

Exam points

- Key phrases: *autonomous computers, network, message passing, coordination, single coherent system*.
- Motivation: **resource sharing**.

🧠 Remember This

> DS = independent computers + network + only message passing + user sees one system.

---

2. CHARACTERISTICS OF DISTRIBUTED SYSTEMS 📘📝 (A1-1a)

Coulouris lists **three core characteristics**; textbooks add others (➕ marked).

#	Characteristic	Meaning	Example
1	**Concurrency**	Many components execute at the same time; they share resources (files, DB)	Many users booking tickets at IRCTC together
2	**No global clock**	Each computer has its own clock; clocks drift; only message passing can coordinate time	Two servers show different times → ordering problem (→ Unit-II)
3	**Independent failures**	Any node or network link can fail without others failing; failure of one may not be known to others	A server crashes but other servers still run
4 ➕	**Resource sharing**	Hardware/software/data shared across network	Shared printers, cloud storage
5 ➕	**Heterogeneity**	Different networks, hardware, OS, languages, implementations; hidden by *middleware*	Android phone talking to Linux server
6 ➕	**Openness**	System can be extended/re-implemented using published interfaces	Web standards (HTTP, REST)
7 ➕	**Scalability**	Works well when users/resources increase	YouTube
8 ➕	**Fault tolerance / Failure handling**	Detect, mask, tolerate failure, recover, use redundancy	RAID, replicated DB
9 ➕	**Transparency**	Hide distribution from user (see below)	Google Drive file looks local

#	Characteristic	Meaning	Example
10 ➕	**Security**	Confidentiality, integrity, availability	HTTPS, authentication
11 ➕	**Quality of Service (QoS)**	Reliability, performance, security, adaptability	Video streaming without buffering

Types of Transparency ➕ (ISO/RM-ODP; Coulouris)

Type	Hides
Access	Differences in data representation & how resource is accessed
Location	Where the resource physically is (URL doesn't reveal server location)
Concurrency	That many users share a resource simultaneously
Replication	That multiple copies exist
Failure	Faults and recovery
Mobility	Movement of resources/clients
Performance	Reconfiguration to improve performance
Scaling	System growth without changing structure

Challenges (➕ Coulouris): heterogeneity, openness, security, scalability, failure handling, concurrency, transparency, quality of service.

Exam points

- If asked "Define and discuss characteristics with examples" (2–3 marks): definition + **the 3 core ones** + 2–3 more + one example each.
- Examiner favourite: **"no global clock"** and **"independent failure"** (they cause Unit-II problems).

🧠 Remember This

> Concurrency · No global clock · Independent failures (core 3) + heterogeneity, openness, scalability, transparency, fault tolerance, security.

---

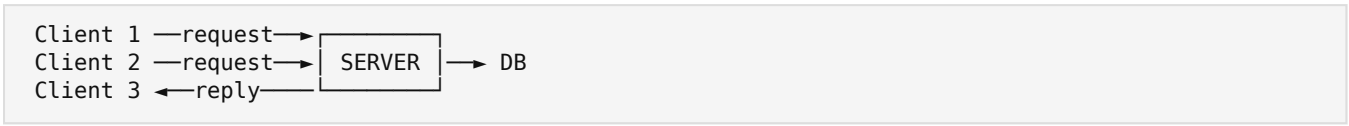
3. EXAMPLES OF DISTRIBUTED SYSTEMS 📘📝 (A1-1b)

3.1 Client–Server

Definition: An architecture where **client** processes send requests to **server** processes, which manage resources and reply.

Simple (🗣️): Aap browser (client) se website maangte ho, web server data bhejta hai. Server hamesha "on" rehta hai, client jab chahe request karta hai.

Working (steps) 1. Client sends request (e.g., HTTP GET). 2. Server receives, processes (maybe queries DB). 3. Server sends response. 4. Client displays result.



Examples: Web browser–web server, email (SMTP/IMAP), ATM–bank server, file server (NFS), database server. **Advantages:** central control, easy security & backup, easy to manage, clear roles. **Disadvantages:** server = **single point of failure**, bottleneck under heavy load, costlier to scale, network dependent. **Variants:** 2-tier, 3-tier (presentation–application–data), n-tier.

3.2 Peer-to-Peer (P2P)

Definition: Every node (**peer**) acts as **both client and server**; peers share resources directly without a central server.

Simple (🗣️): Sab barabar hain. Torrent me aap file download bhi karte ho aur dusron ko upload bhi.



Types: *Centralized-index* (Napster), *pure/unstructured* (Gnutella), *structured with DHT* (Chord, Kademlia) ➕. **Examples:** BitTorrent, Blockchain/Bitcoin nodes, Skype (early), IPFS. **Advantages:** no single point of failure, scales as more peers join, cost-effective, resource utilisation. **Disadvantages:** hard to secure/control, hard searching/management, uneven peer availability, copyright/legal issues.

3.3 Grid Computing

Definition: A system that **coordinates loosely-coupled, heterogeneous, geographically dispersed computers** (belonging to different organisations) to work as a **virtual supercomputer** for large tasks.

Simple (🗣️): Alag-alag jagah ke computers ka idle power jod kar ek bahut bada kaam (jaise weather simulation) karna. Jaise bijli ka *power grid* – socket lagao, power lo.



Examples: SETI@home, CERN LHC Computing Grid, Folding@home, weather/genome projects. **Features:** heterogeneous resources, multiple administrative domains, batch-oriented,

resource discovery & scheduling, uses **Globus toolkit** ➕. **Advantages:** huge computing power at low cost, uses idle resources, scalable, reliability via redundancy. **Disadvantages:** complex middleware, security & trust across organisations, resource scheduling difficulty, slow WAN links.

3.4 Cloud Computing (only as an *example* of a DS here)

Definition: Delivery of computing services (servers, storage, DB, software) **over the Internet on a pay-per-use, on-demand basis** with rapid elasticity (NIST-type definition ➕).

Simple (🗣️): Apna server rakhne ke bajaye AWS/Azure/Google se kiraye pe computer lena – jitna use, utna paisa.

```
Users ──Internet──► [ Cloud Provider Data Centres ]
                     IaaS | PaaS | SaaS (service models)
                     virtualised, shared pool of resources
```

Key features: on-demand self-service, broad network access, resource pooling (virtualisation), rapid elasticity, measured service. **Examples:** AWS, Microsoft Azure, Google Cloud, Gmail/Google Docs (SaaS). **Advantages:** no upfront cost, elastic scaling, high availability, easy access anywhere. **Disadvantages:** internet dependence, data privacy/vendor lock-in, recurring cost, less control.

> ⚠️ **Outside Mid-Sem Syllabus:** service models (IaaS/PaaS/SaaS), deployment types, migration – Unit-IV. Just mention them in one line if asked in Unit-I.

3.5 COMPARISON: Client–Server vs P2P vs Grid vs Cloud 📝

Feature	Client–Server	Peer-to-Peer	Grid Computing	Cloud Computing
Basic idea	Clients request, servers serve	All nodes equal (client+server)	Pool many organisations' resources as virtual supercomputer	On-demand services over Internet
Control	Centralised (server)	Decentralised	Decentralised/virtual organisation, scheduler	Centralised by provider (data centres)
Roles	Fixed	Dynamic, symmetric	Resource provider / consumer	Provider / customer
Main purpose	Service access, resource sharing	File/resource sharing	Solve huge computational problems	Rent IT services (compute, storage, apps)
Scalability	Limited by server	High	Very high	Very high (elastic)
Failure	Server = SPOF	No SPOF	Job rescheduling on failure	Redundancy across data centres
Ownership	One organisation	Individual users	Multiple organisations	Cloud provider
Resource type	Homogeneous mostly	Heterogeneous	Heterogeneous, distributed	Virtualised, homogeneous pool

Feature	Client-Server	Peer-to-Peer	Grid Computing	Cloud Computing
Billing	Usually none/fixed	Free	Usually project/sharing based	**Pay-per-use**
Typical use	Web, e-mail, DB	BitTorrent, blockchain	SETI@home, CERN	AWS, Azure, Gmail
Security	Easier to control	Hard	Hard (trust across domains)	Provider-managed, shared responsibility

Exam point: A 2-mark answer = one-line each + 1 application each; a 5-mark answer = table + diagrams.

🧠 Remember This

> CS = central server · P2P = everyone equal · Grid = many orgs share compute for big jobs · Cloud = rent services pay-per-use.

---

4. ADVANTAGES OF DISTRIBUTED SYSTEMS 📘

Advantage	Explanation
Resource sharing	Printers, files, data, CPUs shared over network
Scalability	Add more nodes to handle more load (horizontal scaling)
Reliability/Availability	One node failure doesn't stop the whole system (replication)
Performance/Speed	Parallel execution of tasks, load distribution
Economics	Many cheap PCs give better price/performance than one mainframe
Geographical distribution	Users/data near each other → low latency (CDNs)
Incremental growth	Add capacity gradually
Openness/Flexibility	Mix hardware/software from different vendors
Fault tolerance	Redundancy hides failures

Disadvantages (limitations): complexity, network dependence, security threats, no global clock/state, difficult debugging, partial failure, higher software cost, coordination overhead.

🧠 Remember This

> "Share, Scale, Survive (reliability), Speed, Save (cost)".

---

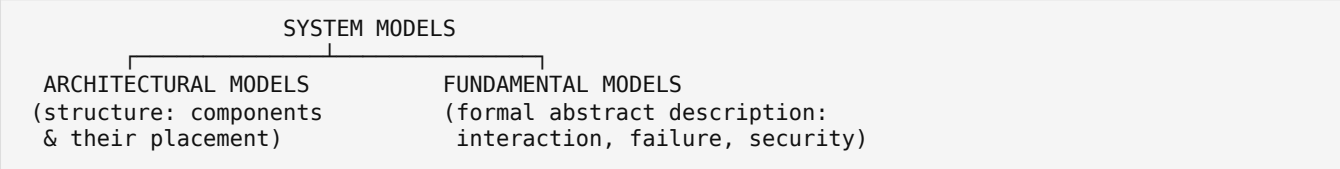
5. SYSTEM MODELS 📘📝 (A1-2a)

5.1 Introduction – need for system models

Definition: A **system model** is an **abstract, simplified description** of the relevant aspects of a distributed system (its components, their interactions, and the properties/assumptions about failures and security).

Why we need them (🗣️: "naksha banake samajhna"): 1. To show the **common properties & design issues** of DS. 2. To make **explicit assumptions** (e.g., messages may be lost, clocks drift). 3. To reason about **what is possible / impossible** and **prove** algorithm correctness. 4. To generalise – an algorithm proved under a model works in all systems satisfying that model. 5. Helps compare systems and decide design.

Classification:



5.2 ARCHITECTURAL MODELS

Definition: Describe the **structure of the system in terms of components, their placement on network, and their relationships/roles**. Ideas: *placement of components*, *relationship (client-server, peer)*.

(a) Software layers & Middleware



- **Platform:** lowest layers (hardware + OS), e.g., Intel x86 + Windows/Linux.
- **Middleware:** software layer that **masks heterogeneity** and gives a uniform programming model (RMI, RPC, CORBA). 🗣️ "Beech ka pul".

(b) System architectures (main styles)

1. **Client–server model** – see §3.1. (Diagram above)
2. **Multiple servers** – a service is provided by **several server processes** on different hosts through **partitioning** (each holds part of data, e.g., web servers) or **replication** (copies for fault tolerance, e.g., DNS). `Client → Server 1 ┐ Client → Server 2 ┼ cooperate to provide one service Client → Server 3 ┘`
3. **Proxy servers and caches** – **cache** stores recently used objects near clients; **proxy** shares cache for many clients. Reduces latency & network load. (browser cache, web proxy, CDN) `Client → Proxy/Cache → Web server`
4. **Peer processes (P2P)** – all processes play similar roles (§3.2).

5. **Mobile code** – code is downloaded to client and runs locally (Java applets, JavaScript). Pros: interactive/faster; cons: security risk.
6. **Mobile agents** – running program (code+data) that **travels** between computers doing work on behalf of user. Pros: reduces communication; cons: security/tracking.
7. **Thin clients / network computers** – minimal local processing; heavy work on server (cloud desktops, Chromebook).
8. **Mobile devices & spontaneous networking** ➕ – phones, IoT discover each other (Bluetooth).

(c) Layered / tier architecture (➕ common exam answer)

Tier	Role
Presentation	UI
Application (logic)	Business rules
Data	Database
- *Layering* = vertical organisation of services (each layer uses the one below). *Tiering* = physical distribution of layers over servers.	

(d) Design requirements for distributed architectures (➕ Coulouris)

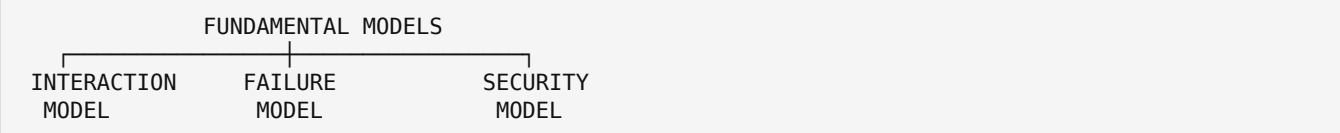
Performance (responsiveness, throughput, load balancing), **Quality of service** (reliability, security, adaptability), **Dependability** (correctness, fault-tolerance, security), caching & replication.

Exam-point table – architectures

Architecture	Key idea	Example
Client–server	Request/reply	Web
Multiple servers	Partition/replicate	DNS, Google
Proxy/cache	Nearby copies	Web proxy
Peer-to-peer	Equal roles	BitTorrent
Mobile code	Code moves to client	JavaScript
Mobile agent	Program moves around	Info-gathering agent
Thin client	Compute on server	Cloud desktop

5.3 FUNDAMENTAL MODELS

Definition: More **formal, abstract** descriptions of the **properties common to all architectures**. Three models:



(i) Interaction Model

Deals with **how processes interact (messages), communication performance, and time**.

- **Performance of communication channels:**
- **Latency** = delay between start of transmission at sender and start of receipt at receiver (network delay + OS delay + queueing).
- **Bandwidth** = total volume of data transmittable per unit time.
- **Jitter** = variation in time to deliver a series of messages (important for audio/video).
- Transmission time ≈ **latency + (message length / bandwidth)**.
- **Computer clocks & timing:** each process has own clock; **clock drift rate** = how fast clock differs from perfect reference; no global time.
- **Two variants of the interaction model:**

Synchronous DS	Asynchronous DS
Known bounds on: process execution time, message delivery time, clock drift rate	**No** bounds on any of these
Timeouts can detect failures reliably	Cannot know if process is slow or crashed
Easier to design algorithms	Realistic (Internet); harder
e.g., real-time embedded systems	e.g., the Internet

- **Event ordering:** Because there's no global clock, we order events by **causality** – logical clocks (Unit-II).

(ii) Failure Model

Defines **how failures can occur** so we can understand their effects.

Class	Sub-type	Description
Omission failures	Process omission (crash)	Process stops and never resumes (fail-stop if others can detect it)
	Channel omission	Message lost: send-omission, receive-omission, channel omission (buffer overflow, corruption)
Arbitrary (Byzantine) failures	–	Process/channel behaves arbitrarily: wrong values, sends contradicting messages, corrupts messages – worst kind
Timing failures	Clock, performance	Response takes longer than allowed (only in synchronous systems)

- **Masking failures:** hide failure or convert to a more acceptable kind (e.g., checksum turns corruption into omission; replication).
- **Reliable communication** = **validity** (every message sent is eventually delivered) + **integrity** (message received is identical to that sent and no duplicates).

(iii) Security Model

Ensures **secure processes and channels** and protects objects. - **Objects & principals:** resources have *access rights*; users/servers = *principals*. - **Threats:** to processes (spoofing/forging identity), to channels (eavesdropping, tampering, replay), **denial of service**, mobile code threats. - **Defences:** cryptography (encryption), **authentication**, **access control**, secure channels (SSL/TLS), digital signatures. - The "enemy" can send/copy/read/modify messages.

Comparison: Architectural vs Fundamental

Architectural	Fundamental
Structure/organisation of components	Abstract properties & assumptions
Concrete (client-server, P2P…)	Abstract (interaction, failure, security)
Answers "how is it built?"	Answers "how does it behave / what can go wrong?"

🧠 Remember This

> Architectural = **who is placed where** (CS, P2P, proxy, mobile code). Fundamental = **Interaction (sync/async, latency, drift) + Failure (omission, arbitrary, timing) + Security (threats, crypto)**.

---

6. NETWORKING AND INTERNETWORKING 📘📝 (A1-2b)

Definition

- **Network:** Set of interconnected devices that exchange data using communication protocols.
- **Internetwork (internet):** A **network of networks** – several different networks connected by **routers/gateways** so that they behave as a single logical network (e.g., the Internet).

Role in Distributed Systems

Distributed systems rely on networks to carry **messages** between components. Networks give **connectivity** (reach), while protocols give **rules** (format, order, error control) so heterogeneous machines can talk.

Types of networks (➕ by scale)

Type	Range	Example	Speed/Latency
PAN	few metres	Bluetooth	–
LAN	building/campus	Ethernet, Wi-Fi	High speed, low latency

Type	Range	Example	Speed/Latency
MAN	city	Cable TV / metro Ethernet	Medium
WAN	country/world	Internet backbone	Slower, higher latency
Wireless (WLAN/WMAN/WWAN)	varied	Wi-Fi, 4G/5G	Lossy, mobile

Key networking concepts (➕)

- **Packet switching:** data broken into packets, each routed independently (vs circuit switching).
- **Protocol:** agreed rules for communication. **Protocol suite/stack:** layered set of protocols.
- **Layering** – OSI 7 layers: Physical, Data link, Network, Transport, Session, Presentation, Application. **TCP/IP** (4 layers): Link, Internet (IP), Transport (TCP/UDP), Application (HTTP, FTP, SMTP, DNS).

```
Application layer   HTTP FTP SMTP DNS      ← messages
Transport layer     TCP / UDP              ← segments (end-to-end, ports)
Internet layer      IP (addressing/routing)← packets
Link/Physical       Ethernet, Wi-Fi        ← frames/bits
```

- **TCP vs UDP:** | TCP | UDP | |---|---| | Connection-oriented, reliable, ordered | Connectionless, unreliable, no ordering | | Flow & congestion control | None | | Slower, more overhead | Faster, low overhead | | Web, e-mail, file transfer | DNS, video/voice streaming, games |
- **IP addressing & routing:** each host has IP address; **routers** forward packets between networks; **gateways** connect networks using different protocols; **switches/bridges** connect within LAN.
- **Internetworking devices:** repeater (L1), hub, bridge/switch (L2), **router** (L3), gateway (all layers).
- **Performance parameters: latency, bandwidth, jitter**, reliability, scalability, security, mobility, **QoS**.

Internetworking diagram

```
LAN-1 (Ethernet)          LAN-2 (Wi-Fi)
[PC]─[PC]─┐                ┌─[Laptop]
          [Router]═WAN═[Router]
[Server]──┘                └─[Phone]
```

Exam points

- Write: network → internet → protocols → layering → TCP/UDP → link to DS (messages, latency, failures).
- Networks are **not reliable, not secure, have latency, bandwidth limits, are heterogeneous** – the **fallacies of distributed computing** ➕.

🧠 Remember This

> Internetwork = network of networks joined by routers. Protocol layers give uniform communication. TCP = reliable, UDP = fast.

---

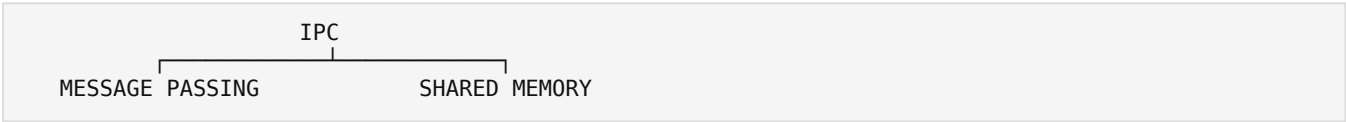
7. INTERPROCESS COMMUNICATION (IPC) 📘📝 (A1-3a)

Definition

IPC = mechanisms that allow **processes to exchange data and synchronise** their actions, whether on one computer or on different computers.

Simple (🗣️)

Do processes ko baat karni hai – ya to **chitthi (message)** bhejo, ya **ek common notice-board (shared memory)** use karo.



7.1 Message Passing

Definition: Processes communicate by **explicitly sending and receiving messages** via a communication channel (no shared variables). Two primitives: `send(destination, message)` and `receive(source, message)`.

Concepts ➕ (Coulouris ch. 4)

Concept	Options
Synchronous (blocking)	Sender blocks until receiver has received; receiver blocks until message arrives
Asynchronous (non-blocking)	Sender continues after copying message to buffer; receive may be blocking or non-blocking (polling/interrupt)
Reliability	Reliable channel (TCP) vs unreliable (UDP)
Ordering	FIFO ordering
Addressing	Sockets = (IP address, port); destination via port
Direct/Indirect	Direct: to process; Indirect: via **mailbox/message queue**
Buffering	Zero, bounded or unbounded buffer

Communication styles/mechanisms: - **Sockets:** endpoint for communication – **UDP datagram sockets** (`DatagramSocket`) and **TCP stream sockets** (`Socket`, `ServerSocket`). -

Message queues, MPI ➕ (parallel), **request–reply protocol**, RPC/RMI (higher-level). - **Marshalling/unmarshalling** (➕): converting data structures to a flat sequence of bytes (external data representation: CORBA CDR, Java serialization, XML, JSON, Protocol Buffers) so heterogeneous machines can understand.

Diagram

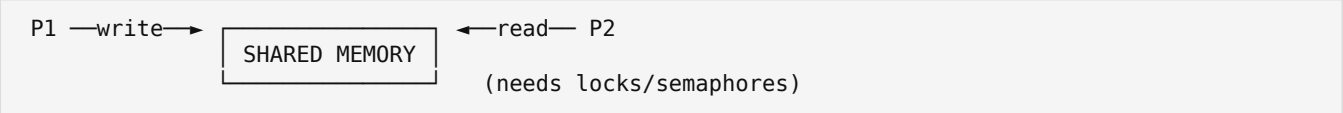


Advantages: works between different computers, no shared memory needed, clear synchronisation, portable, secure/isolated, suitable for distributed systems. **Disadvantages:** slower (network/copy/marshalling), programmer handles message format & failures, overhead of system calls. **Examples:** sockets, e-mail, chat, MPI, RabbitMQ/Kafka.

7.2 Shared Memory

Definition: Processes communicate by **reading and writing a common memory region** that both can access. Needs **synchronisation** (locks/semaphores) to avoid race conditions.

- On a **single machine:** OS provides shared memory segments (`shmget`, `mmap`), threads share address space.
- In a **distributed system:** no physical shared memory → emulate it via **Distributed Shared Memory (DSM)** ➕ – a software layer giving an illusion of one shared address space using page faults & message passing underneath.



Advantages: very **fast** (no copy after set-up), easy for large data, programming simple (like normal variables). **Disadvantages:** only on same machine (or emulated with DSM), **race conditions / need synchronisation**, poor security/isolation, DSM has coherence issues & false sharing, less scalable. **Examples:** threads in one process, OS shared memory (POSIX), DSM systems (Ivy, TreadMarks).

7.3 COMPARISON: Message Passing vs Shared Memory 📝

Point	Message Passing	Shared Memory
Communication	Explicit `send`/`receive`	Read/write to common variables
Location	Same or **different computers**	Same computer (or DSM emulation)
Speed	Slower (copying, network, system calls)	Faster (direct memory access)
Synchronisation	Implicit in blocking send/receive	Explicit (locks, semaphores, monitors)
Data transfer	Copied between address spaces	No copy, data shared in place
Complexity	Programmer handles message format/failures	Programmer handles race conditions

Point	Message Passing	Shared Memory
Scalability	**High** (suitable for distributed)	Low (limited to few processors)
Fault isolation/Security	Better isolation	Poor: one bug corrupts shared data
Suitable for	Distributed systems, loosely coupled	Multiprocessors, tightly coupled
Implementation	Sockets, MPI, queues, RPC/RMI	shm, mmap, threads, DSM
Examples	Chat, client-server	Multi-threaded app, producer–consumer buffer

Exam points

- Concluding line: "Distributed systems use **message passing** because there is no physical shared memory among nodes."
- Also mention: **request–reply protocol** is built on message passing and underlies RPC/RMI.

🧠 Remember This

> Message passing = explicit send/receive, works over network, slower, scalable. Shared memory = common memory, fast, same machine, needs locks.

---

8. DISTRIBUTED OBJECTS 📘📝 (A1-3b)

Definition

A **distributed object** is an object (data + methods) whose **methods can be invoked by processes located on other computers**, as if it were local. The **object model** is extended to work across the network.

Simple (🗣️)

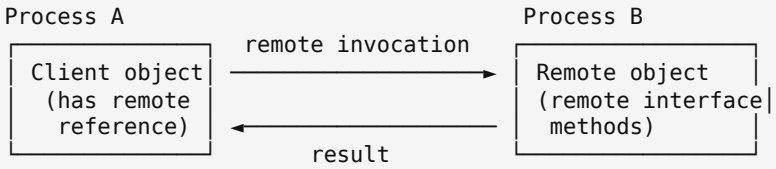
Object ek computer pe hai, uske method ko dusre computer se call karna – bilkul local method jaisa.

Key terms

Term	Meaning
Remote object	Object whose methods can be invoked from another process
Remote object reference	Identifier (ID) used to refer to a remote object: internet address + port + time + object number + interface; passed as argument/result
Remote interface	Specifies which methods of a remote object can be invoked remotely (like a contract); defined in IDL (CORBA) or Java interface extending `Remote`

Term	Meaning
Local vs remote invocation	Remote: arguments/results **marshalled**; failures possible; different semantics (pass by value for objects, by reference for remote objects)
Actions	Invocation may cause state change or invocation of further objects (chain of invocations)
Exceptions	Extra `RemoteException` for communication failures
Garbage collection	Automatic deletion of objects no longer referenced remotely (reference counting / leases)

Distributed Object Model – diagram



Advantages

Object-oriented benefits (encapsulation, reuse), location transparency, easier design of complex distributed apps, interoperability (CORBA).

Disadvantages

Performance overhead, complexity of failures, garbage collection, tight coupling to language (Java RMI) or heavy middleware (CORBA).

Systems ➕: Java RMI, CORBA, DCOM, .NET Remoting.

🧠 Remember This

> Distributed object = object usable from other computers through **remote reference + remote interface**.

---

9. REMOTE METHOD INVOCATION (RMI) 📘📝 (A1-3b, A1-1c)

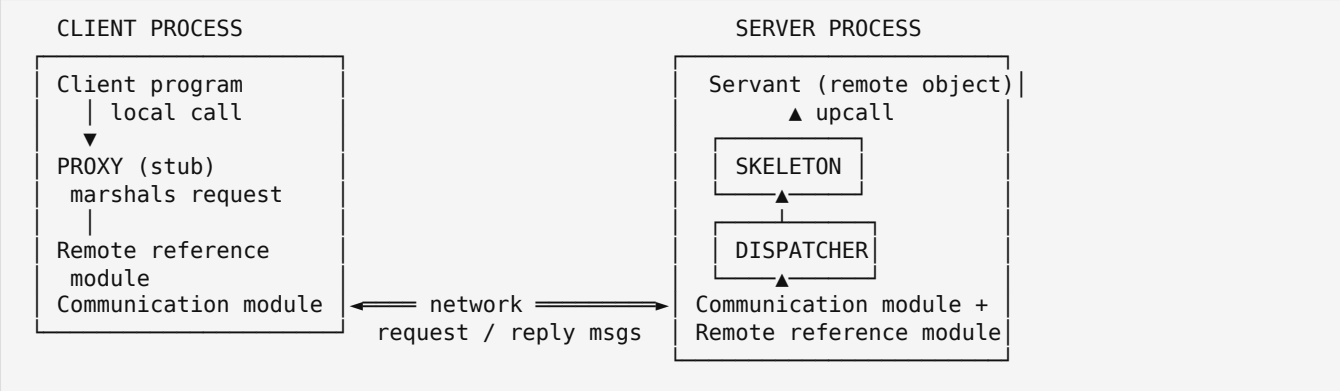
Definition

RMI allows an object running in one JVM/process to **invoke methods on an object in another process** (usually on another machine) using the same syntax as a local call – it is the **object-oriented version of RPC**.

Simple (🗣️)

`result = remoteObj.add(5,3);` – dikhta local call hai, par execute dusre computer pe hota hai. Beech ka kaam **stub/proxy** karta hai.

RMI software architecture (Coulouris 4th ed.)

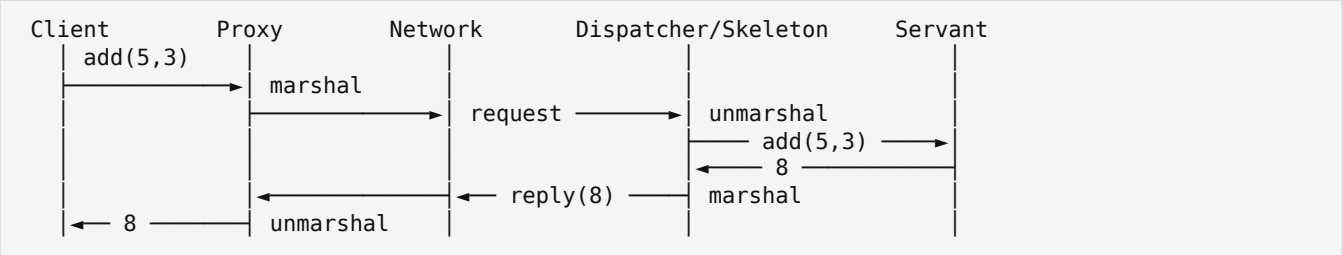


Component	Role
Proxy (Stub)	Local representative of remote object at client; **marshals** method id + arguments into request, sends it, waits, **unmarshals** result; hides remote nature (transparency)
Communication module	Implements request–reply protocol between client & server; uses invocation semantics
Remote reference module	Translates between local and remote object references; maintains **remote object table**
Dispatcher	Server side; receives request, selects the method (using method id) and passes to skeleton; one per class of remote object
Skeleton	Unmarshals arguments, calls the method on the servant, **marshals** result/exception and returns
Servant	The actual implementation object that does the work
Binder / Registry	Naming service mapping textual names to remote references (RMI registry)

Working – steps of a remote method call

1. Client calls method on **proxy** (looks local).
2. Proxy **marshals** (method id + parameters) into a message.
3. **Communication module** sends the request message to the server (using TCP/IP).
4. Server's communication module receives it; **remote reference module** finds the target object.
5. **Dispatcher** selects the correct skeleton/method.
6. **Skeleton unmarshals** arguments and **invokes** the servant method.
7. Servant returns result → skeleton **marshals** the result.
8. Reply sent to client; proxy **unmarshals** and returns result to the caller.

Sequence diagram



Invocation semantics ➕ (fault tolerance measures)

Semantics	Retransmit request?	Filter duplicates?	Re-execute/ retransmit reply	Result
Maybe	No	–	–	Method executed once or not at all (no guarantee)
At-least-once	Yes	No	Re-execute procedure	Executed ≥ 1 times; OK for **idempotent** ops
At-most-once	Yes	Yes	Retransmit old reply	Executed exactly once or not at all; **Java RMI** provides this

Advantages

Object-oriented, transparency, easy programming, type checking, passes objects (serialisation), garbage collection, secure (Java security manager).

Disadvantages

Slower than local call, **Java-only** (Java RMI), firewall issues, failures more complex, parameter passing overhead.

Exam points

- Draw the proxy–skeleton diagram; list 8 steps.
- Mention remote reference passing and parameter passing: **objects by value (serialised), remote objects by reference**.

🧠 Remember This

> RMI = call object's method remotely. Client → **stub/proxy** → network → **skeleton/dispatcher** → **servant** → result back.

---

10. RPC — REMOTE PROCEDURE CALL 📘

Definition

RPC lets a program **call a procedure that runs in another address space (another computer)** as though it were a local procedure call; the network details are hidden (Birrell & Nelson, 1984).

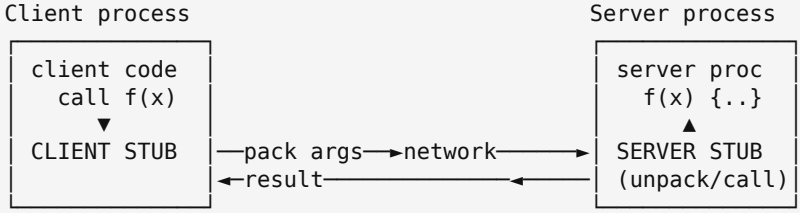
Simple (🗣️)

Function call jaisa likho, execution dusre computer pe. Beech me **client stub** aur **server stub** sab sambhal lete hain.

Components

Component	Role
Client	Calls the procedure
Client stub	Marshals (packs) parameters into message; unpacks result
RPC runtime / communication	Sends/receives messages, retransmission, timeouts
Server stub (skeleton)	Unpacks parameters; calls actual procedure; packs result
Server procedure	The real function
IDL	Interface Definition Language – describes procedure signatures; **IDL compiler generates stubs**
Binder/Portmapper	Locates server (name service)
XDR / data representation	Common format for heterogeneous machines

Diagram



Working – steps

1. Client calls local **client stub** (normal call).
2. Stub **marshals parameters** into message.
3. Message sent to server (via RPC runtime, OS, network).
4. **Server stub unmarshals** the parameters.
5. Server stub calls the **actual procedure**.
6. Procedure executes and returns result to server stub.
7. Server stub **marshals** result and sends back.
8. Client stub **unmarshals** the result and returns to the client program.

Issues in RPC ➕

Parameter passing (**no pointers/references**: pass by value/copy-restore), data representation (endianness, XDR), binding, failures (lost request/reply, server/client crash, orphan), **semantics** (maybe, at-least-once, at-most-once), performance, security, no shared global variables.

Advantages

Simple & familiar programming model, hides network, language-independent via IDL, modular/reusable, widely used (Sun RPC/ONC RPC, gRPC).

Disadvantages

Slower than local calls, failure handling is complex (partial failure), no pointers, tight coupling, less flexible than messaging, security concerns.

Examples ➕: Sun/ONC RPC (used by NFS), DCE RPC, gRPC, XML-RPC, Apache Thrift.

10.5 COMPARISON: RMI vs RPC 📝

Feature	RPC	RMI
Paradigm	**Procedural** (functions)	**Object-oriented** (methods of objects)
What is invoked	Remote procedure/function	Method of a remote object
Parameters	Ordinary data structures (by value); **no object passing**, no pointers	Can pass **objects** (serialised, by value) and **remote references**
Language	Mostly language-independent (C, via IDL/XDR)	Java RMI is **Java-specific** (CORBA IIOP is multi-language)
Stubs	Client stub / server stub	Stub (proxy) / skeleton
Interface definition	IDL (e.g., rpcgen)	Java interface extending `Remote`
Object identity	Not applicable	Remote object references, **garbage collection**
Polymorphism/inheritance	Not supported	Supported (dynamic loading of classes)
Complexity/overhead	Simpler, lighter	Heavier (objects, serialisation, registry)
Naming	Portmapper / binder	RMI Registry / naming service
Typical use	System services (NFS)	Enterprise Java apps

Exam point: "RMI is the OO counterpart of RPC: both hide remote communication behind a local call using stubs and marshalling; RMI adds object references, object parameters and garbage collection."

🧠 Remember This

> RPC = call **function** remotely (procedural). RMI = call **method of object** remotely (OO). Both use stub + marshalling.

---

11. EVENTS AND NOTIFICATIONS 📘

Definition

- **Event:** an occurrence of interest at an object (e.g., button click, file modified, price change, sensor reading).
- **Notification:** a message/object containing information about an event, sent to interested parties.
- **Publish–Subscribe (distributed event-based system):** *publishers* generate events; *subscribers* register interest; an **event service** delivers notifications **asynchronously** to all interested subscribers.

Simple (🗣️)

YouTube channel: creator video upload karta hai (event), jinhone **subscribe** kiya hai unko **notification** milti hai. Subscriber ko baar-baar poochna nahi padta.

Roles in the model (Coulouris)

Role	Function
Event generator / Publisher	Object of interest that changes state and generates events
Event type	Kind of event, with attributes
Subscriber / Observer / Listener	Registers interest; receives notification
Event service (broker)	Middleman that stores subscriptions, filters, and delivers notifications
Observer (in the Coulouris sense)	Object that processes notifications: forwards, filters, patterns, logs (decouples subscriber from publisher)

Diagram



Characteristics

- **Heterogeneous** (different systems interoperate), **asynchronous**, **decoupled** in space, time (and sync) – publisher doesn't know subscribers.
- Types of subscription: **channel/topic-based, content-based, type-based**.

Working steps

1. Subscribers register (subscribe) for event types with the event service.
2. Publisher publishes an event/notification.
3. Event service matches it with subscriptions.
4. Notification delivered to all matching subscribers.
5. Subscribers act; can unsubscribe.

Architecture/Delivery issues ➕: reliability (guaranteed delivery), ordering, scalability, filtering, security, event persistence (e.g., Kafka, MQTT).

Advantages: loose coupling, scalable, asynchronous, easy to add subscribers, real-time updates.

Disadvantages: delivery/ordering hard to guarantee, hard to debug, broker may be bottleneck/SPOF, security/privacy.

Examples: stock ticker, push notifications, Java `EventListener`, RabbitMQ/Kafka/MQTT, Jini distributed events, GUI events, Facebook notifications, IoT sensors.

🧠 Remember This

> Publisher → **Event Service** → Subscribers. Asynchronous, loosely-coupled notifications about events.

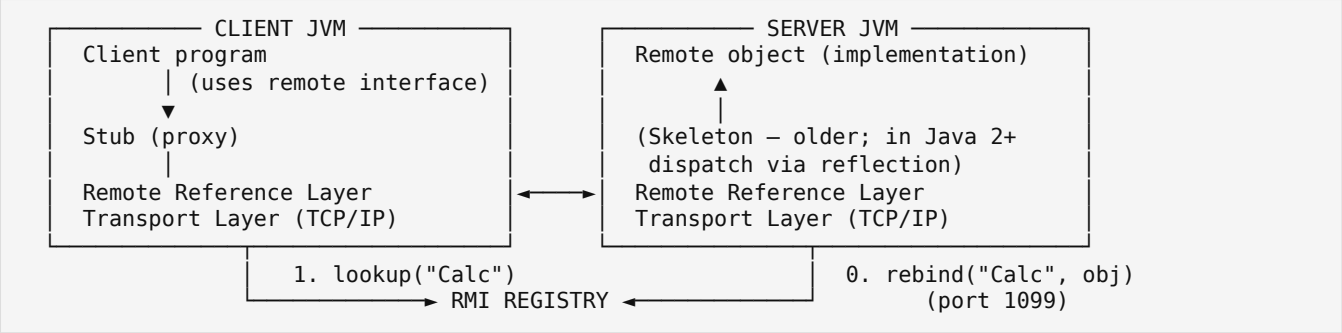
---

12. CASE STUDY: JAVA RMI 📘📝 (A1-1c – 3 marks, the biggest Unit-I question)

12.1 What is Java RMI?

Java RMI is Java's mechanism (package `java.rmi`) for **invoking methods of an object residing in another JVM** (same or different machine). It follows the distributed-object model, is **Java-to-Java**, supports **at-most-once** semantics, serialisation for object passing, and distributed garbage collection.

12.2 Architecture



Three layers: (1) **Stub/Skeleton layer** – intercepts calls, marshals; (2) **Remote Reference Layer** – manages references & invocation semantics; (3) **Transport layer** – TCP connections between JVMs.

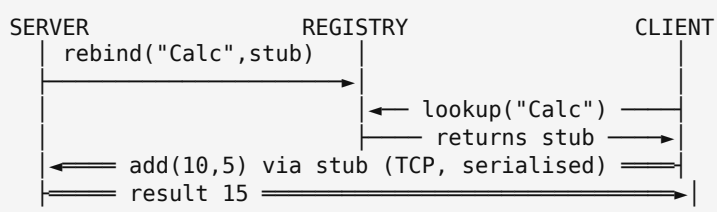
12.3 Components

Component	Explanation
Remote interface	A Java interface that **extends** `java.rmi.Remote`; each method **declares** `throws RemoteException`. Defines what client can call.
Remote object (implementation)	Class implementing the remote interface; usually **extends** `UnicastRemoteObject` (exports the object so it can receive calls). Contains actual logic.
Stub (proxy)	Client-side representative; implements same remote interface; forwards call to server. Since Java 5 generated **dynamically** (no `rmic` needed); earlier created with `rmic`.
Skeleton	Server-side entity that receives call and invokes method; **obsolete since Java 1.2** (handled by RMI runtime). Mention for exam completeness.
RMI Registry	Simple name service (`rmiregistry`, default **port 1099**) mapping name → stub. Server **binds/rebinds**; client **looks up**.
Server	Creates remote object, exports it, registers in registry.
Client	Looks up name in registry, gets stub, calls methods.
Serialization	Objects passed as parameters must implement `java.io.Serializable`.

12.4 Complete execution flow (step-by-step)

1. **Define** remote interface (`Calculator extends Remote`).
2. **Implement** it (`CalculatorImpl extends UnicastRemoteObject`).
3. **Server** starts: creates object → **exports** (stub generated) → starts/gets registry → **`rebind("CalculatorService", obj)`**.
4. **Client** starts: contacts registry host:1099 → **`lookup("CalculatorService")`** → receives **stub**.
5. Client **calls** `stub.add(10,5)`.
6. Stub **marshals (serialises)** method + arguments → sends over TCP.
7. Server-side RMI runtime **unmarshals**, invokes the method on the **real object**.
8. Result **marshalled** and returned; stub **unmarshals** and gives value to client.

9. Client prints result. Unused objects are **garbage-collected** through leases/reference counting.



12.5 Simple Calculator example

Step 1 – Remote interface `Calculator.java`

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    double divide(int a, int b) throws RemoteException;
}
```

Step 2 – Implementation (remote object) `CalculatorImpl.java`

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    public CalculatorImpl() throws RemoteException { super(); }

    public int add(int a, int b)      { return a + b; }
    public int subtract(int a, int b) { return a - b; }
    public int multiply(int a, int b) { return a * b; }
    public double divide(int a, int b) { return (double) a / b; }
}
```

Step 3 – Server `CalculatorServer.java`

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            Calculator calc = new CalculatorImpl();          // create remote object
            Registry registry = LocateRegistry.createRegistry(1099); // start registry
            registry.rebind("CalculatorService", calc);       // register name -> stub
            System.out.println("Calculator Server is ready...");
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Step 4 – Client `CalculatorClient.java`

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            Calculator calc = (Calculator) registry.lookup("CalculatorService");
            System.out.println("10 + 5 = " + calc.add(10, 5));
            System.out.println("10 - 5 = " + calc.subtract(10, 5));
            System.out.println("10 * 5 = " + calc.multiply(10, 5));
            System.out.println("10 / 5 = " + calc.divide(10, 5));
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Step 5 – Compile & run

```
javac *.java
java CalculatorServer        (terminal 1)
java CalculatorClient        (terminal 2)
```

Sample output

```
Server terminal :  Calculator Server is ready...
Client terminal :  10 + 5 = 15
                   10 - 5 = 5
                   10 * 5 = 50
                   10 / 5 = 2.0
```

> Notes: (i) `Naming.rebind("rmi://localhost/CalculatorService", calc)` is an alternative. (ii) With old Java (< 5) run `rmic CalculatorImpl` to generate stub. (iii) The server JVM stays alive because the exported object keeps it running.

12.6 Advantages / limitations of Java RMI

Advantages	Limitations
Simple, pure Java, object-oriented	Java-only (no cross-language)
Automatic stub, serialisation, distributed GC	Slower than local call / heavier than sockets
Dynamic class loading, security manager	Firewall issues (dynamic ports)
Type-safe, passes objects	Registry is simple (no advanced naming)
At-most-once semantics	Objects must be serializable

12.7 Viva questions – Java RMI

1. **What is Java RMI?** Java API to call methods of objects in another JVM as if local.
2. **Which interface must a remote interface extend?** `java.rmi.Remote`.
3. **Why does every remote method throw `RemoteException`?** Network can fail; caller must handle communication errors.
4. **Role of the stub?** Client-side proxy; marshals calls and sends them to server.
5. **What is the RMI registry and its default port?** Naming service, port 1099.

6. **Difference between `bind` and `rebind`?** `bind` throws `AlreadyBoundException` if name exists; `rebind` overwrites.
7. **What does `UnicastRemoteObject` do?** Exports the object (creates stub, listens on port) for remote calls.
8. **How are parameters passed?** Primitives/objects by value (serialised); remote objects by reference (stub).
9. **What is a skeleton?** Server-side dispatcher of older RMI; not needed since Java 1.2.
10. **Which invocation semantics does Java RMI give?** At-most-once.
11. **What is marshalling?** Converting arguments to a byte stream for transmission.
12. **How is a remote object garbage collected?** Distributed GC using reference counting & leases.

🧠 Remember This

> Java RMI = Remote interface (extends Remote) + Impl (extends UnicastRemoteObject) + Server (rebind in registry) + Client (lookup → stub → call).

(End of Unit-I. Next: `02_Unit2_Notes.md`)

UNIT-II — SYNCHRONIZATION (Complete Notes)

Labels: 📘 official syllabus · 📝 assignment-emphasized · ➕ additional/exam-oriented

---

1. INTRODUCTION TO SYNCHRONIZATION 📘📝 (A2-1a)

Definition

Synchronization in a distributed system means **coordinating the actions of independent processes** so that they agree on **time/order of events, access shared resources safely, and reach common decisions**, even though there is **no shared memory and no global clock**.

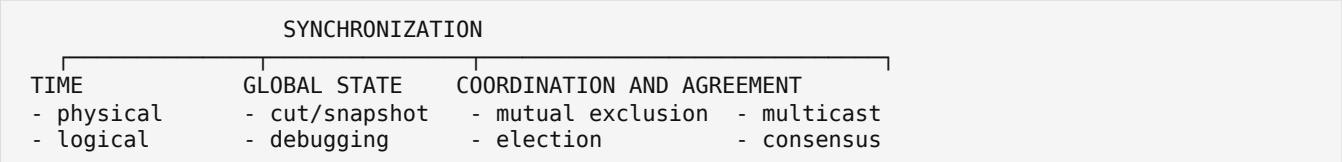
Simple (🗣️)

Alag-alag computers ko ek "team" ki tarah chalana: kaun pehle, kaun baad me, kaun resource use karega, sab kis baat pe agree honge – ye sab synchronization hai.

Why is it needed? (Problems caused by distribution)

Problem	Reason	Solution in this unit
No global clock	each node has own clock, drift	Physical clock sync (Cristian/Berkeley/NTP), Logical clocks
Event ordering	messages have delay	Happened-before, Lamport/Vector clocks
No global state	no node sees whole system at once	Global states, snapshot
Concurrent access to a resource	many processes want the same resource	Distributed mutual exclusion
Need for a leader	someone must coordinate	Election algorithms
Group communication	send to many reliably/ordered	Multicast
Agreement despite faults	processes may crash/lie	Consensus

Map of Unit-II



Exam points

- Answer to "Explain synchronization" (2 marks): definition + 3 reasons (no global clock, concurrency, independent failures) + mention events/process states.

🧠 Remember This

> Synchronization = agree on **time/order**, **shared resources**, **leader**, **state**, **decision** — without shared memory or clock.

---

2. TIME AND GLOBAL STATES – INTRODUCTION 📘

- Each computer has a **hardware clock** (oscillator + counter) → software clock $C(t) = \alpha \cdot H(t) + \beta$.
- Time is needed for: **timestamps** (e-commerce, banking), **auditing/logging**, **authentication (Kerberos)**, **consistency (make)**, **ordering events**.
- Since perfect synchronization is impossible, DS uses (a) **Synchronized physical clocks** (approximate real time), (b) **Logical clocks** (only order matters), (c) **Global state** notion (what is the state "at some time").
- **Key idea:** In DS we often care only about **order of events (causality)**, not exact real time.

```
Real time t ──────────────────────────────►
Process P1 clock: 10:00:00 10:00:03 ...
Process P2 clock: 10:00:02 10:00:05 ...   ← different! (skew)
```

🧠 Remember This

> Time can be **physical** (real, needs sync) or **logical** (order only).

---

3. EVENTS AND PROCESS STATES 📘📝 (A2-1a)

Definitions

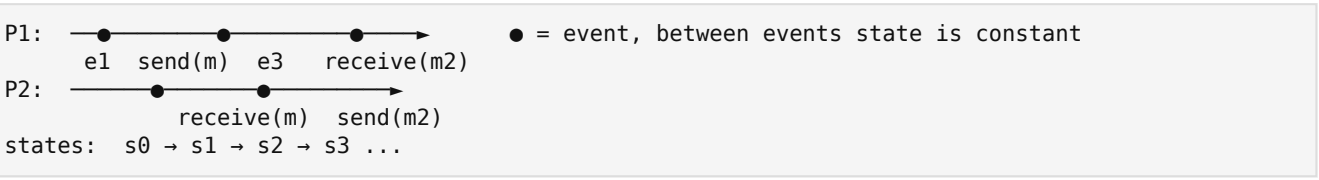
- **Process:** sequence of actions; a distributed system = set of processes p1..pN communicating **only** by messages.
- **Process state:** the values of all variables of the process (registers, memory, program counter, etc.) at a given moment.
- **Event:** an **action** that changes the process state. Three kinds: 1. **Internal (local) event** – computation changes local variable. 2. **Send event** – process sends a message. 3. **Receive event** – process receives a message.
- **History of a process:** ordered sequence of its events: `history(pi) = <e0, e1, e2, …>` (events totally ordered within one process).

- **Local state s_i^k:** state of pi just before event e_k (i.e., after k events).

Simple (🗣️)

Process ek state machine hai: har event (compute/send/receive) use ek state se dusri state me le jata hai.

Timeline diagram



Role of events and process states in synchronization

1. **Ordering events** – within a process events are totally ordered; across processes only via messages (send happens before receive).
2. **Global state** = collection of process states + channel states; consistency depends on which events have occurred.
3. **Detecting properties** (deadlock, termination) & **debugging** need to know sequences of states.
4. **Logical clocks** timestamp events; **state recording** takes snapshots of states.

Exam points

- Draw a timeline with two processes and one message; label send/receive; state clearly "events change process state".

🧠 Remember This

> State = data of process; **event** (internal/send/receive) changes the state; history = ordered list of events.

---

4. HAPPENED-BEFORE RELATION (→) 📘📝 (A2-1b)

Definition (Lamport, 1978)

The **happened-before** relation `a → b` is the **smallest relation** satisfying: 1. **HB1 (same process):** if a and b are events in the same process and a occurs before b, then `a → b`. 2. **HB2 (message):** if a = **send(m)** and b = **receive(m)**, then `a → b`. 3. **HB3 (transitivity):** if `a → b` and `b → c`, then `a → c`.

Concurrent events: if neither `a → b` nor `b → a`, then `a || b` (concurrent).

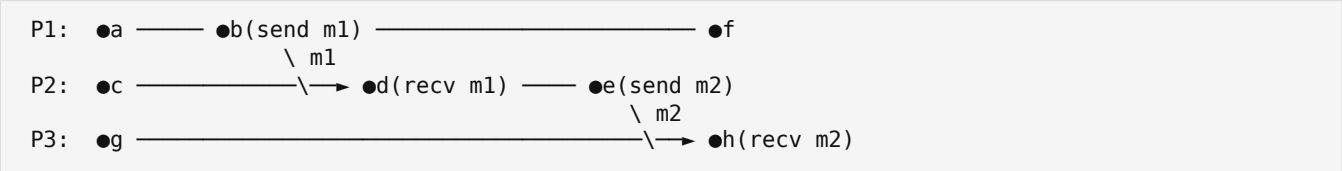
Simple (🗣️)

`a → b` ka matlab: a **cause kar sakta hai** b ko (ya a pehle ho chuka tha jiski khabar b tak pahunch sakti hai). Agar koi information-path nahi, to events **concurrent** hain – unka order decide nahi ho sakta (aur zarurat bhi nahi).

Properties

- **Irreflexive** (a ↛ a), **transitive**, **antisymmetric** → a **partial order** (not total).
- It captures **potential causality**, not real time.

Example



- a → b (same process), b → d (message m1), d → e, e → h (message m2).
- By transitivity: a → d, b → e, **b → h**, a → h.
- **Concurrent:** c ∥ a? c is in P2 before d; nothing links a and c → **a ∥ c**. Also f ∥ d (no path).

Exam points

- Always list the 3 rules. State: "→ gives partial order, so some events are concurrent".

🧠 Remember This

> HB: same-process order + send→receive + transitivity. No path = concurrent (∥).

---

5. LOGICAL CLOCKS — LAMPORT LOGICAL CLOCK 📘📝 (A2-1b)

5.1 Why logical clocks?

Physical clocks can't be perfectly synchronised, and we usually care about **order** not real time. A **logical clock** assigns **numbers (timestamps)** to events in a way that respects causality.

Definition

A **Lamport logical clock** is a **monotonically increasing software counter** `L_i` at each process `p_i`, used to timestamp events such that: **if a → b then L(a) < L(b)** (clock condition).

5.2 Rules (Lamport's implementation rules)

- **LC1 (before every event):** `L_i := L_i + 1` (increment before executing each local or send event).
- **LC2 (send):** when `p_i` sends message m, it first applies LC1, then attaches `t = L_i` to m.
- **LC3 (receive):** when `p_j` receives (m, t): `L_j := max(L_j, t)`, then apply LC1: `L_j := L_j + 1`, then timestamp the receive event.

(Equivalent common form: on receive, `L_j = max(L_j, t) + 1`.)

5.3 Algorithm (pseudocode)

```
Initially:   L_i = 0

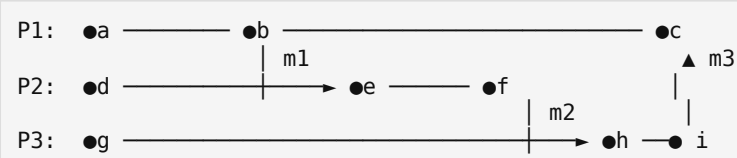
On internal event:      L_i = L_i + 1
On send(m) to p_j:      L_i = L_i + 1
                        send (m, L_i)
On receive (m, t):      L_i = max(L_i, t) + 1
```

5.4 Properties

- If `a → b` then `L(a) < L(b)`.
- **Converse is NOT true:** `L(a) < L(b)` does **not** imply `a → b` (they might be concurrent).
- Cannot detect concurrency. (Vector clocks can.)
- **Total ordering** (➕): break ties by process id: (L(a), pid(a)) < (L(b), pid(b)). Used in mutual exclusion (Lamport / Ricart-Agrawala).

5.5 Worked example 1 – step-by-step

Events: P1: a, b (send m1 to P2), c (receive m3 from P3). P2: d, e (receive m1), f (send m2 to P3). P3: g, h (receive m2), i (send m3 to P1).



Calculation (all clocks start at 0): | Event | Process | Type | Calculation | L | |---|---|---|---|---| | a | P1 | internal | 0+1 | **1** | | b | P1 | send m1 | 1+1 | **2** (m1 carries 2) | | d | P2 | internal | 0+1 | **1** | | e | P2 | receive m1 (t=2) | max(1,2)+1 | **3** | | f | P2 | send m2 | 3+1 | **4** (m2 carries 4) | | g | P3 | internal | 0+1 | **1** | | h | P3 | receive m2 (t=4) | max(1,4)+1 | **5** | | i | P3 | send m3 | 5+1 | **6** (m3 carries 6) | | c | P1 | receive m3 (t=6) | max(2,6)+1 | **7** |

Timeline with timestamps:



Observations: `b → e` and 2 < 3 ✓. `a` (1) and `d` (1) are concurrent and have equal timestamps → break tie with process id. `d(1) < b(2)` but d ∥ b → shows **converse false**.

5.6 Exam-style numerical questions (with answers)

Q1. Three processes P1, P2, P3 start with clock 0. P1 executes event a, then sends m to P2 (event b). P2 executes event c, then receives m (event d), then sends m' to P3 (event e). P3 executes f, then receives m' (event g). Find all Lamport timestamps. **Ans:** a=1, b=2 (m carries 2). P2: c=1, d=max(1,2)+1=3, e=4 (m' carries 4). P3: f=1, g=max(1,4)+1=5.

Q2. P1's clock is 10 and receives a message with timestamp 15. New value? **Ans:** max(10,15)+1 = **16**.

Q3. P2's clock is 20 and receives message with timestamp 12. **Ans:** max(20,12)+1 = **21**.

Q4. P1: events at clock 1,2,3 (3 = send to P2). P2 has local clock values 1,2 then receives message (ts 3), sends to P3, P3 receives. If P2's clock before receive = 2 and P3's clock before receive = 0, find receive timestamps. **Ans:** P2 receive = max(2,3)+1 = 4; P2 send = 5; P3 receive = max(0,5)+1 = 6.

Q5 (concurrency). Event x in P1 has L=5, event y in P2 has L=6, and no messages are exchanged. Can we say x → y? **Ans:** No. Lamport clocks only guarantee a→b ⇒ L(a)<L(b). x and y are concurrent.

Q6 (increment d=1). P1 clock 0: internal, internal, send; P2 clock 0: internal, receive (from P1's send). **Ans:** P1: 1,2,3 (send ts=3). P2: 1, then max(1,3)+1 = 4.

5.7 Advantages / limitations

Advantages	Limitations
Simple, small overhead (one integer)	Cannot detect concurrency
Preserves causality (a→b ⇒ L(a)<L(b))	L(a)<L(b) does not imply a→b
No physical clock needed	Timestamps are not real time
Basis for total ordering & mutual exclusion	Ties must be resolved with process id

🧠 Remember This

> Increment before every event; attach timestamp when sending; on receive: **max(local, msg) + 1**. a→b ⇒ L(a)<L(b) (not vice versa).

---

6. LOGICAL TIME – VECTOR CLOCKS ➕ (extension of logical clocks, often asked)

Definition

A **vector clock** at process p_i (system of N processes) is an **array V_i[1..N]** of integers. `V_i[i]` counts events at p_i; `V_i[j]` = p_i's knowledge of p_j's latest event count.

Rules

1. Initially V_i[j] = 0 for all j.
2. **Before each event** at p_i: `V_i[i] := V_i[i] + 1`.
3. **Send:** after step 2, attach the whole vector `t = V_i` to message.
4. **Receive (m, t):** `V_i[j] := max(V_i[j], t[j])` for all j; then `V_i[i] := V_i[i] + 1`.

Comparison of vectors

- `V = V'` iff all components equal.
- `V ≤ V'` iff **V[j] ≤ V'[j] for all j**.
- `V < V'` iff V ≤ V' and V ≠ V'.
- **a → b ⇔ V(a) < V(b)** (both directions!).
- **a ∥ b** iff neither V(a) < V(b) nor V(b) < V(a).

Example (same events as §5.5 with vector clocks, 3 processes)

Event	Calculation	Vector
a (P1 internal)		(1,0,0)
b (P1 send m1)		(2,0,0)
d (P2 internal)		(0,1,0)
e (P2 receive m1)	max((0,1,0),(2,0,0)) = (2,1,0) → +1 own	(2,2,0)
f (P2 send m2)	own+1	(2,3,0)
g (P3 internal)		(0,0,1)
h (P3 receive m2)	max((0,0,1),(2,3,0))=(2,3,1) → +1 own	(2,3,2)
i (P3 send m3)		(2,3,3)
c (P1 receive m3)	max((2,0,0),(2,3,3))=(2,3,3) → +1 own	(3,3,3)
- a (1,0,0) < e (2,2,0) ⇒ a → e ✓.		
- a (1,0,0) vs d (0,1,0): neither ≤ → **a ∥ d** (Lamport gave equal timestamps 1,1 and couldn't tell).		

Lamport vs Vector

Lamport	Vector
Single integer	Array of N integers
a→b ⇒ L(a)<L(b)	a→b ⇔ V(a)<V(b)
Cannot detect concurrency	**Detects concurrency**
O(1) space per message	O(N) space
Simple	Used for causal ordering, debugging, conflict detection (Dynamo, Riak)

🧠 Remember This

> Vector clock = one counter per process; compare component-wise; can tell **causal vs concurrent**.

---

7. SYNCHRONIZING PHYSICAL CLOCKS 📘📝 (A2-1c)

7.1 Why do physical clocks need synchronization?

- Each computer's clock is a **quartz crystal oscillator**; no two run at exactly the same rate.
- Needed for: **timestamps of transactions** (banking, e-commerce), **log/audit ordering**, **authentication tickets expiry (Kerberos)**, **distributed file timestamps (make, backup)**, **scheduling, real-time control**, **debugging**, **fair ordering of events**.
- Reference: **UTC** (Coordinated Universal Time) from atomic clocks (broadcast by GPS/radio).

7.2 Key terms (assignment-emphasized 📝)

Term	Definition	Example
Clock skew	**Difference between the readings of two clocks at the same instant**	C1=10:00:05, C2=10:00:02 → skew 3 s
Clock drift	**Rate at which a clock deviates from a perfect (reference) clock**; caused by crystal frequency variation, temperature, ageing	Typical crystal drift ≈ 10^{-6} s/s (1 µs per second ≈ **86.4 ms/day**)
Drift rate (ρ)	Max deviation per second, e.g., ρ = 10^{-6}	
Communication delay (latency)	Time taken for a message to travel; **variable** (queueing, routing) and **unknown** ⇒ received time value is already old	RTT 200 ms
Accuracy	How close to UTC	
Precision	How close clocks are to each other	

- **Relationship:** *drift causes skew to grow with time.* Two clocks with max drift ρ each can differ by up to `2ρ·Δt` after Δt.
- To keep skew ≤ δ, must resynchronise at least every **δ / 2ρ** seconds. *Example:* ρ = 10^{-6}, δ = 1 ms ⇒ resync interval ≤ 0.001/(2×10^{-6}) = **500 s**.

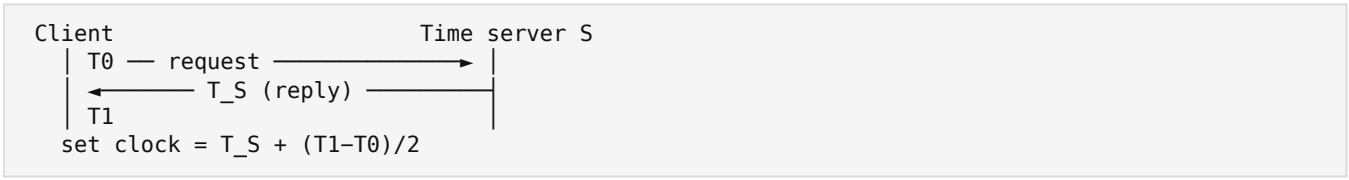
7.3 Problems/challenges with physical clocks

1. **Drift & skew** – clocks diverge continuously.
2. **Unpredictable communication delay** – time message is stale by the time it arrives; asymmetric paths.
3. **Clock cannot be set backwards** – would break time-ordering (e.g., `make`) ⇒ slow it down gradually instead.
4. **Faulty/malicious time servers** – wrong time source.
5. **No global clock** – impossible to get perfect sync.
6. **Scalability & cost** – many nodes, WAN links.
7. **Network failures** – can't reach time server.

7.4 Algorithms

(a) Cristian's Algorithm (external sync, passive time server) ➕

Purpose: sync client to a **time server** connected to a UTC source. **Assumption:** RTT is small vs required accuracy; delay roughly symmetric. **Steps:** 1. Client `p` sends request `mr` to server `S` and records **T0**. 2. S replies with its time **T_S** in message `mt`. 3. Client records reply arrival time **T1**; RTT = T1 − T0. 4. Client sets its clock to **T_S + RTT/2**. 5. Accuracy: ± (RTT/2 − min), where min = minimum one-way delay.



Numerical: T0 = 10:00:00.000, T1 = 10:00:00.800, T_S = 10:00:05.300 → RTT = 0.8 s → new time = 5.300 + 0.4 = **10:00:05.700**; accuracy ± 0.4 s (if min delay ≈ 0). **Improvements:** take several samples, use the one with **minimum RTT**. **Advantages:** simple, one message pair. **Limitations:** single point of failure (server), assumes symmetric delay, server may be faulty (fixed by using multiple servers).

(b) Berkeley Algorithm (internal sync, active master) ➕

Purpose: keep clocks of a group **mutually consistent** (no UTC source needed); average time. **Steps:** 1. **Master** (elected) polls all slaves: "what is your time?" 2. Slaves reply with their times; master compensates for RTT (Cristian-style). 3. Master computes the **average** (ignoring outliers/faulty clocks), including its own. 4. Master sends each slave the **adjustment (offset)** to apply — not the absolute time (avoids delay error). 5. Each slave adjusts its clock (gradually if it needs to go backwards).

```
        ┌──── Master 3:00 ────┐
  ask ▼                       ▼ ask
Slave1 2:50          Slave2 3:25
Master avg offset = (0 −10 +25)/3 = +5 → target 3:05
Send adjustments: Master +5, Slave1 +15, Slave2 −20
```

Numerical: as above → all clocks become **3:05**. **Advantages:** no UTC needed, tolerant to faulty clocks (discard outliers), master can be re-elected. **Limitations:** master failure/bottleneck, not very scalable, accuracy limited by RTT.

(c) NTP – Network Time Protocol ➕

- Internet-scale service; **hierarchy of servers in strata**: Stratum 0 = atomic/GPS clocks, Stratum 1 = servers directly connected, Stratum 2 syncs with Stratum 1, etc.
- Modes: **multicast** (LAN), **procedure-call** (Cristian-like), **symmetric** (highest accuracy).
- Uses timestamps `t0, t1, t2, t3` :
- **Offset θ = ((t1 − t0) + (t2 − t3)) / 2**
- **Delay δ = (t3 − t0) − (t2 − t1)**
- Pick the pair with **minimum delay**; accuracy ~ tens of ms over Internet, <1 ms LAN.
- Fault-tolerant (multiple servers, reconfigures), scalable (hierarchy), secure (authentication).

Comparison table

Feature	Cristian	Berkeley	NTP
Type	External	Internal	External (UTC)
Time source	Server with UTC	Average of group	Hierarchical servers with UTC
Master	Passive server	Active coordinator	Servers in strata
Best for	Small LAN	Group w/o UTC	Internet-wide
Fault handling	Weak	Ignores outliers	Strong

7.5 Logical Time vs Physical Time (see §8 for table).

🧠 Remember This

> **Skew** = difference between two clocks now; **drift** = rate of deviation; **delay** = uncertain message time. Cristian: T_S + RTT/2. Berkeley: average & send adjustments. NTP: stratum hierarchy.

---

8. LOGICAL TIME vs PHYSICAL TIME 📘

Basis	Physical Time	Logical Time
Meaning	Real-world time (seconds, UTC)	Counter representing **order** of events
Based on	Hardware clock/atomic clock	Events & message exchange (causality)

Basis	Physical Time	Logical Time
Purpose	Real time stamps, scheduling, timeouts	Ordering events, consistency
Synchronisation needed?	**Yes** (Cristian/Berkeley/NTP)	**No** external sync
Affected by drift/skew?	**Yes**	**No**
Accuracy	Approximate, limited by delay	Exact causal order (partial/total)
Examples	UTC, NTP time	Lamport clock, Vector clock
Relation to real time	Tied to it	Independent of it
Overhead	Periodic sync messages	Small extra timestamp in each message
Detect concurrency	No	Only vector clocks
Use case	Logs with real time, certificates	Mutual exclusion, causal broadcast, debugging

🧠 Remember This

> Physical = "kitne baje?" (needs sync). Logical = "kaun pehle?" (no clock needed).

---

9. GLOBAL STATES 📘📝 (A2-2a, A2-2b)

9.1 What is a Global State?

Definition: The **global state** of a distributed system is the **collection of the local states of all processes together with the states of all communication channels (messages in transit)** at some instant.

Simple (🗣️): Poore system ki ek "photo" – har process ki state + raaste me phansay messages.

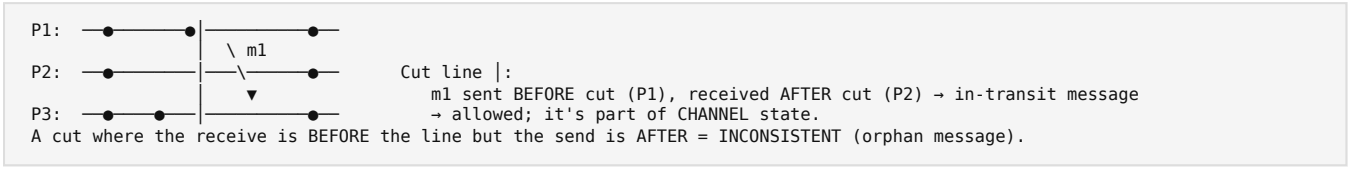
Components of a global state

Component	Meaning
Local state of each process	Values of variables/registers of p_i
State of each channel	The set of messages sent but **not yet received** (in transit)
(Formally)	`GS = { S_1, S_2, …, S_N } ∪ { channel states C_ij }`

9.2 Cuts and consistency

- **Cut:** a subset (prefix) of the system's events – a "line" across the timelines. Frontier of the cut = last event of each process included in the cut.

- **Consistent cut:** for every event **e in the cut**, all events that **happened-before e** are also in the cut. **Equivalent:** *no message is received inside the cut whose send is outside the cut* (no **orphan message**).
- **Inconsistent cut:** contains a receive event without its send (message "from the future").
- A **consistent global state** corresponds to a consistent cut.



9.3 Why is recording a consistent global state important?

1. **Stable-property detection: deadlock detection**, **termination detection**, **garbage collection** (once true, stays true).
2. **Checkpointing & recovery:** restart from a consistent state after failure.
3. **Distributed debugging:** examine global behaviour.
4. **Auditing/consistency:** e.g., total money in a bank distributed on servers must remain constant.
5. **Load balancing, replica consistency** ➕. Problem: **no global clock and no shared memory ⇒ can't take all local states at the same instant**, so a special algorithm is required.

9.4 Distributed Snapshot – Chandy–Lamport Algorithm 📝 (A2-2b)

Purpose: record a **consistent global state** (snapshot) of a running distributed system. **Assumptions:** - Channels are **FIFO, reliable (no loss)** and **unidirectional**; processes are **connected** (strongly). - No failures during snapshot; snapshot doesn't stop the application; any process can initiate. - A special control message called **MARKER** is used.

Algorithm

```
INITIATOR P_i (or any process receiving a marker for the first time):
   1. Record own local state.
   2. Send a MARKER on EVERY outgoing channel (before any further app message).
   3. Start recording messages arriving on every incoming channel.

ON RECEIVING MARKER on incoming channel c at P_j:
   if P_j has NOT yet recorded its state:
        record local state;  channel state of c = EMPTY;
        send MARKER on all outgoing channels;
        start recording on all OTHER incoming channels.
   else (already recorded):
        stop recording channel c;
        channel state of c = messages received on c since P_j recorded its state.

TERMINATION: when every process has received a marker on ALL its incoming channels.
```

Why it works: The marker acts as a **separator** on each FIFO channel: messages before the marker belong to the "before-snapshot" world, messages after the marker to the "after".

Simple example with P1, P2, P3 (bank balances) 📝

Initial: P1 = ₹500, P2 = ₹300, P3 = ₹200 → **Total = ₹1000**. Channels: C12, C21, C13, C31, C23, C32.

Sequence of events: 1. P1 sends ₹50 to P2 (message **m_1** on C12). P1 balance = 450. 2. P1 **initiates snapshot**: records its state **S1 = 450**, sends MARKER on C12 and C13, starts recording C21 and C31. 3. P3 sends ₹20 to P1 (**m_2** on C31) before receiving any marker. P3 balance = 180. 4. P2 receives **m_1** (before marker on C12) → P2 balance = 350. Then P2 receives **MARKER on C12** → first marker: records **S2 = 350**, C12 state = {} (empty) [m_1 was received before], sends MARKERs on C21, C23, starts recording C32. 5. P3 receives **MARKER on C13** → records **S3 = 180**, C13 = {}, sends MARKERs on C31, C32, starts recording C23. 6. P1 receives m_2 on C31 **while recording** → then MARKER on C31 ⇒ **C31 = {₹20 (m_2)}**. P1 also gets MARKER on C21 → C21 = {}. 7. P2 gets marker on C32 → C32 = {}. P3 gets marker on C23 → C23 = {}. Snapshot complete.

```
P1: ──send m1──[S1=450 record, MARKER→P2,P3]──── recv m2 ── recv MARKER(C31)
P2: ──────────── recv m1 ── recv MARKER(C12)[S2=350 record, MARKER→P1,P3]
P3: ──send m2──── recv MARKER(C13)[S3=180 record, MARKER→P1,P2]
```

Recorded global state: S1 = 450, S2 = 350, S3 = 180, C31 = {₹20}, others empty. **Check:** 450 + 350 + 180 + 20 = **₹1000** ✓ (money conserved ⇒ consistent snapshot, even though no single instant ever had exactly these values).

Complexity: O(E) marker messages (one per channel, E = number of channels); at most **N(N−1)** for fully connected; O(D) time (D = diameter). **Advantages:** non-blocking (application continues), simple, works with any topology, records consistent state. **Limitations:** needs **FIFO** channels, no failures, records a state that *may never have occurred* in the actual execution (but is reachable from it), overhead of markers. **Uses:** deadlock detection, termination detection, checkpoint/recovery.

> The assignment (A2-2b) says "design a simple method for recording a consistent global state for P1, P2, P3" → answer = **Chandy–Lamport marker algorithm** with the above example.

🧠 Remember This

> Global state = process states + channel states. Consistent cut = no orphan messages. Chandy-Lamport: **marker** → record state, send markers, record incoming channels till marker.

---

10. DISTRIBUTED DEBUGGING 📘

Definition

Distributed debugging = detecting and analysing **bugs/violations of conditions (predicates)** in a running distributed program by **observing consistent global states**, since no single process can see the whole system and there is no global clock.

Simple (🗣️)

Ek computer pe debugging = breakpoint lagao. Distributed me poore system ki "sahi photo" (consistent global state) leni padti hai, tabhi bug pakadna possible hai.

Why hard?

No global clock, no global state, non-determinism (message timing), concurrency, debugging alters behaviour (**probe effect**).

Approach (Coulouris §14.5)

1. Each process **p_i** sends its **state (or changes) with vector timestamp** to a **monitor process**.
2. Monitor **assembles consistent global states** using vector timestamps (state S is consistent iff no message inconsistency between local states).
3. Monitor **evaluates a global state predicate Φ** on each consistent state.
4. Types of predicates: - **Stable predicate:** once true remains true (deadlock, termination, garbage). Detect by snapshot. - **Non-stable predicate:** can become true and false again (e.g., "x1 = x2 at some time", **safety violation**: `x – y > δ`).

Global state lattice / possibly & definitely

- The set of all consistent global states forms a **lattice**; an execution is a **path** (linearization) through the lattice from initial to final state.
- **Possibly Φ:** there **exists** a consistent global state in which Φ is true, reachable in some linearization (Φ *might* have happened).
- **Definitely Φ: every** linearization passes through a consistent global state where Φ is true (Φ *certainly* happened).
- Relations: `Definitely Φ ⇒ Possibly Φ`.

```
Initial state S00
    /      \
 S10       S01         (states labelled by #events of P1, P2)
    \  ×  /
    S11 ... → final
Φ possibly true if any path/state satisfies Φ; definitely if all paths hit such state.
```

- **Used for:** finding **safety violations**, deadlocks, **data races**, missing/duplicate messages, assertion checking; tools: logging with vector clocks, replay/record, tracing (e.g., distributed tracing).

Advantages / limitations

Advantages: correct detection of global conditions, uses causality (not time). Limitations: **state explosion** (lattice huge), high message/storage overhead, **probe effect**, needs vector timestamps.

🧠 Remember This

> Debugging = monitor collects local states with **vector timestamps** → builds **consistent global states** → tests predicate; **Possibly Φ** vs **Definitely Φ**.

---

11. COORDINATION AND AGREEMENT — INTRODUCTION 📘

- Processes must **coordinate** actions (who enters CS, who is leader, who receives which message) and **agree** on values (commit/abort, common decision) despite **failures** and **no shared memory**.
- **Failure assumptions (Coulouris §15.1):** processes may **crash** (fail-stop) or be **Byzantine**; channels reliable or not; **failure detector** (unreliable vs reliable) – in asynchronous system a timeout can't be certain.
- Topics in this section: **Distributed mutual exclusion → Election → Multicast → Consensus**.

```
COORDINATION & AGREEMENT
  ├─ Mutual exclusion (who uses resource)
  ├─ Election (who is coordinator)
  ├─ Multicast (send to group with guarantees)
  └─ Consensus (all agree on one value)
```

🧠 Remember This

> Coordination = decide who/when. Agreement = decide what. Both under failures.

---

12. DISTRIBUTED MUTUAL EXCLUSION 📘📝 (A2-3a) — ⭐ HIGH PRIORITY

Definition

Mutual exclusion (ME) ensures that **at most one process executes in its critical section (CS)** – accessing a shared resource – at a time, in a distributed system **using message passing only** (no shared memory/semaphores).

Need (🗣️)

Do process ek hi file/DB record pe ek saath likhenge to data corrupt (race condition). Isliye "ek time pe ek hi process CS me." Examples: updating a shared bank record, printing on a shared printer, writing to a file server, updating replicated database.

Requirements (Coulouris)

Requirement	Meaning
ME1 – Safety	At most **one** process in CS at a time
ME2 – Liveness	Every request to enter/exit CS **eventually succeeds** (no deadlock, no starvation)
ME3 – Ordering (→ fairness)	If request A **happened-before** request B, A is granted before B

Also: **no deadlock, no starvation, fault tolerance, fairness (FIFO order)**.

Performance criteria

1. **Bandwidth** – number of messages per CS entry/exit.
2. **Client delay** – delay at entry/exit when nobody else waiting.
3. **Synchronization delay** – time between one process leaving the CS and next entering (messages needed).

Classification

```
DISTRIBUTED MUTUAL EXCLUSION
├── Centralised (central server)
├── Token-based  : Ring algorithm, Suzuki–Kasami ➕
└── Permission-based (non-token): Ricart–Agrawala, Lamport, Maekawa (voting)
```

12.1 Algorithm 1: Central Server Algorithm

Idea: one server (coordinator) holds a **token**; process must get it to enter CS. **Steps:** 1. Process sends **request** to server. 2. If token is free → server sends **grant** immediately; else queues request. 3. After CS, process sends **release**; server grants to next in queue.

```
P1 ──request──► Server ──grant──► P1 … (CS) … P1 ──release──► Server ──grant──► P2
```

Messages: 3 per entry (request, grant, release); entry delay 2 messages; sync delay 2 messages. **Advantages:** simple, fair (queue), few messages. **Disadvantages: single point of failure**, bottleneck, needs election on server crash, can't tell dead server from long wait.

12.2 Algorithm 2: Ring-based Algorithm (token ring)

Assumption: processes arranged in a **logical ring**; each knows its neighbour; one **token** circulates. **Steps:** 1. Token passes around ring in one direction. 2. A process wanting CS **waits for token**, enters CS while holding it. 3. On exit (or if it doesn't need CS) passes token to neighbour.

```
P1 ──► P2 ──► P3
 ▲             │        token travels; only the holder may enter CS
 └─── P5 ◄── P4 ◄┘
```

Messages: 1 to N−1 per entry (avg N/2); **continuous consumption of bandwidth** even when no one wants CS. Sync delay 1 to N−1 messages. **Adv:** simple, no starvation (bounded wait ≤ N−1), deadlock-free. **Disadv: token loss** (needs detection/regeneration), wasted messages, process crash breaks ring, ordering not FIFO/happened-before.

12.3 Algorithm 3: Ricart–Agrawala Algorithm (multicast + logical clocks) — ⭐ MOST ASKED

Purpose: ME **without a token or central server**, using **Lamport timestamps** for a total order of requests. **Assumptions:** N processes; reliable message delivery; each process has unique id and Lamport clock; processes know each other; no failures. **States:** RELEASED (not interested), WANTED (wants CS), HELD (in CS).

Algorithm

```
On initialisation:   state := RELEASED

To ENTER the CS:
   state := WANTED
   multicast REQUEST(T, p_i)  to all other processes     (T = timestamp of request)
   wait until N-1 REPLIES received
   state := HELD

On RECEIPT of REQUEST(T_j, p_j) at p_i:
   if ( state == HELD  or  (state == WANTED and (T_i, p_i) < (T_j, p_j)) )
        queue the request (defer reply)
   else
        reply to p_j immediately

To EXIT the CS:
   state := RELEASED
   send REPLY to all queued requests
```

(Comparison (T,p) uses **smaller timestamp = higher priority; tie broken by process id**.)

Example (3 processes P1, P2, P3): P1 and P2 request simultaneously with timestamps (5,P1) and (7,P2). P3 is RELEASED. 1. P1 → REQUEST(5,P1) to P2, P3; P2 → REQUEST(7,P2) to P1, P3. 2. P3 (RELEASED) replies to both immediately. 3. P1 receives P2's request: P1 is WANTED with (5,P1) < (7,P2) ⇒ **defers** (queues) P2's reply. 4. P2 receives P1's request: P2 is WANTED with (7,P2) > (5,P1) ⇒ **replies** to P1. 5. P1 has replies from P2 and P3 ⇒ **enters CS**. 6. On exit P1 sends the deferred REPLY to P2 ⇒ P2 has replies from P1 and P3 ⇒ **enters CS**.

```
P1 ──REQ(5)──► P2, P3          P2 ──REQ(7)──► P1, P3
P3 ── REPLY ─► P1, P2
P2 ── REPLY ─► P1  → P1 enters CS
P1 (exit) ── REPLY ─► P2 → P2 enters CS
```

Message complexity: 2(N−1) per CS entry: (N−1) requests + (N−1) replies. **Synchronization delay:** 1 message transmission time. **Client delay:** round-trip. **Advantages:** no token/coordinator, satisfies ME1–ME3 (fair, FIFO by timestamps), deadlock- & starvation-free, no release messages. **Disadvantages: N−1 messages each entry ⇒ poor**

scalability, failure of **any** process blocks everyone (no reply), every process involved even if uninterested, needs reliable delivery. (Optimisation: **Maekawa's** voting sets.)

12.4 Algorithm 4: Lamport's Distributed Mutual Exclusion ➕ (also in your lab list)

Idea: each site keeps a **request queue ordered by (timestamp, id)**. **Steps:** 1. To request CS, P_i sends **REQUEST(ts_i, i)** to all others and puts it in its own queue. 2. On receiving a request, P_j puts it in its queue and sends **REPLY** (timestamped). 3. P_i **enters CS** when (a) its request is **at the head of its queue**, and (b) it has received a **reply with a larger timestamp from every other process**. 4. On exit, P_i removes its request and sends **RELEASE** to all; others remove that request from their queue. **Messages: 3(N−1)** per CS (N−1 REQUEST + N−1 REPLY + N−1 RELEASE). Sync delay = 1 message. Requires **FIFO channels**.

12.5 Algorithm 5: Maekawa's Voting Algorithm ➕

Idea: each process needs permission only from a **voting set V_i** (subset of size ≈ √N) such that every two voting sets overlap ($V_i \cap V_j \neq \emptyset$). A process can vote for only one candidate at a time. **Steps:** request → send to all members of V_i; a member replies (votes) if it hasn't voted since the last release; otherwise queues; enter CS after getting **all** votes; on exit send **release** to V_i. **Messages:** ≈ **3√N** (2√N entry + √N exit). **Problem: deadlock-prone** (fixed by ordering requests by happened-before).

12.6 Comparison table – mutual exclusion algorithms (⭐ exam)

Algorithm	Type	Messages per entry	Sync delay (message times)	Problems	Remarks
Central server	Centralised	**3** (req, grant, release)	2	Server crash = SPOF	Simple, fair
Token ring	Token	**1 to N−1** (continuous ring traffic)	1 to N−1	Token loss, node crash	No starvation
Ricart–Agrawala	Permission	**2(N−1)**	1	Any crash blocks; heavy	Timestamps, no release msg
Lamport	Permission	**3(N−1)**	1	Needs FIFO; heavy	Uses request queues
Maekawa	Voting	**~3√N**	2	Deadlock possible	Scalable
Suzuki–Kasami ➕	Broadcast token	**N** (N−1 requests + 1 token) or 0 if token held	1	Token loss	Efficient

Numerical (N = 5): RA = 2×4 = **8**; Lamport = 3×4 = **12**; Central = **3**; Ring = 1–4; Maekawa ≈ 3√5 ≈ 7 (roughly).

12.7 Exam points

- ⭐ For a 4-mark answer "requirements + any two algorithms": ME1, ME2, ME3 (1 mark) + **Central server + Ricart–Agrawala** (or Ring) with diagram and message count (3 marks).
- Be ready to say how each **fails** (crash, token loss) and which is best in message count (central server) vs fully distributed (RA).

🧠 Remember This

> ME1 safety, ME2 liveness, ME3 ordering. Central: 3 msgs. Ring: token circulates. **Ricart-Agrawala: 2(N−1), timestamp comparison, defer replies while HELD/higher priority.** Lamport: 3(N−1).

---

13. ELECTION ALGORITHMS 📘📝 (A2-3b) — ⭐ HIGH PRIORITY

Definition

An **election algorithm** chooses **one process as coordinator (leader)** from a group of processes (e.g., the process with the **highest ID**) so that all processes agree on who it is.

Need for election (🗣️: "leader kaun?")

- A **coordinator** is needed for central-server mutual exclusion, time server (Berkeley master), primary in replication, token regeneration, etc.
- **When the coordinator crashes** or at system start, the remaining processes must **elect a new one**.
- Requirements: **E1 Safety** – every participating process has elected the same process (the non-crashed one with the **largest identifier**); **E2 Liveness** – all processes participate and eventually set elected variable or crash.
- Assumption: every process has a **unique ID** (priority) and knows the IDs of others (bully) or its neighbour (ring). Message types in Bully: **ELECTION, ANSWER (OK), COORDINATOR**.

13.1 Bully Algorithm (Garcia-Molina)

Idea: The process with the **highest ID** always wins ("bullies" others). **Assumptions: synchronous** system (timeouts reliable); reliable message delivery; each process knows the IDs of **all** others; failures only crash; crashed process may recover.

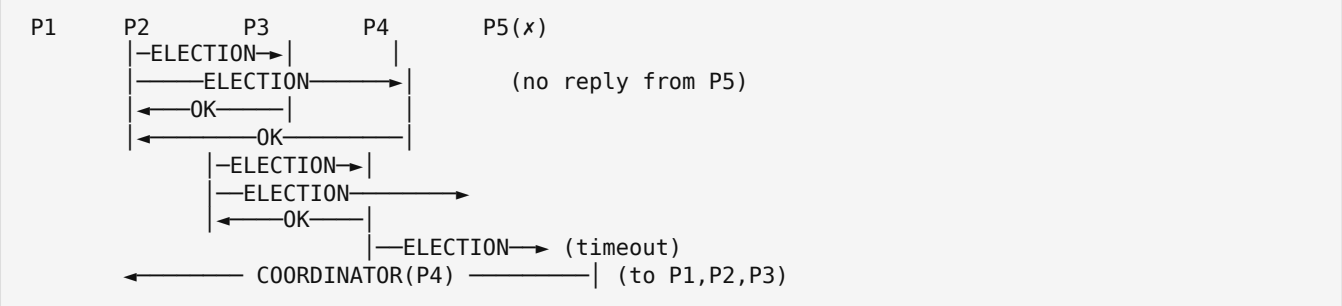
Steps 1. A process P that **notices coordinator failure** starts an **election**: sends **ELECTION** to all processes with **higher ID**. 2. If **no one responds** (timeout) → P **wins**; it sends **COORDINATOR** message to all lower-ID processes (in fact to all). 3. If a **higher process responds with ANSWER (OK)** → P's job is over; it waits for a COORDINATOR message. 4. Each higher process that receives ELECTION replies **OK** and **starts its own election** (step 1).

5. Eventually the **highest alive process** gets no answer, and announces **COORDINATOR**. 6. If a crashed process **recovers**, it starts an election; if it has the highest ID it takes over ("bully").

Example (5 processes P1–P5, P5 was coordinator and crashes ✗):

```
Step 1: P2 detects P5 down → sends ELECTION to P3, P4, P5.
Step 2: P3 and P4 reply OK to P2. (P5 silent ✗)  P2 stops.
Step 3: P3 sends ELECTION to P4, P5.   P4 sends ELECTION to P5.
Step 4: P4 replies OK to P3. P5 silent. P3 stops.
Step 5: P4 timeouts waiting for P5 → P4 wins → sends COORDINATOR(4) to P1,P2,P3.
```

Message flow diagram:



Message complexity: Best case: the process with the second-highest ID detects failure → it sends **N−2** COORDINATOR messages immediately. **Worst case:** the process with the **lowest ID** detects failure → **O(N²)** messages (all higher processes start elections). **Advantages:** simple, fast when high-ID processes are alive, tolerates recovery; guarantees highest-ID wins. **Disadvantages: many messages O(N²)** worst case, requires **timeouts (synchronous)**, every process must know all IDs, if high-priority process flaps (crashes/recovers repeatedly) it causes repeated elections, incorrect timeouts can cause two coordinators.

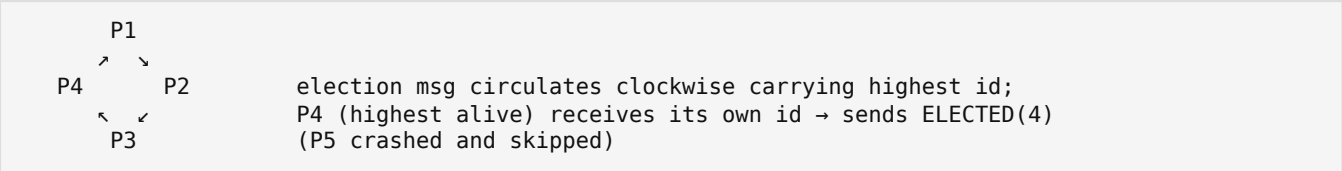
13.2 Ring Algorithm (Chang–Roberts)

Idea: processes arranged in a **logical ring** (each knows its successor); an **election message** circulates carrying the **highest ID seen** so far; the process that receives **its own ID back** is the winner. **Assumptions:** ring topology, **unidirectional**; each process knows its **next neighbour** (skips crashed); works in an **asynchronous** system; unique IDs; no failure during election.

Steps 1. Initially every process is **non-participant**. A process P noticing failure marks itself **participant** and sends **ELECTION(id_P)** to its neighbour. 2. On receiving ELECTION(id) at Q: - if **id > id_Q**: forward ELECTION(id); mark Q participant. - if **id < id_Q** and Q is **not participant**: **substitute** own id, send ELECTION(id_Q), mark participant. - if **id < id_Q** and Q **already participant**: **discard** (don't forward). - if **id == id_Q**: Q is the **coordinator**; it marks itself non-participant and sends **ELECTED(id_Q)** around the ring. 3. When ELECTED message comes to a process, it records the coordinator, marks itself non-participant and forwards; it stops when it returns to the coordinator.

Example: ring P1→P2→P3→P4→P5→P1 with P5 crashed (skip it): P4→P1. P2 starts election with id 2.

```
P2 —ELECTION(2)► P3 (3>2 → substitute 3) —ELECTION(3)► P4 (4>3 → substitute 4) —ELECTION(4)► P1(1<4 → forward 4)
—ELECTION(4)► P2 (2<4 → forward) —► P3 (forward) —► P4 receives its own id 4 → P4 = COORDINATOR
P4 —ELECTED(4)► P1 —► P2 —► P3 —► back to P4 (end)
```



Message complexity: Best: 2N (election N + elected N) — when the initiator has highest id. **Worst: 3N − 1** (when the initiator's **anticlockwise neighbour** is the highest: N−1 messages to reach it, N to complete the circle, N for elected). **Advantages:** fewer messages than bully, no need to know all IDs, works in asynchronous systems, simple ordering. **Disadvantages:** slow for large rings (O(N) latency), a **failure of a node during election breaks the ring**, needs ring reconstruction, several simultaneous elections possible (handled by participant flag).

13.3 Comparison: Bully vs Ring (📝 A2-3b: "compare their message communication and assumptions")

Feature	Bully Algorithm	Ring Algorithm
Topology	**Fully connected** (any process can message any other)	**Logical ring** (only to neighbour)
Knowledge needed	IDs of **all** processes	Only **next neighbour**
Messages used	ELECTION, ANSWER/OK, COORDINATOR	ELECTION (with id), ELECTED
Best-case messages	**N−2**	**2N**
Worst-case messages	**O(N²)**	**3N−1** = O(N)
System assumption	**Synchronous** (timeouts)	**Asynchronous** OK
Winner	Highest-ID alive process	Highest-ID alive process
Who announces	Winner sends COORDINATOR to all	ELECTED message circulates ring
Failure during election	Handled via timeouts, may restart	Ring broken; needs repair
Speed	Fast when initiator has high id	Slower (message travels whole ring)
Simultaneous elections	Multiple starters possible	Handled by participant flag
Recovery of process	Recovered process may start election / bully	Rejoins ring, may trigger election

Exam points

- Draw a 5-process example for each algorithm; state message types; give best/worst counts; **the key line: "highest ID wins in both"**.

🧠 Remember This

> **Bully:** ELECTION→higher ids; OK; highest alive sends COORDINATOR; worst $O(N^2)$. **Ring:** id circulates, own id returns → winner; ELECTED circulates; worst $3N-1$.

---

14. MULTICAST COMMUNICATION 📘

Definition

Multicast = sending **one message to all members of a group** of processes (one-to-many), as opposed to unicast (one-to-one) and broadcast (to everyone).

Simple (🗣️)

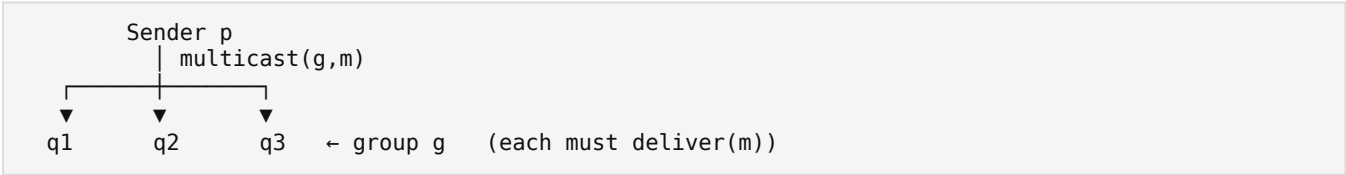
WhatsApp group me ek message sabhi members ko jaata hai. Yahi multicast hai – par DS me guarantee chahiye: sabko mile, ek order me mile.

Uses (why needed)

Fault tolerance via **replicated services**, **finding discovery services** in spontaneous networking, **better performance** through replicated data, **event notification/propagation** (stock prices, news), online games, video conferencing, cache consistency.

Group system model

- **Closed group:** only members can multicast (replicated servers). **Open group:** outsiders can send too (event dissemination).
- **Overlapping groups** possible; **synchronous/asynchronous** systems.
- Operations: `multicast(g, m)`, `deliver(m)` (delivered to application ≠ received on the network; **hold-back queue** may delay delivery).
- **IP multicast** (➕): efficient, unreliable UDP-based; class D addresses; routers replicate packets.



14.1 Basic (B-)multicast

`B-multicast(g,m)`: for each q in g: `send(q, m)` (over reliable 1-to-1 channels). `B-deliver(m)` on receive. **Problem:** if sender crashes midway, some members get m and some don't ⇒ no agreement. (Also **ack-implosion** if all ack.)

14.2 Reliable multicast – properties (Coulouris)

Property	Meaning
Integrity	A correct process delivers a message **at most once**, and only if sent by a group member
Validity	If a correct process multicasts m, it **eventually delivers** m itself
Agreement	If a correct process delivers m, then **all other correct processes** in the group eventually deliver m
- **Reliable multicast R-multicast** built over B-multicast: when a process first receives message m, if it is not the sender it **re-multicasts (B-multicasts) m to the group** before delivering; ensures agreement even if sender crashes. Costly ($O(N^2)$ messages). Alternatively use **piggybacked acknowledgements** and **negative acks (NACK)** for efficiency.	
- **Uniform agreement** (➕): if *any* process (even one that later crashes) delivers m, all correct processes do.	

14.3 Ordered multicast

Ordering	Guarantee	Typical implementation
FIFO ordering	If a process multicasts m1 before m2, every correct process delivers m1 before m2	**Sequence numbers** per sender, hold-back queue
Causal ordering	If multicast(g,m1) → multicast(g,m2) (happened-before) then every process delivers m1 before m2	**Vector timestamps**
Total ordering	If a correct process delivers m1 before m2, **all** correct processes deliver m1 before m2 (same order everywhere)	**Sequencer process** (assigns sequence numbers) or **ISIS agreed-priority** algorithm
- Hierarchy: **Causal ⊃ FIFO** (causal implies FIFO); Total is independent; **FIFO-total, causal-total** combinations exist.		

Ordering	Guarantee	Typical implementation
- **Total ordering with a sequencer:** sender sends m to sequencer; sequencer assigns sequence number s and multicasts (m,s); receivers deliver in order of s (hold-back until earlier numbers arrive).		
- **ISIS algorithm:** sender B-multicasts m; each receiver replies with a **proposed priority**; sender picks the **maximum** as **agreed priority** and multicasts it; receivers deliver in agreed-priority order.		

```
Total order example: P1 sends m1, P2 sends m2 concurrently.
All processes must deliver in the same order: either (m1,m2) everywhere or (m2,m1) everywhere.
```

Hold-back queue (diagram):

```
Network ─► [Hold-back queue] ─(when ordering condition met)─► [Delivery queue] ─► Application
```

Advantages: efficient dissemination, replication support, less network traffic than N unicasts.
Disadvantages: ordering/reliability complexity, message overhead (acks), scalability, failure handling, group membership management.

Exam points

- Draw group diagram; define reliable multicast (integrity, validity, agreement); compare FIFO/causal/total ordering with one example.

🧠 Remember This

> Multicast = one→group. Reliable: **integrity, validity, agreement**. Ordered: **FIFO, causal (vector clocks), total (sequencer/ISIS)**.

---

15. CONSENSUS AND RELATED PROBLEMS 📘📝 (A2-4a, A2-4b) — ⭐ HIGH PRIORITY

Definition

Consensus: a set of processes each **proposes a value**; they must all **agree on one value** (decide), even if some processes fail.

Simple (🗣️)

5 dost movie choose karte hain – har koi apni pasand batata hai, aakhir me **sabko ek hi movie** par agree hona hai, chahe ek dost phone band kar de.

Need for consensus

Commit or abort a **transaction** (2PC), **leader election**, **replicated state machines**/replicated database consistency, **agreeing on order of messages** (total-order multicast), **mutual exclusion / access decision**, blockchain, **agreeing on time or sensor value** in fault-tolerant control, **airplane control systems** (redundant computers).

Requirements (Coulouris §15.5) 📝

Each process p_i starts in **undecided** state, proposes value v_i, then **decides** value d_i (decided state, cannot change). | Condition | Meaning | |---|---| | **C1 – Termination** | Every correct process **eventually decides** (sets its decision variable) | | **C2 – Agreement** | The decision value of **all correct processes is the same**: if p_i and p_j are correct and decided, then d_i = d_j | | **C3 – Integrity / Validity** | If all correct processes proposed the **same value v**, then any correct process that decides has decided **v** (decided value must be one that was **proposed**; not arbitrary) |

> Your syllabus/assignment wording: **Agreement, Validity, Termination** – same three conditions.

Simple example (assignment 4a)

Three replicas of a database decide whether to **commit** a transaction. Proposals: P1 = commit, P2 = commit, P3 = abort. Using a protocol (e.g., majority): decision = **commit** by all. Agreement ✓ (all commit), validity ✓ (commit was proposed), termination ✓ (all decided). If all proposed commit, decision must be commit (validity).



Consensus in a synchronous system with crash failures – flooding algorithm ➕

- Up to **f** processes may crash; **f + 1 rounds** needed.
- Each process keeps set `Values_i`. In each round, it **B-multicasts** the values it hasn't sent yet, then adds received values.
- After **f+1 rounds**, each process decides on **minimum (or majority function) of Values_i**.
- Why f+1: at least one round has **no crash**, ensuring all correct processes have the same set.

```
Round 1: each Pi multicasts its value; crash may cause partial delivery
Round 2..f+1: multicast newly learned values
Decide: d = min(Values_i)
```

Complexity: (f+1) rounds; O(N²) messages per round.

Process failures & communication delays (assignment 4b challenges) 📝

Challenge	Effect
Crash failure	A process stops silently; others can't distinguish crash from slowness in asynchronous systems
Byzantine (arbitrary) failure	Process sends **conflicting/wrong values** to different peers
Message loss / omission	Some messages never arrive → incomplete view
Communication delay (async)	Unbounded delays make timeouts unreliable; process may be wrongly declared dead
Partial multicast	Failed process sends its value to only some processes → disagreement
Termination vs agreement conflict	Waiting for failed process ⇒ never terminates
- **FLP impossibility (Fischer–Lynch–Paterson, 1985):** in a **purely asynchronous** system, **no deterministic consensus algorithm** can guarantee termination if even **one** process may crash. Practical systems use **timeouts, randomisation, failure detectors** (Paxos, Raft).	

🎯 Case study 4b: 5 processes, at most 1 may fail

Case 1 – Crash failure (f = 1): N = 5, use **f+1 = 2 rounds**. 1. Round 1: each process broadcasts its proposed value to all. 2. Suppose P5 crashes after sending its value to only P1 and P2. 3. Round 2: processes re-broadcast any new values they learnt (P1, P2 forward P5's value to P3, P4). 4. After round 2 all four **correct** processes have the **same set**. Each applies the same deterministic rule (e.g., **minimum** or **majority**) ⇒ **agreement**. 5. Validity: chosen value was proposed; Termination: decision after 2 rounds.

Case 2 – Byzantine failure (f = 1): need **N ≥ 3f + 1 = 4** processes (we have 5 ✓) and **f+1 = 2 rounds**. Each process collects values; the faulty one may send different values to different peers; correct ones **exchange what they received** and take **majority** ⇒ faulty process's lies are outvoted by the 4 correct processes. **Challenges:** failure detection uncertain, delays cause timeouts, message overhead grows (O(N²) per round), need reliable channels, need common decision rule.

Consensus-related problems 📘 ("consensus and related problems")

Problem	Description	Condition
Byzantine Generals Problem	Commander sends order to lieutenants; some (possibly the commander) may be traitors. Loyal lieutenants must **agree** (BG1) and if commander is loyal, they follow his order (BG2 – integrity)	Solvable only if **N ≥ 3f + 1** (oral messages); with signed messages any number tolerated (in sync systems)
Interactive Consistency	Each process proposes value; all must agree on a **vector of values** (one per process)	IC1 agreement on vector; IC2 if p_i correct, everyone's entry for p_i is v_i
Total-ordered multicast	All deliver messages in same order (equivalent to consensus)	Solves & is solved by consensus
Leader election	Agree on one leader (consensus on id)	
Atomic commit (2PC/3PC) ➕	All participants commit or all abort	
Reliable multicast	Agreement on delivered messages	
Relationships: Consensus (C), Byzantine Generals (BG), Interactive Consistency (IC) can each be **derived from** the others, so they are **equivalent** (Coulouris).		

Byzantine tolerance bound: with **f** faulty processes need **N ≥ 3f+1**; **f+1 rounds** (synchronous). Example: **N=3, f=1 impossible** – loyal process can't tell whether commander or another lieutenant is lying.

```
N=4, f=1: commander C loyal sends v to L1,L2,L3; L3 faulty tells others "commander said w".
L1 majority(v, v, w) = v; L2 majority = v → agreement.
```

Advantages/limitations of consensus protocols

Advantages: fault-tolerant, consistent replicas. Limitations: high message cost, FLP impossibility, complex, requires majority/quorum. **Practical protocols ➕:** Paxos, Raft (majority quorum, tolerate f crash failures with 2f+1 nodes).

🧠 Remember This

> Consensus = agree on one value. **Termination + Agreement + Validity(Integrity)**. Crash: f+1 rounds. Byzantine: N ≥ 3f+1. Async + 1 crash ⇒ no deterministic guarantee (FLP).

(End of Unit-II. Next: `03_Faculty_Emphasis_and_Quick_Tables.md`)

FACULTY EMPHASIS · DEFINITIONS · COMPARISONS · DIAGRAMS · ALGORITHMS

> This file = Sections 4–8 of the final structure. Everything here is a **fast-access summary** of the detailed notes in files 01 and 02.

SECTION 4 — 🔥 FACULTY-EMPHASIZED / HIGH-PRIORITY TOPICS

4.1 Analysis of your BPIT assignments (facts read from the uploaded files)

Assignment-1 (Unit-I) — 7 sub-questions, 16 marks total, all CO1

Q	Type	Topic	Bloom	Marks
1a	Conceptual	Define DS + characteristics with examples	Understand	2
1b	Analytical	Client–Server, P2P, Grid, Cloud – architectures & applications compared	Analyze	2
1c	Problem solving	**Case study – Java RMI** (calculator/student info): architecture, remote interface, server, client, registry, steps, code, output	Analyze	3
2a	Conceptual	System models: need, Architectural & Fundamental with diagrams	Understand	2
2b	Analytical	Networking & internetworking; role of networks, internetworks, protocols	Analyze	2
3a	Conceptual	IPC: message passing vs shared memory (working, advantages, limitations, applications)	Understand	3
3b	Analytical	Distributed objects & RMI: remote objects, remote interfaces, stubs/proxies, invocation process	Analyze	2

Bloom split (Unit-I): Understand = 3 questions (7 marks); Analyze = 4 questions (9 marks). No "Remember" or "Apply/Create" level.

Assignment-2 (Unit-II) — 9 sub-questions, 32 marks total, CO2 and CO3

Q	Type	Topic	Bloom	Marks
1a	Conceptual	Synchronization; role of events & process states	Understand	2
1b	Conceptual	Happened-before (→) + Lamport clocks with example	Understand	3

Q	Type	Topic	Bloom	Marks
1c	Analytical	Need for physical clock sync; clock drift, skew, communication delay	Analyze	3
2a	Conceptual	Components of global state; importance of consistent global state	Understand	4
2b	Problem solving	**Case study – Distributed snapshot** with P1, P2, P3	Analyze	4
3a	Conceptual	Requirements of mutual exclusion + any two algorithms	Understand	4
3b	Analytical	**Election: Bully & Ring; compare messages & assumptions**	Analyze	4
4a	Conceptual	Consensus: agreement, validity, termination, example	Understand	4
4b	Problem solving	**Case study – Consensus with 5 processes, 1 may fail**; challenges	Analyze	4

Bloom split (Unit-II): Understand = 5 questions (17 marks); Analyze = 4 questions (15 marks). **Marks weight (Unit-II):** Coordination & agreement (3a, 3b, 4a, 4b) = **16 of 32**; Global state (2a, 2b) = **8**; Time/clocks (1a–1c) = **8**.

> Observation: the syllabus tags coordination/agreement as CO3 in your assignment, though the official CO3 text in the PDF is about Hadoop – this is a labelling quirk of the assignment; the *topics* are in the Unit-II syllabus, so study them.

4.2 Patterns noticed

1. **Every question is either "Explain/Describe…" (Understand) or "Explain + compare/case-study" (Analyze).** Expect *explain + diagram + example* type questions rather than pure one-liners.
2. **Case-study/problem-solving questions appear 3 times** (Java RMI, Distributed Snapshot, Consensus with 5 processes). Practise applying concepts to a scenario.
3. **Comparison is asked explicitly** in 1b (four architectures), 3a (message passing vs shared memory), 3b (Bully vs Ring).
4. **Diagrams/timelines/message-sequence diagrams are explicitly encouraged** in the Assignment-2 instructions.
5. **Higher marks (4 each)** are placed on global state, mutual exclusion, election, consensus – the "algorithm" part of Unit-II.
6. Nothing in the assignments on: *advantages of DS, RPC (explicitly), events & notifications, distributed debugging, multicast, vector clocks, logical-vs-physical table*. **They are in the official syllabus**, so they can still come (especially as short questions or in a compulsory objective question).

4.3 PRIORITY RANKING

(Priority is my inference from assignment weightage + official syllabus; it is not a prediction of the exact paper.)

🔥🔥🔥 VERY HIGH PRIORITY

Topic	Why
Java RMI case study (architecture, code, steps, registry)	Appears as a dedicated 3-mark case study (A1-1c) — highest single Unit-I mark — and RMI again in A1-3b; also in official syllabus "Case study – Java RMI"
RMI/Distributed objects, stub/proxy, remote interface (+ RMI vs RPC)	A1-3b + A1-1c; RPC is in the syllabus and is the natural comparison
Message passing vs shared memory	A1-3a explicitly asks comparison (3 marks)
Client-server vs P2P vs Grid vs Cloud	A1-1b explicitly asks comparison; syllabus lists all four
System models (Architectural + Fundamental, with diagrams)	A1-2a; syllabus
Lamport logical clocks + happened-before (numerical)	A2-1b; core numerical topic
Global state + Chandy-Lamport snapshot (P1,P2,P3)	A2-2a + A2-2b = 8 marks (25% of Unit-II assignment marks)
Distributed mutual exclusion (requirements + algorithms)	A2-3a (4 marks); syllabus
Election – Bully & Ring, comparison	A2-3b (4 marks)
Consensus (agreement/validity/termination, 5-process case study)	A2-4a + A2-4b = 8 marks

🔥🔥 HIGH PRIORITY

Topic	Why
Characteristics of DS + definition	A1-1a; base for all theory answers
Physical clock sync: skew, drift, delay, Cristian/Berkeley/NTP	A2-1c (3 marks); Cristian/Berkeley added as exam-oriented
Events & process states; synchronization intro	A2-1a
Networking & internetworking	A1-2b (2 marks)
IPC – sockets, request-reply, marshalling	supports IPC/RMI questions
Logical vs physical time	Natural comparison question; in syllabus ("logical time and logical clocks")
RPC (stub, steps, semantics)	In syllabus; needed for RMI vs RPC

🔥 MEDIUM PRIORITY

Topic	Why
Events and notifications (publish–subscribe)	Syllabus topic, no assignment question ⇒ likely short/2–3 mark

Topic	Why
Advantages of distributed systems	Syllabus topic; typically 1–2 mark or part of a definition question
Multicast communication (reliable, FIFO/causal/total ordering)	Syllabus topic, not in assignment ⇒ short/medium
Distributed debugging (possibly/definitely)	Syllabus topic, not in assignment ⇒ short
Vector clocks	Not named in syllabus (➕), but the natural improvement over Lamport; good for "compare" or tough-paper questions
Mutual exclusion: Lamport/Maekawa	Beyond "any two algorithms" — lab list includes Lamport ME

> ⚠️ These are indicators only. **Do not skip a medium-priority topic** — the term-end pattern in the handbook says Question 1 is compulsory, covers the *entire* syllabus and is made of short answers.

SECTION 5 — IMPORTANT DEFINITIONS (learn word-for-word)

Unit-I

Term	Definition
Distributed system	A system in which hardware/software components located at networked computers communicate and coordinate their actions only by passing messages.
Transparency	Concealing the separation/distribution of components from user and application programmer so the system is seen as a whole.
Concurrency	Multiple processes executing at the same time and sharing resources.
Scalability	Ability of a system to work effectively as the number of users/resources grows.
Client–server	Architecture where client processes request services from server processes that manage resources.
Peer-to-peer	Architecture in which all processes play similar roles, acting as both client and server, sharing resources without central control.
Grid computing	Coordinated sharing of geographically distributed, heterogeneous resources of multiple organisations to solve large problems as a virtual supercomputer.
Cloud computing	On-demand delivery of computing services over the Internet with pay-per-use and elastic resource pooling.
Middleware	Software layer between OS and applications that masks heterogeneity and provides a uniform programming model (RMI, RPC, CORBA).

Term	Definition
System model	An abstract, simplified description of relevant aspects of a distributed system.
Architectural model	Description of system structure in terms of components, placement and relationships.
Fundamental model	Abstract model describing common properties: interaction, failure and security.
Latency	Delay between sending of message by sender and its start of receipt at receiver.
Bandwidth	Total amount of information that can be transmitted in a unit of time.
Jitter	Variation in the time taken to deliver a series of messages.
Synchronous DS	System with known upper bounds on process execution time, message delay and clock drift.
Asynchronous DS	System with no bounds on execution speed, message delay or drift.
Omission failure	Process or channel fails to perform an action (crash, message loss).
Arbitrary (Byzantine) failure	Any type of error may occur including wrong values or malicious behaviour.
Internetwork	Network of networks connected by routers, functioning as one logical network.
IPC	Mechanisms allowing processes to exchange data and synchronise, by message passing or shared memory.
Marshalling	Converting data structures into a flat, transmittable byte format.
Distributed object	Object whose methods can be invoked remotely by processes on other computers.
Remote object reference	An identifier for a remote object valid throughout the DS.
Remote interface	Specification of the methods of an object that can be invoked remotely.
RMI	Invocation of a method of a remote object from another process, appearing like a local call.
RPC	Calling a procedure in another address space as if it were local.
Stub / Proxy	Client-side local representative that marshals the request and unmarshals the reply.
Skeleton / Dispatcher	Server-side component that receives request, unmarshals, calls servant.
Event / Notification	Occurrence of interest / message about that occurrence delivered to subscribers.
Publish–subscribe	Model where publishers emit events and an event service delivers them to interested subscribers.
Java RMI registry	Naming service (port 1099) binding names to remote object stubs.

Unit-II

Term	Definition
Synchronization	Coordinating actions of processes in time and order in the absence of a global clock/shared memory.
Event	An action (internal, send, receive) that changes a process's state.
Process state	Values of all variables of a process at an instant.
Happened-before (→)	Smallest relation with: same-process order; send → receive; transitivity.
Concurrent events	Events with no happened-before relation between them ($a \parallel b$).
Lamport clock	Software counter giving timestamps such that $a \to b \Rightarrow L(a) < L(b)$.
Vector clock	Array of counters (one per process) such that $a \to b \Leftrightarrow V(a) < V(b)$.
Clock skew	Difference between two clocks' readings at the same instant.
Clock drift	Rate at which a clock deviates from perfect reference time.
UTC	Coordinated Universal Time, standard derived from atomic clocks.
Cristian's algorithm	Client sets time = T_server + RTT/2.
Berkeley algorithm	Master averages the clocks of group and sends each an adjustment.
NTP	Hierarchical Internet time synchronisation protocol using strata.
Global state	Local states of all processes plus states of all channels (messages in transit).
Cut	A subset of system events; consistent cut has no orphan (receive-without-send) messages.
Consistent global state	One corresponding to a consistent cut — could have occurred in an execution.
Snapshot	A recorded consistent global state (Chandy–Lamport).
Marker	Special control message used to delimit a snapshot on channels.
Stable predicate	Property that once true stays true (deadlock, termination).
Possibly Φ / Definitely Φ	Φ true in some / every linearization's consistent global state.
Mutual exclusion	At most one process in critical section at a time.
Critical section	Code segment accessing shared resource.
Election	Choosing a unique coordinator among processes.
Multicast	Sending one message to all members of a group.
Reliable multicast	Multicast with integrity, validity, agreement.
Total order	All processes deliver messages in the same order.
Consensus	Processes with proposals agree on one value (termination, agreement, validity).
Byzantine generals problem	Agreement problem with possibly traitorous commander/lieutenants; needs $N \geq 3f+1$.
Interactive consistency	All processes agree on a vector of proposed values.

Term	Definition
FLP impossibility	No deterministic consensus in async system with even one crash failure.

SECTION 6 — IMPORTANT COMPARISONS (quick tables)

(Full versions of the 4 most important ones are in file 01/02: CS vs P2P vs Grid vs Cloud; Message passing vs Shared memory; RMI vs RPC; Bully vs Ring; Logical vs Physical time.)

6.1 Architectural vs Fundamental models

	Architectural	Fundamental
Focus	Components & placement	Abstract properties
Includes	client-server, P2P, proxy, mobile code	interaction, failure, security

6.2 Synchronous vs Asynchronous system

	Synchronous	Asynchronous
Time bounds	Known	None
Failure detection	Reliable via timeout	Impossible to be sure
Example	Real-time control	Internet

6.3 Message passing vs Shared memory (summary)

	Message passing	Shared memory
Works across machines	✔	✘ (DSM emulation)
Speed	slower	faster
Sync	implicit	explicit locks

6.4 TCP vs UDP

TCP	UDP
Reliable, ordered, connection	Unreliable, no order, connectionless

6.5 RMI vs RPC

RPC	RMI
Procedures	Objects/methods
No object passing	Passes objects, remote refs
Language-neutral (IDL)	Java-specific (Java RMI)

6.6 Local vs Remote invocation

Local	Remote
Same address space	Different processes/machines
Pointers work	No pointers; marshalling
No partial failure	Can fail (lost msg, crash)
Fast	Slow (network)
Exactly-once	maybe / at-least-once / at-most-once

6.7 Lamport vs Vector clock

Lamport	Vector
1 integer	N integers
$a \to b \Rightarrow L(a) < L(b)$	$a \to b \Leftrightarrow V(a) < V(b)$
No concurrency detection	Detects concurrency

6.8 Clock skew vs clock drift

Skew	Drift
Difference between two clocks at an instant	Rate of deviation of a clock from real time
Measured in seconds	Measured in s/s (ppm)
Result of drift	Cause of skew

6.9 Cristian vs Berkeley vs NTP

	Cristian	Berkeley	NTP
Sync type	External	Internal	External
Uses	time server	average by master	strata hierarchy
Scale	small	small	Internet

6.10 Logical vs Physical clocks

Logical	Physical
Order of events	Real time
No sync needed	Sync needed
Unaffected by drift	Affected

6.11 Consistent vs Inconsistent cut

Consistent	Inconsistent
For each receive in cut, send also in cut	Has receive whose send is outside (orphan)
Valid global state	Impossible state

6.12 Centralised vs Distributed vs Token-based ME

	Central	Ricart-Agrawala	Token ring
Messages	3	2(N−1)	1..N−1
SPOF	Yes	No coordinator but each process	Token loss

6.13 Bully vs Ring (summary)

	Bully	Ring
Messages best/worst	N−2 / O(N^2)	2N / 3N−1
Knowledge	all IDs	neighbour only
System	synchronous	asynchronous

6.14 Unicast vs Broadcast vs Multicast

Unicast	Broadcast	Multicast
1 → 1	1 → all	1 → group

6.15 FIFO vs Causal vs Total ordering

FIFO	Causal	Total
per-sender order	happened-before order	same order everywhere
sequence numbers	vector clocks	sequencer/ISIS

6.16 Crash vs Byzantine failure

Crash	Byzantine
Process stops	Behaves arbitrarily/maliciously
Need N ≥ 2f+1 (majority protocols)	Need N ≥ 3f+1
f+1 rounds (sync)	f+1 rounds (sync)

6.17 Stub vs Skeleton

Stub/Proxy	Skeleton
Client side	Server side
Marshals request	Unmarshals request, calls servant

SECTION 7 — IMPORTANT DIAGRAMS (draw-quickly versions)

D1. Distributed system

```
[Node A]──┐
[Node B]──┼── NETWORK (messages only) ──  looks like ONE system
[Node C]──┘
```

D2. Client–Server / P2P

```
C1─┐                    P1──P2
C2──┼──► SERVER         │ ╲╱ │
C3─┘                    P4──P3
```

D3. Software layers with middleware

```
Applications, services
Middleware (RMI, RPC, CORBA)
Operating system
Computer & network hardware      (Platform = OS + HW)
```

D4. Fundamental models

```
Interaction (latency, bandwidth, jitter, drift; sync/async)
Failure (omission, arbitrary, timing)
Security (threats: eavesdrop/tamper/replay/DoS; defence: crypto, auth, access control)
```

D5. RMI architecture

```
Client → Proxy → Comm.module ═net═ Comm.module → Dispatcher → Skeleton → Servant
         (marshal)   +Remote ref module                          (unmarshal, call)
```

D6. RPC

```
Client → Client stub ═ RPC runtime ═ Server stub → Server procedure
```

D7. Java RMI

```
Server: obj = new Impl(); registry.rebind("Name", obj)
Client: stub = registry.lookup("Name"); stub.method()
REGISTRY (1099) sits between both.
```

D8. Publish–subscribe

```
Publishers ─► EVENT SERVICE ─► Subscribers
```

D9. Happened-before timeline & Lamport timestamps (see 02 §5.5)

```
P1: a(1)──b(2)──────────────c(7)
P2: d(1)────e(3)──f(4)
P3: g(1)──────────h(5)──i(6)
```

D10. Consistent vs inconsistent cut

```
P1 ──●──●─┃─────
P2 ──●────┃─●──   send outside, receive inside = INCONSISTENT
```

D11. Chandy–Lamport snapshot (marker flow)

```
Initiator: record state → MARKER on all out-channels → record in-channels
Other:     1st MARKER → record state, chan=∅, send MARKERs, record other in-channels
           later MARKER → chan = messages recorded since
```

D12. Central-server ME

```
P ──request──► Coordinator ──grant──► P ─(CS)─ release ──►
```

D13. Token ring ME `P1→P2→P3→P4→P1` (token circulates)

D14. Ricart–Agrawala

```
Pi: WANTED → multicast REQUEST(T,i) → wait N−1 REPLIES → HELD → RELEASED → send deferred replies
```

D15. Bully message flow

```
P2 ELECTION→P3,P4,P5 ; OK from P3,P4 ; … ; P4 COORDINATOR→all
```

D16. Ring election

```
P1→P2→P3→P4→P1  ELECTION(id max so far) … own id returns → ELECTED circulates
```

D17. Reliable multicast/hold-back queue

```
Network → [hold-back queue] → delivery queue → application
```

D18. Consensus

```
proposals v1,v2,v3 → protocol → all decide same d
```

SECTION 8 — IMPORTANT ALGORITHMS (one-page pseudo-code)

A1. Lamport clock

```
Internal:  L = L + 1
Send:      L = L + 1; send(m, L)
Receive:   L = max(L, t) + 1
```

A2. Vector clock

```
Before event: V[i]++
Send: attach V
Receive (t): V[j] = max(V[j], t[j]) for all j; V[i]++
```

A3. Cristian

```
T0 = local time; ask server; get Ts; T1 = local time
new time = Ts + (T1 - T0)/2
```

A4. Berkeley

```
Master polls all → computes average offset (ignore outliers) → sends adjustment to each
```

A5. NTP offset

```
θ = ((t1 − t0) + (t2 − t3))/2 ;  δ = (t3 − t0) − (t2 − t1)
```

A6. Chandy–Lamport (see 02 §9.4)

A7. Central-server mutual exclusion

```
request → server: if free grant else queue;  release → server: grant to head of queue
```

A8. Token ring mutual exclusion

```
wait for token; enter CS; pass token to next (also if no need)
```

A9. Ricart–Agrawala

```
ENTER:  state=WANTED; multicast REQUEST(T,i); wait N−1 REPLY; state=HELD
ON REQUEST(Tj,j): if HELD or (WANTED and (Ti,i)<(Tj,j)) queue else REPLY
EXIT:   state=RELEASED; REPLY to all queued
```

A10. Lamport ME (REQUEST → REPLY → enter when head of queue & all replies → RELEASE) messages 3(N−1)

A11. Bully

```
detect failure: send ELECTION to higher ids
 if no OK within timeout → I am coordinator: send COORDINATOR to all
 else wait for COORDINATOR (higher process runs its own election)
```

A12. Ring election (Chang–Roberts)

```
start: participant=true; send ELECTION(myid)
recv ELECTION(id): id>myid → forward ; id<myid & !participant → send ELECTION(myid);
                   id<myid & participant → discard ; id==myid → I win, send ELECTED(myid)
```

A13. Reliable multicast (R-multicast)

```
B-multicast(g,m) ; on B-deliver(m) first time: if sender ≠ me B-multicast(g,m); R-deliver(m)
```

A14. Total ordering with sequencer

```
sender → sequencer: (m); sequencer assigns seq s, multicasts (m,s); receivers deliver in order of s
```

A15. Flooding consensus (crash, f failures)

```
for round 1..f+1: multicast newly learnt values; add received values
decide d = min(Values)
```

A16. Message complexity cheat-sheet

Algorithm	Messages
Central ME	3
Ring ME	1..N−1 (+ continuous)
Ricart–Agrawala	2(N−1)
Lamport ME	3(N−1)
Maekawa	≈3√N
Bully best / worst	N−2 / O(N²)
Ring election best / worst	2N / 3N−1
Chandy–Lamport	one marker per channel: up to N(N−1)
Flooding consensus	(f+1) rounds × O(N²)

MID-SEM QUESTION BANK (Unit-I + Unit-II ONLY)

How to use: Part 1 has **only questions** – attempt them on paper first. Part 2 has **complete model answers** with the same numbers: every question from A to I is fully answered, with diagrams and tables written inside the answer. Tags: 📝 = question is based on a topic emphasized in your BPIT assignment (not copied) · 📘 = syllabus topic · ➕ = additional/exam-oriented. *These are practice questions written from the syllabus; not leaked or guaranteed questions.*

PART 1 — QUESTIONS

A. 1-MARK / VERY SHORT (30)

A1. Define a distributed system. A2. Name the three core characteristics of a distributed system. A3. What is transparency? (name any two types) A4. Give one example each of client–server and peer-to-peer systems. A5. What is grid computing? A6. Cloud computing follows which billing model? A7. What is middleware? A8. Name the two categories of system models. A9. What is latency? A10. Expand TCP and UDP. Which is connectionless? A11. What is an internetwork? A12. Name two IPC mechanisms. A13. What is marshalling? A14. What is a remote interface? A15. Which Java interface must every remote interface extend? A16. What is the default port of the RMI registry? A17. What is the role of a stub? A18. What does RPC stand for? A19. What is a notification? A20. What is an omission failure? A21. What is a Byzantine failure? A22. Define clock skew. A23. Define clock drift. A24. State the happened-before rules (names only). A25. State the Lamport rule for receiving a message. A26. What is a critical section? A27. Name two election algorithms. A28. What does the Bully algorithm elect? A29. What are the three conditions of consensus? A30. What is a consistent cut?

B. 2-MARK QUESTIONS (20)

B1. 📝 Define a distributed system and list four characteristics. B2. 📝 Differentiate between client–server and peer-to-peer in two points. B3. State any four advantages of distributed systems. B4. Why do we need system models? B5. 📝 Differentiate Architectural and Fundamental models (two points). B6. Write short note on synchronous and asynchronous distributed systems. B7. 📝 Why is internetworking needed? Name two devices used. B8. Distinguish TCP and UDP (two points). B9. What is distributed shared memory? B10. 📝 What are remote objects and remote object references? B11. What is the role of the RMI registry? B12. State the three invocation semantics. B13. What are events and notifications? Give an example. B14. 📝 What is meant by "no global clock"? Why is it a problem? B15. 📝 Explain process state and event. B16. Differentiate between Lamport and vector clocks (two points). B17. 📝 What are clock drift and skew? How are they related? B18. State the requirements of distributed mutual exclusion. B19. Give message complexity of Ricart–Agrawala, central server and Lamport ME algorithms. B20. What are the types of ordering in multicast?

C. 3-MARK QUESTIONS (12)

C1. 📝 Explain the characteristics of distributed systems with examples. C2. 📝 Explain client–server architecture with a diagram, advantages and disadvantages. C3. Explain the interaction model with latency, bandwidth and jitter. C4. Explain omission, arbitrary and timing failures. C5. 📝 Explain message passing as an IPC mechanism. C6. 📝 What is a stub and a skeleton? How does a remote call proceed? C7. Explain RPC with a neat diagram. C8. 📝 Explain the happened-before relation with an example. C9. 📝 Explain Cristian's algorithm with example. C10. 📝 What is global state? List its components. C11. Explain publish–subscribe model. C12. Explain reliable multicast properties.

D. 5-MARK QUESTIONS (12)

D1. 📝 Compare client–server, peer-to-peer, grid and cloud computing. D2. 📝 Explain architectural models of distributed systems with diagrams. D3. 📝 Explain networking and internetworking and the role of protocols in distributed applications. D4. 📝 Compare message passing and shared memory. D5. 📝 Explain distributed objects and RMI: remote object, remote interface, stub/proxy and invocation steps. D6. Compare RPC and RMI. D7. 📝 Explain Lamport logical clock with rules and example. D8. 📝 Explain need of physical clock synchronisation; discuss drift, skew, delay. D9. 📝 Explain Chandy–Lamport snapshot algorithm. D10. 📝 Explain Bully algorithm with example. D11. 📝 Explain Ricart–Agrawala algorithm. D12. 📝 Explain consensus and its conditions with an example.

E. 7 / 8 / 10-MARK LONG ANSWERS (8)

E1. 📝 Explain system models: need, architectural models and fundamental models with diagrams. E2. 📝 Design a simple Java RMI application (calculator): architecture, remote interface, server, client, registry, execution steps, code and output. E3. 📝 Explain Interprocess communication in detail; compare message passing and shared memory. E4. 📝 Explain global states, consistent cut and the distributed snapshot algorithm with a P1–P2–P3 example. E5. 📝 Explain distributed mutual exclusion: requirements and four algorithms with message complexity. E6. 📝 Explain election algorithms: Bully and Ring, with examples and comparison. E7. 📝 Explain consensus in distributed systems: requirements, process failures, communication delays, Byzantine generals, related problems. E8. Explain Synchronization: physical clock algorithms (Cristian, Berkeley, NTP) and logical clocks.

F. NUMERICAL / PROBLEM-BASED (8)

F1. 📝 P1: events a, send m1→P2 (b), receive m3 (c). P2: d, receive m1 (e), send m2→P3 (f). P3: g, receive m2 (h), send m3→P1 (i). Assign Lamport timestamps. F2. A process has clock 14 and receives a message timestamped 20; another has clock 30 and receives timestamp 25. Find new clocks. F3. Using Cristian's algorithm: T0 = 12:00:00.000, T1 = 12:00:00.600, server time = 12:00:10.200. Find client's new time and the accuracy bound. F4. 📝 Berkeley: master 2:00, slaves 1:50, 2:05, 2:20 (slave times as clock readings; master included in average). Find each adjustment. F5. Clock drift rate $\rho = 2\times10^{-6}$. Skew allowed 2 ms. How often to resynchronise? How much does a clock drift in one day? F6. Ricart–Agrawala with N = 8: messages per CS entry? Lamport ME? Central server? F7. Bully algorithm: processes P1–P6, P6 fails, P3 detects. List messages sent and the winner. Count messages. F8. Ring election: ring

P1→P4→P2→P6→P3→P1 (order as arranged), IDs = numbers, coordinator failed, P2 starts. Show messages until winner.

G. DIAGRAM-BASED (8)

G1. Draw and explain the RMI architecture (proxy, dispatcher, skeleton, communication module). G2. 📝 Draw the software-layer/middleware architecture of a distributed system. G3. Draw the message sequence for an RPC call. G4. 📝 Draw a timeline showing happened-before and concurrent events for 3 processes. G5. 📝 Draw a consistent and an inconsistent cut. G6. 📝 Draw message flow of Chandy–Lamport snapshot for P1, P2, P3. G7. 📝 Draw message flow of Bully election with 5 processes. G8. Draw the hold-back queue for ordered multicast.

H. COMPARE / DIFFERENTIATE (10)

H1. 📝 Client–server vs peer-to-peer. H2. 📝 Message passing vs shared memory. H3. 📝 RMI vs RPC. H4. Synchronous vs asynchronous DS. H5. 📝 Logical vs physical time. H6. Lamport vs vector clocks. H7. 📝 Bully vs Ring algorithm. H8. Cristian vs Berkeley. H9. Token-based vs non-token-based mutual exclusion. H10. Crash vs Byzantine failures.

I. CASE-STUDY QUESTIONS (5)

I1. 📝 A college wants a Student Information Service that lets any lab computer call a method to fetch a student's record from a central server. Design it using Java RMI. I2. 📝 Three bank branch servers P1, P2, P3 hold balances 500, 300, 200. Design a method to record the total consistently while transfers are happening. I3. 📝 Five replicated database servers; one may crash. How do they agree to commit or abort? Discuss challenges. I4. A file server is shared by 6 clients that all update the same file. Choose and justify a mutual-exclusion scheme. I5. In a cluster of 7 servers the coordinator crashes. Show how a new coordinator is chosen with the Bully algorithm and with the ring algorithm.

PART 2 — MODEL ANSWERS

A. 1-MARK ANSWERS

A1. A system whose components on networked computers communicate and coordinate **only by passing messages**. A2. Concurrency, no global clock, independent failures. A3. Hiding the distribution of components from users; e.g., access transparency, location transparency (also replication, failure, mobility, concurrency, performance, scaling). A4. Client–server: web browser–web server. P2P: BitTorrent. A5. Pooling geographically distributed heterogeneous resources of multiple organisations into a virtual supercomputer. A6. Pay-per-use (on-demand, metered). A7. Software layer between OS and applications masking heterogeneity (e.g., RMI, CORBA). A8. Architectural models and fundamental models. A9. Delay between start of transmission at sender and start of receipt at receiver. A10. Transmission Control Protocol, User Datagram Protocol; UDP is connectionless. A11. A network of networks connected by routers acting as a single logical network. A12. Message passing and shared memory (sockets, message queues, DSM). A13. Converting data structures into a flat byte sequence for transmission (and reverse = unmarshalling). A14. Specification of the methods of a remote object that clients may invoke remotely. A15. `java.rmi.Remote`. A16. 1099. A17. Client-side proxy that marshals the call and forwards it to the server, hiding remoteness. A18. Remote Procedure Call. A19. A message describing an event, sent to interested subscribers. A20. A process or channel fails to perform an action (crash or message loss). A21. Arbitrary behaviour: wrong values, contradictory messages, corruption. A22. Difference between readings of two clocks at the same instant. A23. Rate at which a clock deviates from a perfect reference clock. A24. (1) Same-process order, (2) send → receive, (3) transitivity. A25. `L = max(L, t) + 1`. A26. Code segment that accesses a shared resource and must be executed by one process at a time. A27. Bully and Ring algorithms. A28. The alive process with the highest ID. A29. Termination, agreement, validity (integrity). A30. A cut in which every receive event's send event is also inside the cut (no orphan messages).

B. 2-MARK ANSWERS

B1. Def (A1). Characteristics: concurrency (many users act simultaneously), no global clock (each node has own clock), independent failures (one node can fail alone), resource sharing/heterogeneity/openness/scalability. B2. C-S: fixed roles, central server (SPOF). P2P: symmetric roles, decentralised, no SPOF. B3. Resource sharing, scalability, reliability/availability, better performance, cost-effectiveness, geographical distribution. B4. To make assumptions explicit, identify common design issues, reason about correctness/feasibility of algorithms, and generalise across systems. B5. Architectural: structure/placement of components (client-server, P2P). Fundamental: abstract properties – interaction, failure, security. B6. Synchronous: known bounds on execution time, message delay, drift → timeouts reliable. Asynchronous: no bounds (Internet) → can't distinguish slow from crashed. B7. To connect heterogeneous networks so hosts communicate globally (Internet). Devices: router, gateway (also bridge/switch). B8. TCP – connection-oriented, reliable, ordered. UDP – connectionless, unreliable, faster. B9. DSM gives the illusion of one shared address space over physically separate memories using page faults and message passing. B10. Remote object: object whose methods can be invoked from other processes. Remote object reference: system-wide identifier (address, port, time, object number, interface) used to invoke it. B11. Name service: server binds a name to the stub, client looks it up to obtain a stub (port 1099). B12. Maybe, at-least-once, at-most-once (Java RMI: at-most-once). B13. Event: occurrence of interest (e.g., stock price change). Notification: message about it sent to subscribers via event service. B14. Each computer has its own clock; clocks drift, so events on different machines can't be ordered by their timestamps → need logical clocks/sync algorithms. B15. State = values of a process's variables; event = action (internal/send/receive) that changes state. B16. Lamport: one integer, $a \to b \Rightarrow L(a) < L(b)$ only. Vector: N integers, $a \to b \Leftrightarrow V(a) < V(b)$, detects concurrency. B17. Skew = difference of two clocks at an instant; drift = rate of deviation. Drift makes skew grow with time (skew $\leq 2\rho\Delta t$). B18. Safety (one in CS), liveness (no deadlock/starvation), ordering/fairness (happened-before order). B19. RA = $2(N-1)$; central = 3; Lamport = $3(N-1)$. B20. FIFO, causal, total ordering.

C. 3-MARK ANSWERS (complete)

C1. Explain the characteristics of distributed systems with examples.

Definition: A distributed system is a system in which components located on networked computers communicate and coordinate their actions **only by passing messages**.

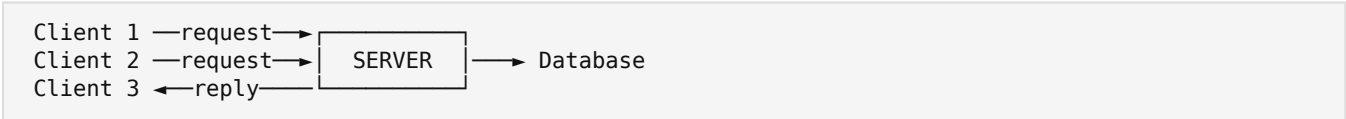
Characteristics: 1. **Concurrency:** Many programs run at the same time on different computers and share resources. *Example:* thousands of users booking IRCTC tickets at the same moment. 2. **No global clock:** Every computer has its own clock and clocks drift apart, so there is no single notion of "correct time". Coordination is done by messages. *Example:* two bank servers stamp the same transfer with different times. 3. **Independent failures:** One computer or network link can fail while others keep running, and the others may not even know about the failure. *Example:* one Google server crashes but search still works. 4. **Resource sharing:** Hardware, software and data are shared over the network. *Example:* shared printer, Google Drive. 5. **Heterogeneity:** Different hardware, OS, networks and languages work together (hidden by middleware). *Example:* Android phone talking to a Linux server. 6. **Openness:** New services can be added using published interfaces. *Example:* HTTP/REST APIs. 7. **Scalability:** The system keeps working well as users grow. *Example:* YouTube. 8. **Transparency:** The user sees one system, not many computers. *Example:* a URL doesn't reveal the server's location.

Conclusion: The first three (concurrency, no global clock, independent failures) are the core characteristics and are the reason synchronization algorithms (Unit-II) are needed.

C2. Explain client–server architecture with a diagram, advantages and disadvantages.

Definition: Client–server is an architecture in which **client processes send requests** to **server processes**, which manage resources and **send back replies**.

Diagram:



Working: 1. The client sends a request (for example an HTTP GET for a web page). 2. The server receives the request and processes it (possibly querying a database). 3. The server sends a reply with the result. 4. The client displays or uses the result.

Advantages: - Central control of data, so security and backup are easy. - Clear roles make the system easy to design and manage. - Clients can be simple (thin).

Disadvantages: - The server is a **single point of failure**. - The server becomes a **bottleneck** under heavy load. - Scaling needs expensive powerful servers or replication.

Examples: web browser and web server, e-mail (SMTP/IMAP), ATM and bank server, database server.

C3. Explain the interaction model with latency, bandwidth and jitter.

Definition: The interaction model describes **how processes interact by messages**, the **performance of communication channels**, and the problem of **time** in a distributed system.

1. Performance of communication channels - **Latency:** delay between the start of a message's transmission by the sender and the start of its receipt by the receiver. It includes network delay, operating-system delay and queueing. - **Bandwidth:** total amount of data that can be transmitted per unit time (for example 100 Mbps). - **Jitter:** variation in the time taken to deliver a series of messages. It matters for audio and video, where frames must arrive at regular intervals. - Transmission time ≈ **latency + (message length ÷ bandwidth)**.

2. Computer clocks: each process has its own clock with a **drift rate**, so local times differ and there is no global time.

3. Two variants: | Synchronous system | Asynchronous system | |---|---| | Known bounds on execution time, message delay and clock drift | No bounds on any of these | | Timeouts can detect failure | Cannot tell a slow process from a crashed one | | Example: real-time control | Example: the Internet |

4. Event ordering: because clocks are not reliable, events are ordered by causality (happened-before, logical clocks).

C4. Explain omission, arbitrary and timing failures.

The **failure model** defines the ways in which processes and channels can fail.

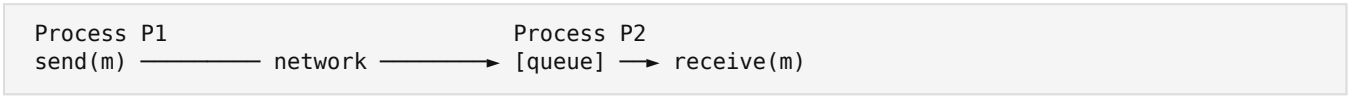
Failure	Description	Example
Omission – process (crash)	Process halts and does nothing more. If others can detect it reliably it is called *fail-stop*	Server power failure
Omission – channel	Message is lost: *send-omission* (lost in sender buffer), *channel omission* (lost in network), *receive-omission* (lost in receiver buffer)	Packet dropped by a congested router
Arbitrary (Byzantine)	Process or channel behaves in any way: sends wrong or contradictory values, corrupts or duplicates messages, omits steps. Worst type	Hacked node sending false data
Timing	Response is outside the allowed time interval: clock failure, performance failure. Only meaningful in **synchronous** systems	Video frame arrives too late

Masking failures: a system can hide failures, for example replication hides a crash, and a checksum converts a corrupted message into an omission (the message is discarded and re-sent). **Reliable communication** = **validity** (every sent message is eventually delivered) + **integrity** (the received message is identical to the one sent, with no duplicates).

C5. Explain message passing as an IPC mechanism.

Definition: In message passing, processes communicate by **explicitly sending and receiving messages** over a channel; they share no variables. The primitives are `send(destination, message)` and `receive(source, message)`.

Diagram:



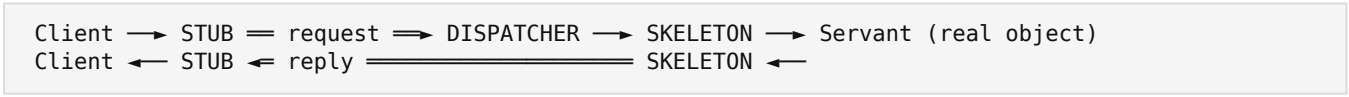
Important concepts: - **Synchronous (blocking):** the sender waits until the message is received; receive waits until a message arrives. - **Asynchronous (non-blocking):** the sender continues after the message is copied to a buffer. - **Sockets:** endpoint = (IP address, port). **UDP datagram sockets** are unreliable and have no connection; **TCP stream sockets** are reliable and ordered. - **Marshalling:** converting data into a byte stream in an agreed external format (Java serialization, XML, JSON) so different machines understand it. - **Direct or indirect:** directly to a process, or via a mailbox or message queue.

Advantages: works between different computers; processes are isolated from each other (more secure); synchronisation is built into send/receive; scalable. **Limitations:** slower because of copying and network delay; the programmer must handle message formats, lost messages and failures. **Applications:** chat applications, client–server systems, MPI for parallel programs, message queues (RabbitMQ, Kafka).

C6. What is a stub and a skeleton? How does a remote call proceed?

- **Stub (proxy):** client-side local object that implements the same remote interface as the remote object. It **marshals** the method identifier and arguments into a request message, sends it, waits, and **unmarshals** the reply. It makes the remote call look local.
- **Skeleton:** server-side object that receives the request, **unmarshals** the arguments, calls the method on the real object (servant), then **marshals** the result and sends it back. A **dispatcher** chooses the right skeleton method.

Diagram:

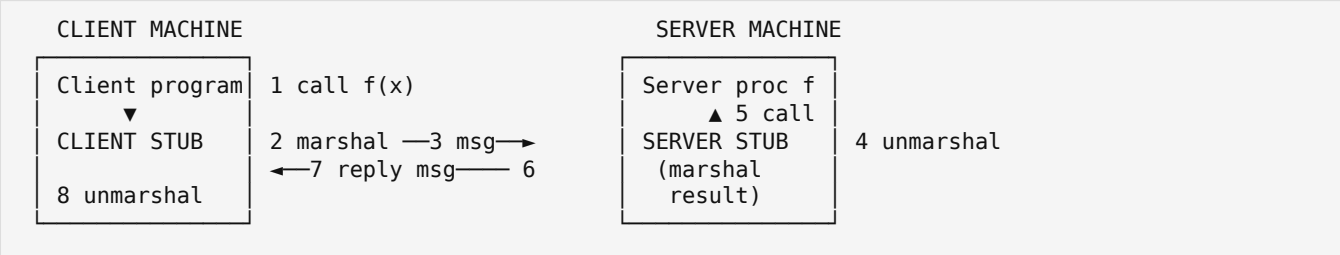


Steps of a remote call: 1. The client calls a method on the stub like a normal call. 2. The stub marshals the method id and parameters. 3. The communication module sends the request over the network. 4. At the server, the dispatcher selects the method and passes the request to the skeleton. 5. The skeleton unmarshals the arguments and invokes the servant. 6. The servant executes and returns the result to the skeleton. 7. The skeleton marshals the result and sends the reply. 8. The stub unmarshals the result and returns it to the client.

C7. Explain RPC with a neat diagram.

Definition: Remote Procedure Call (RPC) allows a program to **call a procedure that executes on another computer** as if it were a local procedure; the network communication is hidden by stubs.

Diagram:



Steps: 1. The client calls the client stub with normal parameters. 2. The client stub marshals the parameters into a message. 3. The RPC runtime and OS send the message to the server. 4. The server stub unmarshals the parameters. 5. The server stub calls the actual procedure. 6. The procedure returns its result, and the server stub marshals it. 7. The reply message is sent back. 8. The client stub unmarshals the result and returns it to the client.

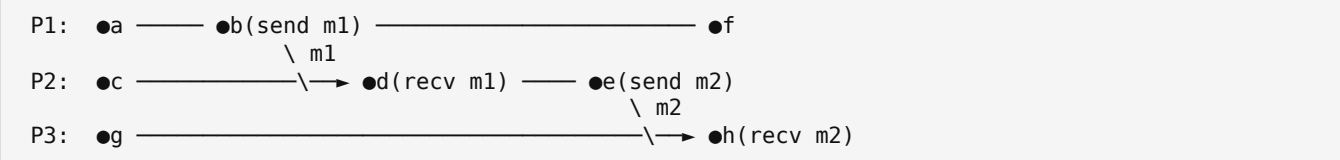
Supporting parts: IDL (Interface Definition Language) describes procedures, and an IDL compiler (for example rpcgen) generates the stubs. A **binder/portmapper** helps the client locate the server. **XDR** is a common data representation. **Issues:** no pointers (pass by value), failures (lost request or reply, server crash), invocation semantics (maybe, at-least-once, at-most-once). **Examples:** Sun RPC (used by NFS), gRPC.

C8. Explain the happened-before relation with an example.

Definition (Lamport, 1978): The happened-before relation (→) captures the **causal order** of events in a distributed system. It is defined by three rules: 1. **HB1:** if a and b are events in the **same process** and a occurs before b, then a → b. 2. **HB2:** if a is the **sending** of message m and b is the **receipt** of m, then a → b. 3. **HB3 (transitivity):** if a → b and b → c, then a → c.

If neither a → b nor b → a, the events are **concurrent** (a ∥ b).

Example:



- a → b (HB1), b → d (HB2, message m1), d → e (HB1), e → h (HB2, m2).
- By transitivity: a → d, a → h, b → h.
- **Concurrent:** a ∥ c (no path between them), f ∥ d, g ∥ b.

Key point: → is a **partial order**; it shows potential causality, not real time.

C9. Explain Cristian's algorithm with example.

Purpose: external clock synchronisation of a client with a **time server** that receives UTC. **Assumption:** the round-trip time is short compared with the required accuracy, and the request and reply take roughly equal time.

Steps: 1. The client records its local time **T0** and sends a request to the time server. 2. The server replies with its current time **Ts**. 3. The client records the reply arrival time **T1**. Round trip time RTT = T1 − T0. 4. The client sets its clock to **Ts + RTT/2**. 5. Accuracy = ± (RTT/2 − min), where min is the minimum one-way delay.

Diagram:

```
Client                                Time server
 T0 ──── request ──────────────────►
    ◄──────── reply (Ts) ────────────
 T1        new time = Ts + (T1 − T0)/2
```

Example: T0 = 10:00:00.000, T1 = 10:00:00.800, Ts = 10:00:05.300. RTT = 0.8 s → new time = 10:00:05.300 + 0.400 = **10:00:05.700**; accuracy ± 0.4 s (taking min ≈ 0).

Limitations: the server is a single point of failure, and a faulty server gives wrong time. Fixes: use several servers, and take the reading with the smallest RTT.

C10. What is global state? List its components.

Definition: The global state of a distributed system is the **set of local states of all processes together with the states of all communication channels** at some instant.

Components: 1. **Local state of each process (S1, S2, …, Sn):** the values of its variables, for example the account balance at each bank server. 2. **State of each channel (Cij):** the messages that have been **sent but not yet received** (in transit).

GS = {S1, S2, …, Sn} ∪ {Cij for every channel}.

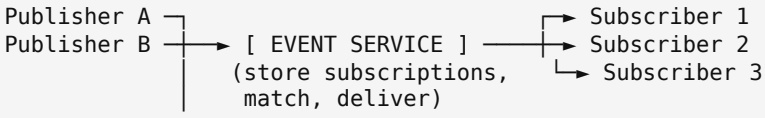
Consistent global state: a state that could actually have happened. It corresponds to a **consistent cut**: every message received in the state was also sent in the state (no orphan message).

Why it is needed: deadlock detection, termination detection, distributed garbage collection, checkpointing and recovery, and debugging. **Difficulty:** there is no global clock, so all processes cannot record their states at the same instant. The Chandy–Lamport snapshot algorithm solves this.

C11. Explain the publish–subscribe model.

Definition: Publish–subscribe is a **distributed event-based** model in which **publishers** generate events, **subscribers** register interest in types of events, and an **event service** delivers **notifications** of matching events to subscribers.

Diagram:



Working: 1. Subscribers `subscribe(event type or filter)` with the event service. 2. A publisher `publish(event)`. 3. The event service matches the event against subscriptions. 4. It `notify`s every matching subscriber, asynchronously.

Types of subscription: topic/channel-based, content-based (filter on attributes, for example price > 500), type-based. **Features:** asynchronous; publishers and subscribers do not know each other (decoupled in space and time); heterogeneous. **Advantages:** loose coupling, scalable, real-time updates, no polling. **Disadvantages:** delivery and ordering guarantees are hard; the broker can be a bottleneck; hard to debug. **Examples:** stock price alerts, YouTube and mobile push notifications, MQTT for IoT, Apache Kafka.

C12. Explain reliable multicast properties.

Multicast sends one message to all members of a group. **Reliable multicast** gives the following guarantees (Coulouris): 1. **Integrity:** a correct process delivers a message **at most once**, and only if it was actually multicast by some process (no duplicates, no spurious messages). 2. **Validity:** if a correct process multicasts message m, it will **eventually deliver m** itself. 3. **Agreement:** if a correct process delivers m, then **all other correct processes** in the group eventually deliver m ("all or none").

Implementation (R-multicast over B-multicast):

```
To R-multicast m:          B-multicast(g, m)
On B-deliver(m) at q:      if m not received before:
                               if q ≠ sender: B-multicast(g, m)   // re-send to all
                               R-deliver(m)
```

Because every receiver re-multicasts m, even if the original sender crashes midway, all correct processes still get m. **Cost:** $O(N^2)$ messages. More efficient versions use sequence numbers, piggy-backed acknowledgements and negative acknowledgements (NACKs).

D. 5-MARK ANSWERS (complete)

D1. Compare client–server, peer-to-peer, grid and cloud computing.

Introduction: These are four common examples (styles) of distributed systems. They differ in who controls resources, the roles of the nodes, and their purpose.

Brief definitions: - **Client–server:** clients request services from central servers. *Example:* web browser and web server. - **Peer-to-peer (P2P):** every node (peer) is both client and server and shares resources directly. *Example:* BitTorrent. - **Grid computing:** geographically spread, heterogeneous computers of many organisations are pooled to work like a virtual supercomputer on large problems. *Example:* CERN LHC Computing Grid, SETI@home. - **Cloud computing:** computing services (servers, storage, software) are delivered over the Internet on demand, on a pay-per-use basis. *Example:* AWS, Microsoft Azure, Gmail.

Diagrams:



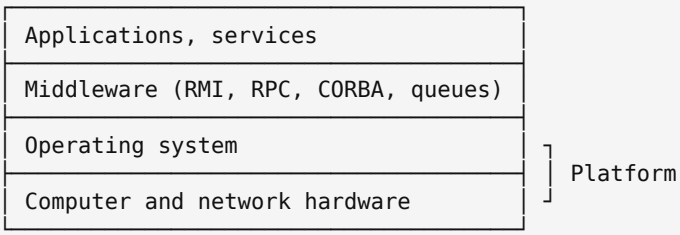
Comparison table: | Feature | Client–Server | Peer-to-Peer | Grid | Cloud | |---|---|---|---|---| | Control | Centralised (server) | Decentralised | Distributed across organisations (virtual organisation) | Centralised by provider | | Roles | Fixed: client and server | Equal: every peer is both | Resource providers and users | Provider and customer | | Main purpose | Access services and data | Share files and resources | Solve very large computations | Rent IT resources on demand | | Scalability | Limited by server | High | Very high | Very high, elastic | | Single point of failure | Yes (server) | No | Scheduler, partly | No (redundant data centres) | | Ownership | One organisation | Individual users | Many organisations | Cloud provider | | Resources | Mostly homogeneous | Heterogeneous | Heterogeneous | Virtualised pool | | Cost model | Own servers | Free | Shared/project based | Pay-per-use | | Security | Easy to control | Hard | Hard (trust between organisations) | Managed by provider | | Applications | Web, e-mail, banking | BitTorrent, blockchain | Scientific simulation, genome research | Web hosting, SaaS apps, storage |

Conclusion: Client–server is simple but centralised; P2P removes the central server; grid pools the power of many organisations for big jobs; cloud offers elastic, pay-per-use services and is the most common today.

D2. Explain architectural models of distributed systems with diagrams.

Definition: An architectural model describes the **structure** of a distributed system: its **components**, where they are **placed** on the network, and their **relationships and roles**.

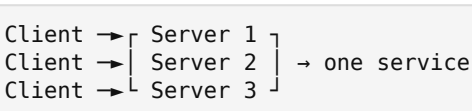
1. Software layers and middleware



Middleware hides heterogeneity and gives one programming model to applications.

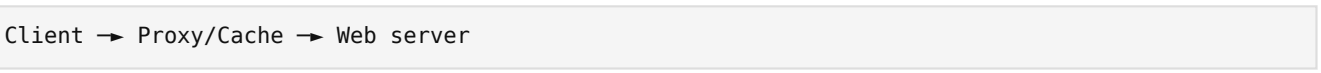
2. Client–server: clients send requests, servers reply. A server can itself be a client of another server (for example a web server calling a database).

3. Multiple servers: one service is provided by several servers that cooperate by **partitioning** data (each holds part of it) or **replicating** it (each holds a copy, for availability).



Example: DNS, Google search clusters.

4. Proxy servers and caches: a cache keeps recently used objects close to the client; a proxy server shares a cache among many clients. This reduces latency and load.



5. Peer processes (P2P): all processes have similar roles and cooperate without a server (BitTorrent).

6. Mobile code: code is downloaded from a server and runs at the client (JavaScript, applets). It gives fast interaction but carries security risk.

7. Mobile agents: a running program (code + data) moves from computer to computer to do a task for the user. It reduces communication but is hard to secure.

8. Thin clients: the client only displays; the processing happens on a powerful server (cloud desktop).

Conclusion: The choice of architecture affects performance, reliability, scalability and security. Most real systems combine several of these models.

D3. Explain networking and internetworking and the role of protocols in distributed applications.

Definitions: - **Network:** a set of computers and devices connected so that they can exchange data. - **Internetwork:** a **network of networks**: many different networks joined by **routers/gateways** so they work as one logical network (the Internet). - **Protocol:** a set of rules for the format, order and meaning of messages exchanged between computers.

Types of network: | Type | Range | Example | |---|---|---| | LAN | Building or campus | Ethernet, Wi-Fi | | MAN | City | Metro Ethernet | | WAN | Country or world | Internet backbone | | Wireless |

Varied | Wi-Fi, 4G/5G |

Protocol layering (TCP/IP):

```
Application   HTTP, FTP, SMTP, DNS        ← messages
Transport     TCP (reliable) / UDP (fast) ← ports, end-to-end
Internet      IP (addressing, routing)    ← packets
Link          Ethernet, Wi-Fi             ← frames
```

Each layer uses the services of the layer below, which hides complexity.

Internetwork diagram:

```
LAN-1 [PC]—[PC]—[Router]═ WAN ═[Router]—[Laptop]—[Phone] LAN-2
```

Role in distributed applications: 1. **Connectivity:** carries the messages between components; without a network there is no distributed system. 2. **Protocols give a common language:** heterogeneous machines can talk (IP addresses, standard formats). 3. **TCP gives reliability** (ordering, retransmission) for RPC/RMI and the web; **UDP gives speed** for streaming and DNS. 4. **Routing** lets messages cross many networks worldwide. 5. **Performance parameters** such as latency, bandwidth and jitter directly affect distributed applications. 6. **Security protocols** (TLS/SSL) protect messages.

Conclusion: Networks and internetworks provide the communication infrastructure, and protocols make communication standard, reliable and possible between different systems.

D4. Compare message passing and shared memory.

Message passing: processes communicate by explicit `send` and `receive` of messages; no memory is shared. **Shared memory:** processes communicate by reading and writing a **common memory area**, with locks or semaphores for synchronisation. In a distributed system it is emulated by **Distributed Shared Memory (DSM)**.

Diagrams:

```
Message passing:   P1 ── send(m) ──network──► receive(m) P2
Shared memory:     P1 ──write──► [ SHARED MEMORY ] ◄──read── P2
```

Comparison: | Point | Message passing | Shared memory | |---|---|---| | Working | Explicit send/receive | Read/write common variables | | Location of processes | Same or different computers | Same computer (or DSM) | | Speed | Slower (copying, network) | Faster (direct access) | | Synchronisation | Built in (blocking receive) | Programmer uses locks/semaphores | | Data copy | Data copied between address spaces | No copy needed | | Main risk | Lost messages, format errors | Race conditions, data corruption | | Scalability | High | Low | | Isolation/security | Good | Poor | | Implementation | Sockets, MPI, message queues, RPC/RMI | shm, mmap, threads, DSM | | Applications | Client–server apps, chat, clusters | Multithreaded programs, producer–consumer buffer |

Advantages and limitations: - Message passing works over networks and is safer, but is slower and needs the programmer to handle messages. - Shared memory is fast and simple to use, but only works on one machine and needs careful locking.

Conclusion: Distributed systems mainly use **message passing** because separate computers have no physical common memory.

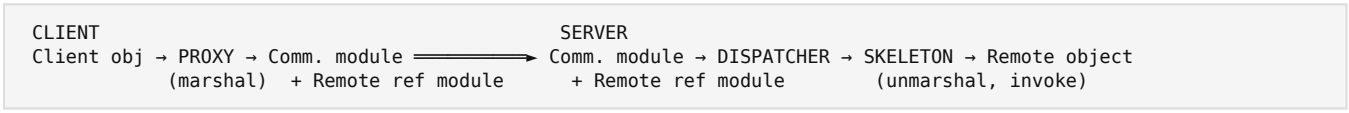
D5. Explain distributed objects and RMI: remote object, remote interface, stub/proxy and invocation steps.

Distributed object: an object whose methods can be called by processes on **other computers**. The object model is extended across the network.

Key terms: - **Remote object:** an object that can receive remote invocations. - **Remote object reference:** a unique identifier for the remote object across the whole system (IP address + port + time + object number + interface). It can be passed as an argument or result. - **Remote interface:** lists the methods of a remote object that can be called remotely (in Java, an interface extending `Remote`). - **Stub/proxy:** the client-side representative that marshals calls. - **Skeleton/dispatcher:** the server side, which unmarshals and calls the object.

RMI (Remote Method Invocation): calling a method of a remote object as if it were local.

Architecture:



Invocation steps: 1. The client calls the method on the proxy. 2. The proxy marshals the method id and arguments into a request. 3. The communication module sends the request. 4. The server's remote reference module finds the local object. 5. The dispatcher selects the method and the skeleton unmarshals the arguments. 6. The skeleton calls the remote object's method. 7. The result is marshalled into a reply and sent back. 8. The proxy unmarshals the result and gives it to the client.

Differences from a local call: parameters are copied (no pointers); there are extra failures (`RemoteException`); the semantics are at-most-once in Java RMI. **Example:** `double r = calc.add(5, 3);` executes on the server and returns 8.

D6. Compare RPC and RMI.

RPC: calls a **procedure** on a remote machine as if it were local (procedural style). **RMI:** calls a **method of a remote object** (object-oriented style). RMI is the OO counterpart of RPC.

Similarities: both hide the network with stubs, use marshalling and the request–reply protocol, and have the same failure semantics issues.

Feature	RPC	RMI
Programming paradigm	Procedural	Object-oriented
What is called	Remote function/procedure	Method of a remote object

Feature	RPC	RMI
Parameters	Simple data types, by value; no pointers	Objects (serialised, by value) and remote object references
Language	Language-independent (C, via IDL)	Java RMI works only Java-to-Java
Interface definition	IDL file (rpcgen)	Java interface extending `Remote`
Client/server components	Client stub / server stub	Stub (proxy) / skeleton
Naming service	Portmapper / binder	RMI registry (port 1099)
Object features	None	Inheritance, polymorphism, dynamic class loading
Garbage collection	Not needed	Distributed garbage collection
Overhead	Lower	Higher (serialisation)
Security	Depends on implementation	Java security manager
Example	Sun RPC used by NFS, gRPC	Java RMI, CORBA (similar idea)

Conclusion: RPC suits simple procedural services in any language; RMI suits Java object-oriented applications where objects and references need to be passed.

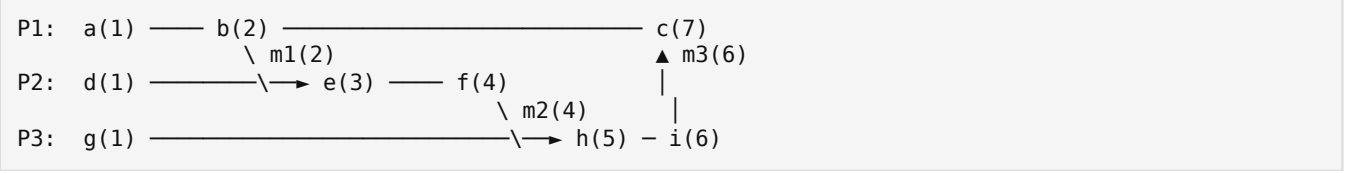
D7. Explain Lamport logical clock with rules and example.

Definition: A Lamport logical clock is a counter L at each process that gives every event a timestamp such that **if a → b then L(a) < L(b)**. It orders events by causality without physical clocks.

Why needed: distributed systems have no global clock, and physical clocks drift, so real time cannot order events on different machines.

Rules: - **LC1:** before every event (internal or send), L = L + 1. - **LC2:** when sending message m, attach the timestamp t = L. - **LC3:** when receiving (m, t): L = max(L, t) + 1.

Example (clocks start at 0):



Event	Calculation	L
a	0 + 1	1
b (send m1)	1 + 1	2
d	0 + 1	1
e (recv m1, t = 2)	max(1, 2) + 1	3
f (send m2)	3 + 1	4

Event	Calculation	L
g	0 + 1	1
h (recv m2, t = 4)	max(1, 4) + 1	5
i (send m3)	5 + 1	6
c (recv m3, t = 6)	max(2, 6) + 1	7

Properties: - a → b ⇒ L(a) < L(b). - The converse is **false**: L(d) = 1 < L(b) = 2 but d and b are concurrent. - **Total order:** break ties using the process id, (L, pid).

Limitations: cannot detect concurrent events (vector clocks solve this); timestamps are not real time.

D8. Explain the need for physical clock synchronisation; discuss drift, skew and delay.

Need: 1. Correct **timestamps** of transactions (banking, e-commerce, stock trading). 2. Ordering of **log files** when auditing or debugging. 3. **Security:** Kerberos tickets and certificates have expiry times. 4. **File systems and tools:** `make` compares file modification times. 5. **Real-time scheduling** and timeouts.

Clock skew: the difference between the readings of two clocks at the same instant. *Example:* C1 = 10:00:05, C2 = 10:00:02 → skew = 3 s.

Clock drift: the rate at which a clock gains or loses time compared with a perfect clock, caused by the quartz crystal, temperature and ageing. A typical drift rate is $\rho \approx 10^{-6}$ s/s, which is about **86.4 ms per day**. - Drift causes skew to grow: two clocks can differ by up to **2ρΔt** after time Δt. - To keep skew below δ, resynchronise at least every **δ / 2ρ** seconds. *Example:* δ = 1 ms, $\rho = 10^{-6}$ → every 500 s.

Communication delay: the time value sent by a server is already old when it arrives, and the delay is variable and unknown (queueing, routing, asymmetric paths). So the receiver cannot know the exact current server time.

Challenges: - Clocks cannot be set backwards (this would break ordering), so they are slowed down gradually instead. - Time servers may fail or give wrong time. - Network delays are unpredictable. - Perfect synchronisation is impossible.

Solutions: - **Cristian's algorithm:** time = Ts + RTT/2. - **Berkeley algorithm:** the master averages the clocks of the group. - **NTP:** a hierarchy of time servers across the Internet. - **Logical clocks:** when only the order of events matters.

D9. Explain the Chandy–Lamport snapshot algorithm.

Purpose: to record a **consistent global state** (process states + channel states) of a running distributed system without stopping it.

Assumptions: channels are FIFO, reliable (no loss) and one-directional; the graph of processes is strongly connected; there are no failures during the snapshot; any process can start the

snapshot. A special control message called a **MARKER** is used.

Algorithm: 1. **Initiator:** records its own local state, sends a MARKER on every outgoing channel, and starts recording messages on every incoming channel. 2. **On receiving a MARKER on channel c:** - **If the process has not yet recorded its state:** it records its state, sets the state of c = empty, sends MARKERs on all outgoing channels, and starts recording its other incoming channels. - **Otherwise:** it stops recording c. The state of c = all messages received on c since it recorded its own state. 3. The snapshot ends when every process has received a MARKER on every incoming channel.

Example (bank): P1 = ₹500, P2 = ₹300, P3 = ₹200 (total ₹1000). 1. P1 sends ₹50 to P2 (m1), so P1 = 450. P1 then starts the snapshot: records **S1 = 450** and sends markers. 2. P3 sends ₹20 to P1 (m2), so P3 = 180, before P3 gets a marker. 3. P2 receives m1 (P2 = 350), then P1's marker: records **S2 = 350**, C12 = ∅. 4. P3 receives the marker: records **S3 = 180**. 5. P1 receives m2 while recording C31, then P3's marker: **C31 = {₹20}**.

```
P1 — send m1 —[record 450, MARKER→]——— recv m2 — recv MARKER(C31)
P2 ————————— recv m1 — recv MARKER —[record 350, MARKER→]
P3 — send m2 — recv MARKER —[record 180, MARKER→]
```

Recorded: 450 + 350 + 180 + 20 (in transit) = **₹1000** ✓ consistent.

Complexity: one marker per channel, so O(E) messages (N(N−1) if fully connected). **Uses:** deadlock detection, termination detection, checkpointing. **Limitation:** needs FIFO channels; the recorded state may not have actually occurred at any single instant, but it is a consistent, reachable state.

D10. Explain Bully algorithm with example.

Purpose: to elect a new **coordinator** (the alive process with the **highest ID**) when the current one fails.

Assumptions: the system is synchronous (timeouts are reliable); every process knows the IDs of all other processes; message delivery is reliable; processes fail only by crashing.

Messages: ELECTION (starting an election), **OK/ANSWER** (reply from a higher process), **COORDINATOR** (announcing the winner).

Steps: 1. Process P notices the coordinator has failed and sends ELECTION to all processes with a **higher ID**. 2. If no one answers within the timeout, P wins and sends COORDINATOR to all processes. 3. If a higher process answers OK, P stops and waits for a COORDINATOR message. 4. Every higher process that received ELECTION replies OK and starts its own election. 5. Finally the highest alive process gets no answer and announces itself as coordinator.

Example (P1–P5, P5 crashes, P2 notices):

```
P2 —ELECTION→ P3, P4, P5(✗)
P3 —OK→ P2      P4 —OK→ P2               (P2 stops)
P3 —ELECTION→ P4, P5(✗)      P4 —OK→ P3  (P3 stops)
P4 —ELECTION→ P5(✗) ..... timeout, no reply
P4 —COORDINATOR(4)→ P1, P2, P3           (P4 is the new coordinator)
```

Message complexity: - **Best case:** the second-highest process detects the failure and sends **N − 2** COORDINATOR messages. - **Worst case:** the lowest process detects the failure, giving **O(N²)** messages.

Advantages: simple; the highest-ID process always wins; handles recovered processes (a recovered high-ID process "bullies" its way back). **Disadvantages:** many messages; needs a synchronous system; every process must know all IDs; a wrong timeout can produce two coordinators.

D11. Explain the Ricart–Agrawala algorithm.

Purpose: distributed mutual exclusion **without a token or coordinator**, using multicast and **Lamport timestamps**.

Assumptions: N processes with unique IDs; each process knows all the others; reliable message delivery; no failures. **States of a process:** RELEASED (not interested), WANTED (wants the critical section, CS), HELD (inside the CS).

Algorithm:

```
Init:      state = RELEASED
Enter CS:  state = WANTED
           multicast REQUEST(T, pi) to all other processes
           wait until N−1 REPLY messages are received
           state = HELD
On REQUEST(Tj, pj) at pi:
           if state == HELD or (state == WANTED and (Ti, pi) < (Tj, pj))
                 queue the request (defer the reply)
           else  send REPLY to pj
Exit CS:   state = RELEASED
           send REPLY to all queued requests
```

The smaller timestamp has higher priority; ties are broken by the smaller process id.

Example (P1, P2, P3): P1 requests with (5, P1) and P2 requests with (7, P2) at the same time; P3 is RELEASED. 1. P3 replies to both. 2. P2 receives P1's request: (7, P2) > (5, P1), so P2 replies to P1. 3. P1 receives P2's request: (5, P1) < (7, P2), so P1 **defers** its reply. 4. P1 now has 2 replies and **enters the CS**. 5. When P1 exits, it sends the deferred REPLY to P2, and **P2 enters the CS**.

```
P1 —REQ(5)—► P2, P3        P2 —REQ(7)—► P1, P3
P3 —REPLY—► P1, P2         P2 —REPLY—► P1   ⇒ P1 enters CS
P1 exits —REPLY—► P2                        ⇒ P2 enters CS
```

Message complexity: 2(N − 1) per entry (N − 1 requests + N − 1 replies). Synchronisation delay = 1 message time. **Advantages:** fully distributed; fair (timestamp order); free from deadlock and starvation; no separate release messages. **Disadvantages:** many messages; the crash of any process blocks others (no reply); every process takes part in every decision.

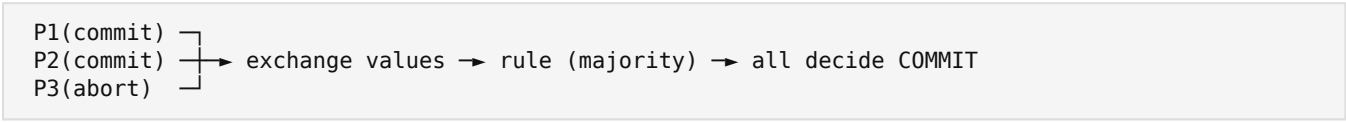
D12. Explain consensus and its conditions with an example.

Definition: In the consensus problem, each process **proposes a value**, and all correct processes must **agree on (decide) the same single value**, even if some processes fail.

Need: committing or aborting a distributed transaction, keeping replicas consistent, leader election, total-order multicast, blockchain, fault-tolerant control (aircraft computers).

Conditions: 1. **Termination:** every correct process eventually decides a value. 2. **Agreement:** all correct processes decide the **same** value. 3. **Validity (integrity):** if all correct processes proposed the same value v, then the decision is v. (The decided value must be one that was proposed.)

Example: three database replicas decide whether to commit a transaction. P1 = commit, P2 = commit, P3 = abort. They exchange votes and apply the same rule (majority), so all decide **commit**. - Agreement ✓: all decide commit. - Validity ✓: commit was proposed. - Termination ✓: all decide after the exchange.



Solution with crash failures (synchronous system): with f possible crashes, run **f + 1 rounds**. In each round every process multicasts the new values it has learned; at the end each decides with the same function (for example the minimum or majority).

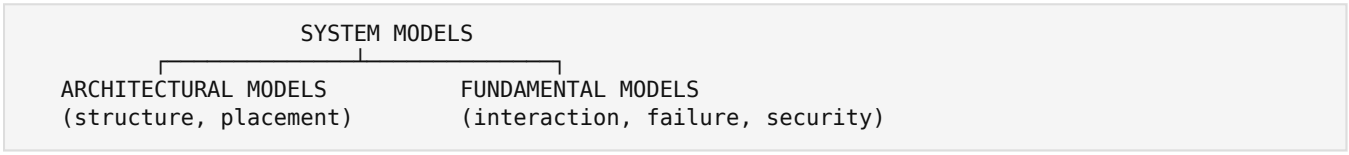
Difficulties: crashes cause partial message delivery; Byzantine processes may lie ($N \geq 3f + 1$ is needed); in an asynchronous system, no deterministic algorithm can guarantee consensus with even one crash (**FLP result**).

E. LONG ANSWERS — 7/8/10 MARKS (complete)

E1. Explain system models: need, architectural models and fundamental models with diagrams.

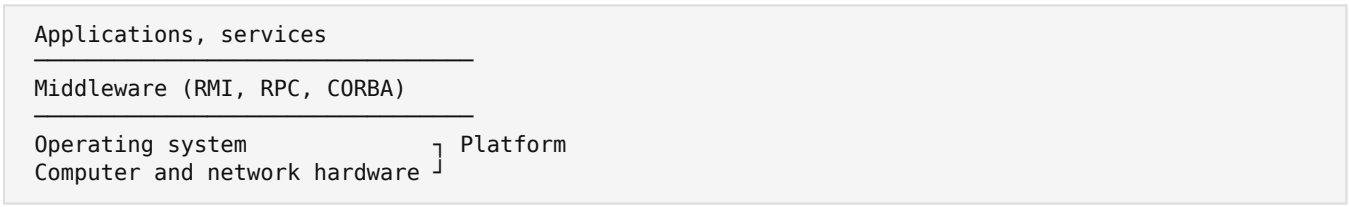
1. Introduction and definition A **system model** is an abstract, simplified description of the important aspects of a distributed system: its components, how they interact, and what assumptions are made about failures and security.

2. Need for system models 1. They **make assumptions explicit** (for example "messages may be lost", "clocks drift"). 2. They show the **common design problems** of all distributed systems. 3. They let us **reason about correctness** and prove whether an algorithm works (or is impossible). 4. They **generalise:** an algorithm proved for a model works for every real system that fits the model. 5. They help **compare** designs.



3. Architectural models These describe how the system is organised: its components, their placement on computers, and their roles.

(a) Software layers



The **platform** (hardware + OS) gives basic communication. **Middleware** hides heterogeneity and gives a common programming model.

(b) System architectures 1. **Client–server:** clients request, servers reply. `Client —request→ Server → (may call another server) Client ←reply—` 2. **Multiple servers:** a service is spread over several servers by **partitioning** (each server holds part of the data) or **replication** (each holds a copy). Examples: DNS, the web. 3. **Proxy servers and caches:** copies of recently used objects are kept near clients to reduce delay and load. `Client → Proxy/Cache → Web server` 4. **Peer processes:** all processes are equal and share resources (BitTorrent). `P1 ←→ P2 ▲ \ / ▲ ▼ / \ ▼ P4 ←→ P3` 5. **Mobile code:** code moves from server to client and runs there (JavaScript). 6. **Mobile agents:** a running program travels between computers, doing work. 7. **Thin clients:** the client only handles the display; the server does the processing.

(c) Design requirements: performance (responsiveness, throughput, load balancing), quality of service (reliability, security, adaptability), and use of caching and replication.

4. Fundamental models These are abstract models describing properties common to **all** distributed systems.

(i) Interaction model - Processes interact only by messages. - **Latency:** delay in delivering a message. **Bandwidth:** data carried per second. **Jitter:** variation in delivery time. - Each computer's clock has a **drift rate**, so there is no global time. - **Synchronous system:** known bounds on execution time, message delay and drift. **Asynchronous system:** no bounds (the Internet). - Event ordering is done by logical time (happened-before).

(ii) Failure model | Failure | Meaning | |---|---| | Omission (process) | Crash; fail-stop if it can be detected | | Omission (channel) | Message lost (send, channel or receive omission) | | Arbitrary (Byzantine) | Any wrong behaviour, including incorrect or malicious values | | Timing | Response outside the time limit (synchronous systems only) | - **Masking:** hide a failure (replication) or turn it into a milder kind (a checksum turns corruption into omission). - **Reliable communication = validity + integrity.**

(iii) Security model - Objects (resources) are protected by access rights; **principals** (users, processes) have authority. - **Enemy/attacker** can read, copy, alter or inject messages. - **Threats:** to processes (spoofing identity), to channels (eavesdropping, tampering, replay), denial of service, mobile code. - **Defences:** cryptography, authentication, secure channels (TLS/SSL), access control, digital signatures.

```
Process A ─── secure channel (encrypted, authenticated) ──► Process B
                  ▲ enemy tries to copy / change messages
```

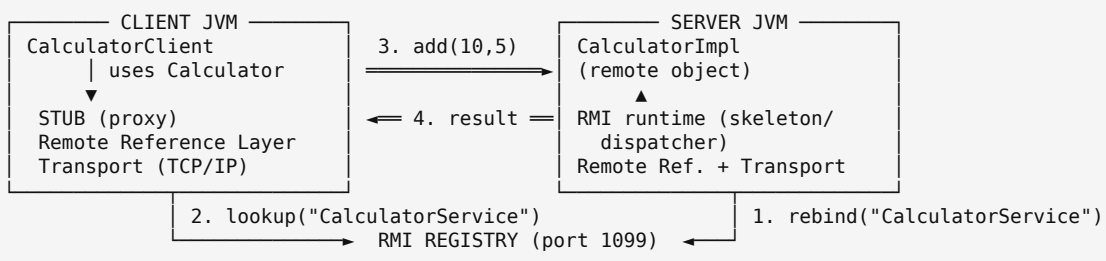
5. Comparison | Architectural model | Fundamental model | |---|---| | Structure and placement of components | Abstract behaviour and assumptions | | Client–server, P2P, proxy, mobile code | Interaction, failure, security | | "How is it built?" | "How does it behave and what can go wrong?" |

6. Conclusion: Architectural models tell us how to organise components; fundamental models tell us what assumptions our algorithms must handle. Together they guide the design of correct and efficient distributed systems.

E2. Design a simple Java RMI application (calculator): architecture, remote interface, server, client, registry, execution steps, code and output.

1. Introduction Java RMI (Remote Method Invocation) lets a Java program in one JVM call methods of an object located in another JVM, usually on another computer, as if the object were local. It uses **stubs** to hide communication, **serialisation** to pass objects, and provides **at-most-once** semantics.

2. Architecture



Layers: - **Stub/skeleton layer:** intercepts calls and marshals data. - **Remote reference layer:** manages references and invocation semantics. - **Transport layer:** TCP connections between the JVMs.

3. Components | Component | Description | |---|---| | Remote interface | `Calculator` extends `java.rmi.Remote`; every method declares `throws RemoteException` | | Remote object | `CalculatorImpl` implements `Calculator` and extends `UnicastRemoteObject` (this exports it so it can receive calls) | | Stub | Client-side proxy, generated automatically (Java 5+); marshals calls | | Skeleton | Server-side receiver; built into the RMI runtime since Java 1.2 | | RMI registry | Naming service on port 1099 that maps a name to a stub | | Server | Creates the object and registers it with `rebind` | | Client | Looks up the stub with `lookup` and calls methods |

4. Code

```
// ---------- Calculator.java (remote interface) ----------
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    double divide(int a, int b) throws RemoteException;
}

// ---------- CalculatorImpl.java (remote object) ----------
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    public CalculatorImpl() throws RemoteException { super(); }
    public int add(int a, int b)       { return a + b; }
    public int subtract(int a, int b)  { return a - b; }
    public int multiply(int a, int b)  { return a * b; }
    public double divide(int a, int b) { return (double) a / b; }
}

// ---------- CalculatorServer.java ----------
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            Calculator calc = new CalculatorImpl();                 // create remote object
            Registry registry = LocateRegistry.createRegistry(1099); // start registry
            registry.rebind("CalculatorService", calc);             // register name
            System.out.println("Calculator Server is ready...");
        } catch (Exception e) { e.printStackTrace(); }
    }
}

// ---------- CalculatorClient.java ----------
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            Calculator calc = (Calculator) registry.lookup("CalculatorService");
            System.out.println("10 + 5 = " + calc.add(10, 5));
            System.out.println("10 - 5 = " + calc.subtract(10, 5));
            System.out.println("10 * 5 = " + calc.multiply(10, 5));
            System.out.println("10 / 5 = " + calc.divide(10, 5));
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

5. Execution steps 1. Compile: `javac *.java`. 2. Start the server: `java CalculatorServer`. It creates the object, exports it (stub created), starts the registry and binds the name "CalculatorService". 3. Start the client: `java CalculatorClient`. It contacts the registry on port 1099 and looks up "CalculatorService", receiving the **stub**. 4. The client calls `calc.add(10, 5)` on the stub. 5. The stub serialises the method and arguments and sends them over TCP. 6. The server's RMI runtime unmarshals them and calls `add` on `CalculatorImpl`. 7. The result 15 is marshalled and returned; the stub unmarshals it and returns it to the client. 8. The client prints the result. The same happens for the other methods.

6. Sample output

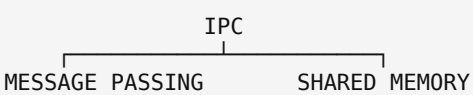
```
Server terminal:  Calculator Server is ready...
Client terminal:  10 + 5 = 15
                  10 - 5 = 5
                  10 * 5 = 50
                  10 / 5 = 2.0
```

7. Advantages and limitations | Advantages | Limitations | |---|---| | Object-oriented and simple | Works only Java-to-Java | | Stubs generated automatically | Slower than a local call | | Passes objects via serialisation | Firewall issues with dynamic ports | | Distributed garbage collection | Registry is very basic |

8. Conclusion: Java RMI hides network programming behind normal method calls, so a calculator on a server can be used from any client as if it were a local object.

E3. Explain interprocess communication in detail; compare message passing and shared memory.

1. Definition Interprocess Communication (IPC) is the set of mechanisms that let processes **exchange data and synchronise** their actions, either on the same computer or on different computers.



2. Message passing - Processes use `send(dest, msg)` and `receive(src, msg)`; no memory is shared. - **Synchronous (blocking):** the sender waits until the receiver takes the message. **Asynchronous (non-blocking):** the sender continues immediately. - **Direct communication:** to a named process. **Indirect:** through a mailbox or message queue. - **Sockets:** communication endpoint = IP address + port. - **UDP datagram communication:** connectionless, unreliable, messages may be lost or out of order; low overhead (DNS, streaming). - **TCP stream communication:** connection-oriented, reliable, ordered byte stream (web, e-mail, RMI). - **Marshalling and external data representation:** data is flattened to bytes in an agreed format (Java serialization, CORBA CDR, XML, JSON) so that heterogeneous machines understand it. - **Request–reply protocol:** doOperation (client), getRequest and sendReply (server). This is the basis of RPC and RMI.

```
Client                          Server
doOperation ── Request ──────► getRequest
  (wait)                        select & execute
           ◄────── Reply ─────── sendReply
```

3. Shared memory - Processes share a common memory area and communicate by reading and writing it. - Access must be synchronised with **locks, semaphores or monitors** to avoid race conditions. - On one computer: OS shared segments (`shmget`, `mmap`) or threads in one process. - In a distributed system: **Distributed Shared Memory (DSM)** gives the illusion of one shared address space; underneath, pages are moved by message passing.

```
P1 ──write──► [ SHARED MEMORY ] ◄──read── P2      (lock / semaphore)
```

4. Comparison | Basis | Message passing | Shared memory | |---|---|---| | Communication | Explicit send/receive | Read/write shared variables | | Where it works | Same or different machines | Same machine (or DSM) | | Speed | Slower | Faster | | Synchronisation | Implicit (blocking calls) | Explicit locks/semaphores | | Data copying | Yes | No | | Programming burden | Message formats, failures | Race conditions, deadlocks | | Scalability | High | Low | | Security/isolation | Good | Weak | | Failure effect | Lost messages | Corrupted shared data | | Examples | Sockets, MPI, RPC/RMI, Kafka | Threads, POSIX shm, DSM |

5. Advantages and limitations - **Message passing:** + works across networks, + isolation, + scalability; − slower, − more programming for messages. - **Shared memory:** + very fast, + natural programming; − limited to one machine, − needs careful synchronisation, − DSM has coherence overhead.

6. Applications - **Message passing:** client–server systems, chat, distributed databases, cluster computing (MPI). - **Shared memory:** multithreaded servers, producer–consumer buffers, multiprocessor scientific programs.

7. Conclusion: Since computers in a distributed system do not share physical memory, **message passing** is the basic IPC method; shared memory is used inside a single machine or emulated through DSM.

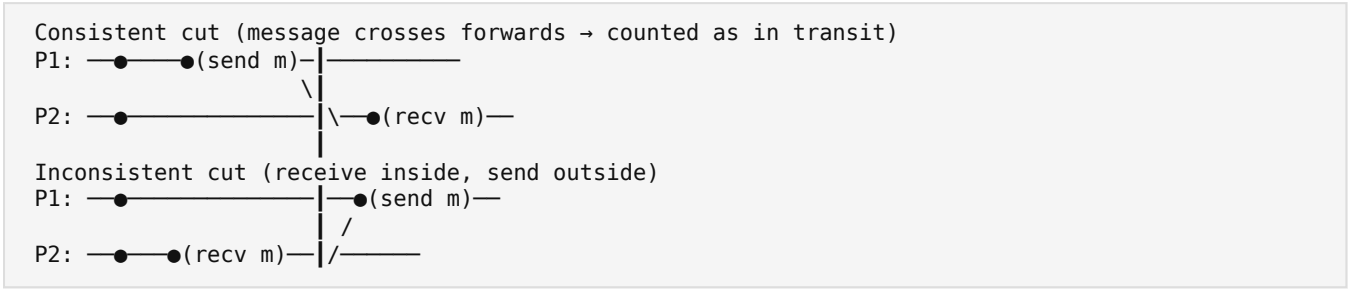
E4. Explain global states, consistent cut and the distributed snapshot algorithm with a P1–P2–P3 example.

1. Global state The **global state** is the collection of the **local states of all processes** plus the **states of all channels** (messages in transit) at an instant. - Local state Si = the values of the variables of process Pi. - Channel state Cij = messages sent by Pi but not yet received by Pj.

2. Why a consistent global state is important 1. **Deadlock detection:** find cycles of waiting processes. 2. **Termination detection:** check that all processes are idle and no messages are in transit. 3. **Distributed garbage collection:** find objects no longer referenced. 4. **Checkpointing and recovery:** restart from a saved consistent state after a failure. 5. **Debugging and auditing:** for example, check that total money is preserved.

Problem: there is no global clock, so processes cannot record their states at exactly the same moment.

3. Cut and consistency - A **cut** is a set of events of the processes, drawn as a line across the timelines (a prefix of each process's history). - A cut is **consistent** if for every event in the cut, all events that happened before it are also in the cut. In practice: **if a receive event is in the cut, its send event must also be in the cut.** - An **inconsistent cut** contains a message that is received but never sent (an **orphan message**), which is impossible in reality.



4. Chandy–Lamport snapshot algorithm Assumptions: FIFO, reliable, one-directional channels; strongly connected processes; no failures during the snapshot; a MARKER control message is used.

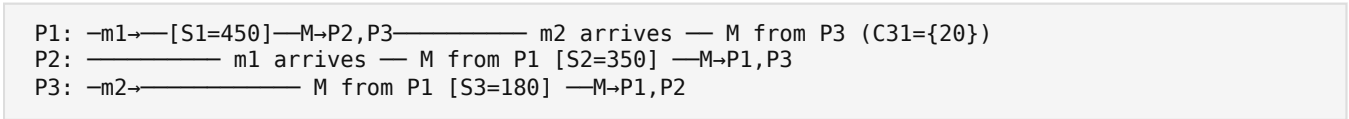
Marker sending rule (process P): record P's state, then send a marker on every outgoing channel before sending any other message.

Marker receiving rule (process Q, on channel c): - **If Q has not recorded its state:** record the state, set the state of c = ∅, follow the marker sending rule, and start recording messages on all other incoming channels. - **Else:** the state of c = messages received on c since Q recorded its state; stop recording c.

The algorithm terminates when every process has received a marker on every incoming channel.

5. Example (P1, P2, P3 bank) Initial balances: P1 = ₹500, P2 = ₹300, P3 = ₹200 (total ₹1000).

Step	Event	Effect
1	P1 sends ₹50 to P2 (m1)	P1 = 450
2	P1 starts snapshot	Records **S1 = 450**; markers on C12, C13; records C21, C31
3	P3 sends ₹20 to P1 (m2)	P3 = 180
4	P2 receives m1	P2 = 350
5	P2 receives marker on C12	Records **S2 = 350**, C12 = ∅; markers on C21, C23
6	P3 receives marker on C13	Records **S3 = 180**, C13 = ∅; markers on C31, C32
7	P1 receives m2 (recording C31), then marker on C31	**C31 = {₹20}**
8	Remaining markers arrive	C21 = C23 = C32 = ∅



Recorded global state: S1 = 450, S2 = 350, S3 = 180, C31 = {₹20}. Total = 450 + 350 + 180 + 20 = **₹1000** ✓. The money is conserved, so the snapshot is consistent.

6. Properties - **Messages:** one marker per channel, O(E); for full connectivity N(N−1). - The application **keeps running** during the snapshot. - The recorded state is consistent and reachable, though it may not have existed at one real instant.

7. Conclusion: The Chandy–Lamport algorithm records a consistent global state using markers, and it is the basis for detecting stable properties such as deadlock and termination.

E5. Explain distributed mutual exclusion: requirements and four algorithms with message complexity.

1. Introduction Distributed mutual exclusion ensures that only **one process at a time** executes its **critical section (CS)** to access a shared resource, using only messages (no shared variables or semaphores). *Examples:* updating a shared file or bank record, using a shared printer.

2. Requirements - **ME1 Safety:** at most one process is in the CS at any time. - **ME2 Liveness:** requests to enter and exit the CS eventually succeed (no deadlock, no starvation). - **ME3 Ordering:** if one request happened-before another, it is granted first (fairness).

Performance measures: messages per entry (bandwidth), client delay, and synchronisation delay (time between one process exiting and the next entering).

3. Central server algorithm

```
P ──REQUEST──► Server ──GRANT──► P  ...CS...  P ──RELEASE──► Server ──GRANT──► next
```

- The server keeps a token and a queue. If the token is free it grants immediately; otherwise it queues the request.
- **Messages: 3** per entry (request, grant, release). Sync delay: 2 messages.
- **Pros:** simple and fair. **Cons:** single point of failure, bottleneck.

4. Ring-based (token ring) algorithm

```
P1 → P2 → P3 → P4 → P1   (the token circulates; only its holder may enter the CS)
```

- A process waits for the token, enters the CS, then passes the token on.
- **Messages:** 1 to N−1 per entry, and the token keeps circulating even when no one needs it.
- **Pros:** no starvation, simple. **Cons:** token loss, a crash breaks the ring, wastes bandwidth.

5. Ricart–Agrawala algorithm - The requester multicasts REQUEST(T, id) and waits for N−1 REPLIES. - A receiver replies immediately unless it is in the CS, or wants the CS and has an earlier timestamp; in that case it defers the reply. - On exit, the process replies to all deferred requests. - **Messages: 2(N−1)**. Sync delay: 1 message. - **Pros:** fully distributed and fair. **Cons:** many messages; any crash blocks the others.

6. Lamport's mutual exclusion algorithm - Each process keeps a queue of requests ordered by timestamp. - The requester sends REQUEST to all; each receiver adds it to its

queue and replies. - A process enters when its request is at the head of its own queue and it has received a later-timestamped message from every other process. - On exit it sends RELEASE to all. - **Messages: 3(N−1)**. Needs FIFO channels.

7. (Extra) Maekawa's voting algorithm: a process asks only its voting set (about √N processes, and any two sets overlap). About **3√N** messages; can deadlock.

8. Comparison (N processes) | Algorithm | Messages per entry | Sync delay | Main problem | |---|---|---|---| | Central server | 3 | 2 | Server crash | | Token ring | 1 … N−1 | 1 … N−1 | Token loss | | Ricart–Agrawala | 2(N−1) | 1 | Any crash blocks | | Lamport | 3(N−1) | 1 | Heavy, needs FIFO | | Maekawa | ≈3√N | 2 | Deadlock |

Example with N = 5: central = 3, RA = 8, Lamport = 12.

9. Conclusion: A central server uses the fewest messages but is a single point of failure; token and permission-based algorithms remove the central server at the cost of more messages. No algorithm tolerates process crashes without extra mechanisms.

E6. Explain election algorithms: Bully and Ring, with examples and comparison.

1. Need for election Many algorithms need a **coordinator** (central-server mutual exclusion, the Berkeley time master, the primary replica). When the coordinator crashes, the remaining processes must choose a new one. An election algorithm chooses one process, usually the alive process with the **highest ID**, and makes all processes agree on it.

Requirements: E1 Safety: every participating process selects the same process (the one with the largest ID). **E2 Liveness:** all processes eventually participate and decide, or crash.

2. Bully algorithm *Assumptions:* synchronous system (reliable timeouts), all IDs known, reliable messages. *Messages:* ELECTION, OK, COORDINATOR. *Steps:* 1. The detecting process sends ELECTION to all higher-ID processes. 2. If there is no reply, it becomes coordinator and sends COORDINATOR to all. 3. If some process replies OK, it waits; each higher process repeats step 1. 4. The highest alive process wins.

Example (P1–P5, P5 crashes, P2 detects):

```
P2 —ELECTION→ P3, P4, P5✗      P3, P4 —OK→ P2
P3 —ELECTION→ P4, P5✗          P4 —OK→ P3
P4 —ELECTION→ P5✗  (timeout)
P4 —COORDINATOR→ P1, P2, P3    ⇒ P4 is the coordinator
```

Messages: best **N−2**, worst **O(N²)**.

3. Ring algorithm (Chang–Roberts) *Assumptions:* logical one-directional ring; each process knows its successor (crashed processes are skipped); works in asynchronous systems. *Steps:* 1. The starter marks itself a participant and sends ELECTION(own id) to its successor. 2. A receiver compares the id: - If the received id is larger: forward it. - If it is smaller and the receiver is not yet a participant: replace it with its own id and forward. - If it is smaller and the receiver is already a participant: discard it. - If it equals the receiver's own id: the receiver has won. 3. The winner sends ELECTED(id) around the ring; every process records the coordinator.

Example (ring P1→P2→P3→P4→P5→P1, P5 crashed so P4→P1; P2 starts):

```
P2: ELECTION(2) → P3: replace → ELECTION(3) → P4: replace → ELECTION(4)
→ P1: forward 4 → P2: forward 4 → P3: forward 4 → P4: own id ⇒ WINNER
P4: ELECTED(4) → P1 → P2 → P3 → P4 (stop)
```

Messages: best **2N**, worst **3N − 1**.

4. Comparison | Feature | Bully | Ring | |---|---|---| | Topology | Any process can reach any other | Logical ring | | Knowledge needed | All IDs | Only the successor | | System model | Synchronous (timeouts) | Asynchronous is fine | | Messages | ELECTION, OK, COORDINATOR | ELECTION, ELECTED | | Best case | N − 2 | 2N | | Worst case | O(N²) | 3N − 1 | | Winner | Highest alive ID | Highest alive ID | | Weakness | Many messages, depends on timeouts | Ring breaks if a process fails during the election; slow |

5. Conclusion: Both elect the highest-ID process. Bully is fast when a high-ID process detects the failure but costs O(N²) messages in the worst case; Ring uses O(N) messages and weaker assumptions but is slower and needs ring maintenance.

E7. Explain consensus in distributed systems: requirements, process failures, communication delays, Byzantine generals and related problems.

1. Definition and need Consensus: every process proposes a value, and all correct processes must **agree on one value**. *Uses:* commit or abort a transaction, keep replicas consistent (Paxos, Raft), elect a leader, total-order multicast, blockchain.

2. Requirements - **Termination:** every correct process eventually decides. - **Agreement:** all correct processes decide the same value. - **Validity/Integrity:** if all correct processes propose v, the decision is v (the decided value was proposed).

3. Example: five replicas vote on commit (1) or abort (0): P1 = 1, P2 = 1, P3 = 1, P4 = 0, P5 = 1. After exchanging votes, all apply the rule "majority", so all decide **1 (commit)**.

4. Consensus with crash failures (synchronous system)

```
Values_i = {v_i}
for round r = 1 .. f+1:
    B-multicast the values not sent before
    add all values received this round to Values_i
decide d_i = min(Values_i)        (or majority)
```

Why f + 1 rounds: at most f processes crash, so at least one round has no crash; after that round all correct processes hold the same set.

Case: 5 processes, at most 1 crash (f = 1) → 2 rounds. 1. Round 1: all send their values; P5 crashes after sending only to P1 and P2. 2. Round 2: P1 and P2 forward P5's value to P3 and P4. 3. Now P1–P4 hold identical sets and apply the same rule, so they agree.

5. Challenges from process failures - A **crash** during a multicast means only some processes get the value. - **Byzantine** processes send different values to different processes. - A process that recovers may have missed messages.

6. Challenges from communication delays - In an **asynchronous** system a slow message looks like a crashed process. - Timeouts may wrongly declare a process dead, or waiting may never end. - **FLP impossibility (1985):** in an asynchronous system, no deterministic algorithm can guarantee consensus if even one process can crash. - Practical solutions: timeouts (partial synchrony), failure detectors, randomisation, and quorum-based protocols such as **Paxos and Raft** (2f + 1 nodes tolerate f crashes).

7. Byzantine Generals Problem A commander sends an order ("attack" or "retreat") to lieutenants; some generals may be traitors. - **BG1 Agreement:** all loyal lieutenants follow the same order. - **BG2 Integrity:** if the commander is loyal, the loyal lieutenants follow his order. - **Result:** with oral (unsigned) messages it is solvable only if **N ≥ 3f + 1**, and f + 1 rounds are needed.

Example: N = 3, f = 1 is impossible. A loyal lieutenant receiving conflicting reports cannot tell whether the commander or the other lieutenant is lying. *Example: N = 4, f = 1 works:*

```
Commander C (loyal) sends "attack" to L1, L2, L3. L3 is a traitor.
L1 hears: C = attack, L2 relays attack, L3 relays retreat → majority = attack
L2 hears: C = attack, L1 relays attack, L3 relays retreat → majority = attack   ✓ agreement
```

8. Related problems | Problem | Meaning | |---|---| | Interactive consistency | All agree on a **vector** of values, one per process | | Byzantine generals | One commander's value must be agreed despite traitors | | Total-order multicast | All deliver messages in the same order (equivalent to consensus) | | Atomic commit (2PC) | All commit or all abort | | Leader election | Agree on one leader | Consensus, Byzantine generals and interactive consistency can each be solved using any of the others, so they are **equivalent** problems.

9. Conclusion: Consensus is the heart of fault-tolerant distributed computing. It needs f + 1 rounds against crashes in synchronous systems and N ≥ 3f + 1 against Byzantine faults, and it cannot be guaranteed in purely asynchronous systems (FLP).

E8. Explain synchronisation: physical clock algorithms (Cristian, Berkeley, NTP) and logical clocks.

1. Introduction Distributed systems have **no global clock**. Synchronisation means making processes agree on **time** (physical clocks) or on the **order of events** (logical clocks).

2. Physical clocks: basic terms - **Skew:** the difference between two clocks at the same instant. - **Drift:** the rate at which a clock deviates from real time ($\rho \approx 10^{-6}$). - **UTC:** the international standard time. - **External synchronisation:** clocks are kept close to UTC. **Internal synchronisation:** clocks are kept close to each other. - Resynchronise every $\delta / 2\rho$ seconds to keep skew $\leq \delta$.

3. Cristian's algorithm (external)

```
Client T0 ──request──► Time server
Client T1 ◄──Ts──────── Time server
new time = Ts + (T1 − T0)/2
```

Example: T0 = 0 ms, T1 = 20 ms, Ts = 10:00:00.500 → new time = 10:00:00.510. It is simple, but the server is a single point of failure.

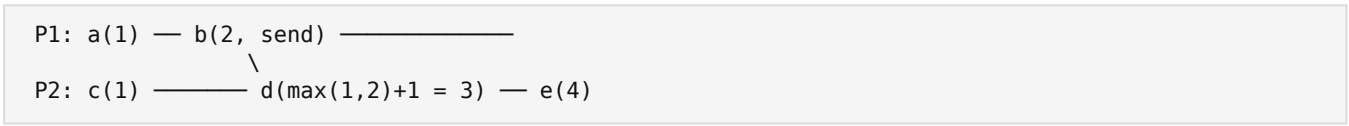
4. Berkeley algorithm (internal) 1. The master polls every slave for its time. 2. It estimates each slave's time (compensating for RTT) and averages them (ignoring faulty values). 3. It sends each machine the **adjustment**, not the absolute time.

Example: master 3:00, slaves 2:50 and 3:25. Offsets 0, −10, +25; average = +5 → target 3:05. Adjustments: master +5, slave 1 +15, slave 2 −20. It needs no UTC source, but the master can fail (a new one is then elected).

5. NTP (Network Time Protocol) - A hierarchy of **strata:** stratum 0 = atomic/GPS clocks, stratum 1 = servers attached to them, stratum 2 syncs from stratum 1, and so on. - Modes: multicast, procedure-call, symmetric. - Offset **θ = ((t1 − t0) + (t2 − t3))/2**; delay **δ = (t3 − t0) − (t2 − t1)**. The sample with the lowest delay is chosen. - Scalable, fault-tolerant and secure; accuracy is tens of milliseconds over the Internet.

6. Logical clocks Happened-before: same process order, send → receive, transitivity. **Lamport clock rules:** increment before each event; send the timestamp; on receive L = max(L, t) + 1. Guarantees a → b ⇒ L(a) < L(b).

Example:



Vector clocks: an array of N counters; on receive take the component-wise maximum and then increment own entry. a → b ⇔ V(a) < V(b), so vector clocks can detect concurrency.

7. Logical vs physical | Physical | Logical | |---|---| | Real time | Order of events only | | Needs synchronisation | No synchronisation needed | | Affected by drift | Not affected | | Cristian, Berkeley, NTP | Lamport, vector clocks |

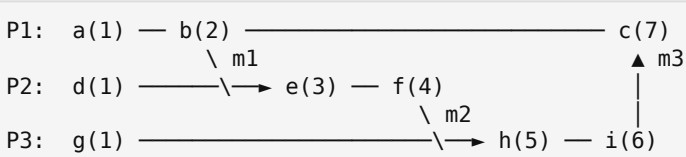
8. Conclusion: Physical clock synchronisation keeps real time approximately correct, while logical clocks give an exact causal order of events. Distributed systems use both depending on the need.

F. NUMERICAL / PROBLEM-BASED ANSWERS (fully worked)

F1. Lamport timestamps for P1 (a, b = send m1, c = recv m3), P2 (d, e = recv m1, f = send m2), P3 (g, h = recv m2, i = send m3).

Rules: increment before each event; attach the clock value when sending; on receive L = max(L, t) + 1.

Event	Process	Type	Working	Timestamp
a	P1	internal	0 + 1	**1**
b	P1	send m1	1 + 1 (m1 carries 2)	**2**
d	P2	internal	0 + 1	**1**
e	P2	receive m1 (t = 2)	max(1, 2) + 1	**3**
f	P2	send m2	3 + 1 (m2 carries 4)	**4**
g	P3	internal	0 + 1	**1**
h	P3	receive m2 (t = 4)	max(1, 4) + 1	**5**
i	P3	send m3	5 + 1 (m3 carries 6)	**6**
c	P1	receive m3 (t = 6)	max(2, 6) + 1	**7**



Ans: a = 1, b = 2, c = 7, d = 1, e = 3, f = 4, g = 1, h = 5, i = 6.

F2. Clock 14 receives timestamp 20; clock 30 receives timestamp 25.

- Case 1: max(14, 20) + 1 = **21**.
- Case 2: max(30, 25) + 1 = **31**. The local clock only moves forward; it never goes back even when the message timestamp is smaller.

F3. Cristian: T0 = 12:00:00.000, T1 = 12:00:00.600, Ts = 12:00:10.200.

- RTT = T1 − T0 = 0.600 s.
- New time = Ts + RTT/2 = 12:00:10.200 + 0.300 = **12:00:10.500**.
- Accuracy = ±(RTT/2 − min) = **±0.3 s** (taking min ≈ 0).

F4. Berkeley: master 2:00, slaves S1 = 1:50, S2 = 2:05, S3 = 2:20 (master included).

Step 1 – offsets from the master: master 0, S1 −10 min, S2 +5 min, S3 +20 min. **Step 2 – average:** (0 − 10 + 5 + 20) / 4 = 15 / 4 = **+3.75 min** (3 min 45 s). **Step 3 – target time:** 2:00 + 3:45 = **2:03:45**. **Step 4 – adjustment = average − own offset:** | Machine | Offset | Adjustment | |---|---|---| | Master | 0 | **+3 min 45 s** | | S1 | −10 min | 3.75 − (−10) = **+13 min 45 s** | | S2 | +5 min | 3.75 − 5 = **−1 min 15 s** | | S3 | +20 min | 3.75 − 20 = **−16 min 15 s** |

All clocks become 2:03:45. Negative adjustments are applied by **slowing the clock down**, not setting it back. The master may also treat S3 (20 min off) as a faulty outlier and exclude it from the average.

F5. ρ = 2 × 10⁻⁶, allowed skew δ = 2 ms.

- Two clocks drift apart by up to 2ρ per second = 4×10^{-6} s per second.
- Resync interval ≤ δ / (2ρ) = 0.002 / (4×10^{-6}) = **500 s** (about 8.3 minutes).
- Drift of one clock in a day = ρ × 86 400 = $2 \times 10^{-6} \times 86\,400$ = **0.1728 s ≈ 173 ms**.

F6. N = 8 processes.

- Ricart–Agrawala = 2(N − 1) = 2 × 7 = **14**.
- Lamport = 3(N − 1) = 3 × 7 = **21**.
- Central server = **3** (request, grant, release).

F7. Bully: P1–P6, P6 fails, P3 detects.

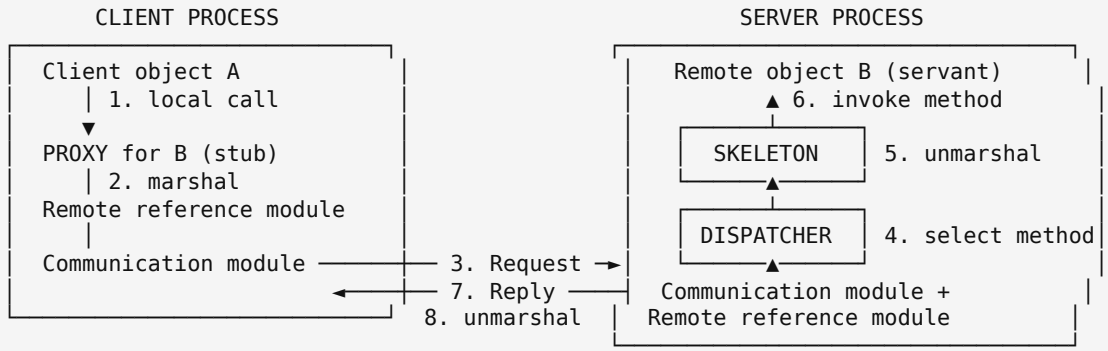
Step	Messages	Count
P3 → ELECTION to P4, P5, P6	3	3
P4, P5 → OK to P3 (P6 silent)	2	5
P4 → ELECTION to P5, P6	2	7
P5 → OK to P4	1	8
P5 → ELECTION to P6 (timeout)	1	9
P5 → COORDINATOR to P1, P2, P3, P4	4	13
Winner: P5 (highest alive ID). **Total = 13 messages.**		

F8. Ring P1→P4→P2→P6→P3→P1; P6 (the coordinator, highest ID) has failed; P2 starts.

Alive ring: P1 → P4 → P2 → P3 → P1 (P6 is skipped). | # | From → To | Message | Reason | |---|---|---|---| | 1 | P2 → P3 | ELECTION(2) | P2 starts | | 2 | P3 → P1 | ELECTION(3) | 3 > 2, P3 not participant → replaces with own id | | 3 | P1 → P4 | ELECTION(3) | 1 < 3 → forward | | 4 | P4 → P2 | ELECTION(4) | 4 > 3 → replaces with own id | | 5 | P2 → P3 | ELECTION(4) | forward | | 6 | P3 → P1 | ELECTION(4) | forward | | 7 | P1 → P4 | ELECTION(4) | forward | | – | P4 | — | receives its own id ⇒ **P4 is the coordinator** | | 8–11 | P4 → P2 → P3 → P1 → P4 | ELECTED(4) | announcement circulates once | **Winner: P4. Total = 7 election + 4 elected = 11 messages.**

G. DIAGRAM-BASED ANSWERS (diagram + explanation)

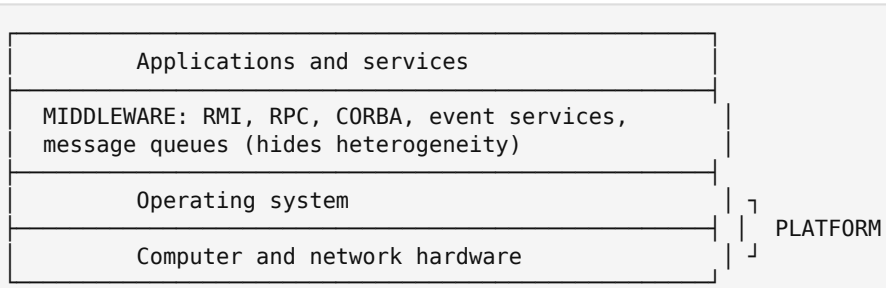
G1. RMI architecture (proxy, dispatcher, skeleton, communication module).



Explanation: - **Proxy (stub):** makes the remote call look local; marshals the request and unmarshals the reply. - **Remote reference module:** translates between local and remote object references using the remote object table. - **Communication module:** carries out the request–reply protocol. - **Dispatcher:** receives the request and selects the right method using the method id. - **Skeleton:** unmarshals the arguments, calls the servant, and marshals the result. - **Servant:** the real object that does the work.

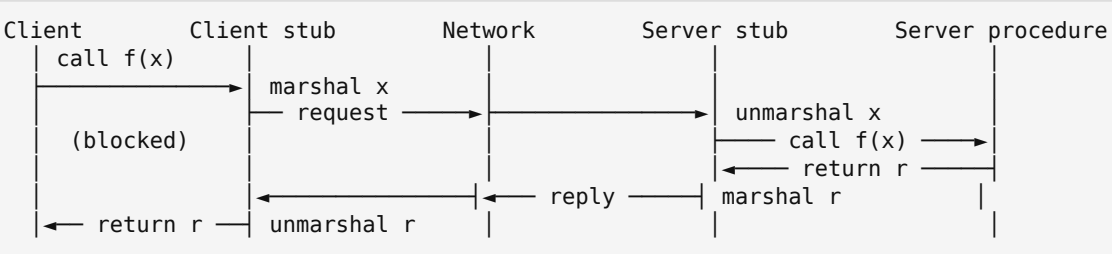
The numbers 1–8 in the diagram give the order of a complete invocation.

G2. Software layers / middleware architecture.



Explanation: - **Platform:** the lowest layers (hardware + OS), for example x86 + Linux. They provide basic communication. - **Middleware:** masks differences in OS, hardware, network and programming language, and gives a uniform model (remote invocation, events, groups). - **Applications:** use middleware services without worrying about distribution. - Each layer uses only the services of the layer below it.

G3. Message sequence for an RPC call.



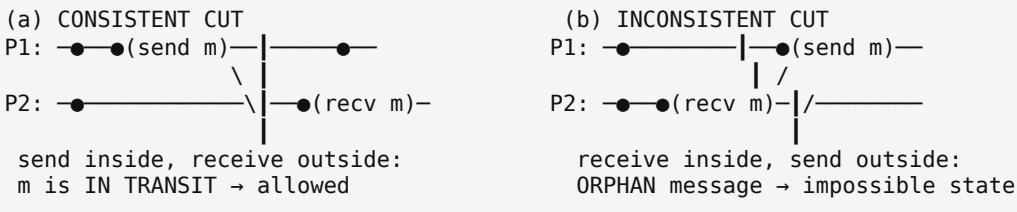
Explanation: the client is blocked while waiting; the stubs handle marshalling; the reply returns along the same path. If the reply is lost, the client stub retransmits according to the invocation semantics.

G4. Timeline showing happened-before and concurrent events (3 processes).



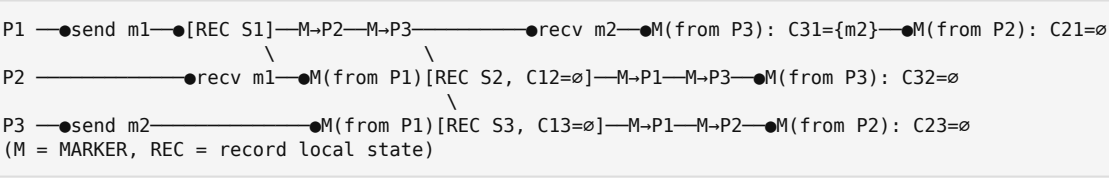
Happened-before: a → b, b → d, d → e, e → h; so a → h and b → h. **Concurrent pairs:** a ‖ c, a ‖ g, f ‖ d, f ‖ e, f ‖ h, c ‖ b. **Rule:** two events are concurrent if no chain of process-order and message arrows connects them.

G5. Consistent and inconsistent cut.



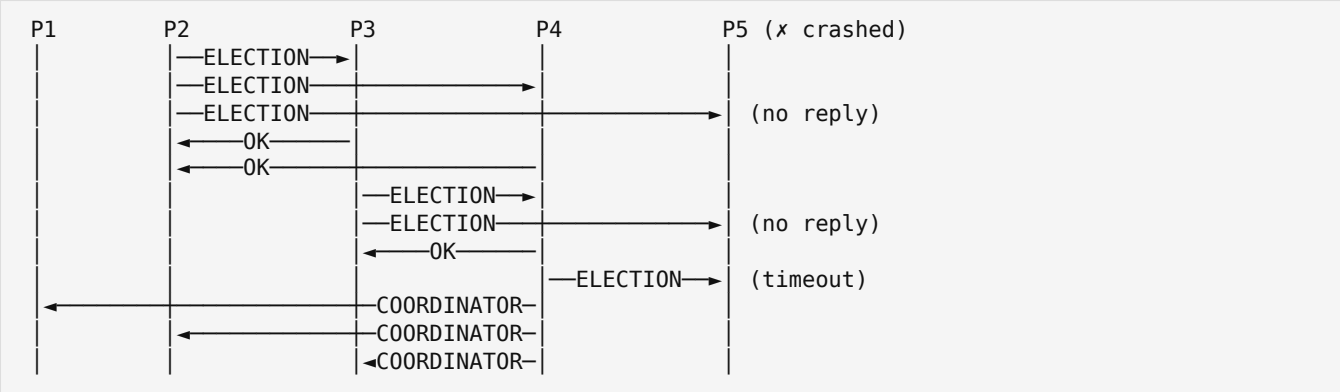
Explanation: - A cut is consistent if, for every receive event inside it, the matching send is also inside. - In (a), m is counted in the channel state. - In (b), P2 has received a message that P1 has "not yet sent", so the state could never have occurred.

G6. Chandy–Lamport snapshot message flow for P1, P2, P3.



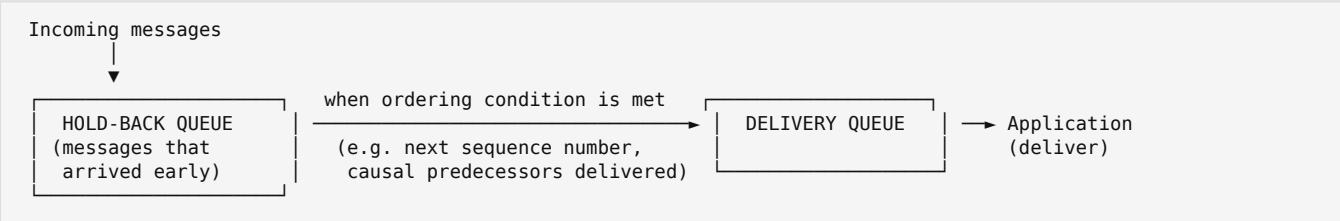
Explanation: P1 initiates; every process records its state on the first marker and forwards markers; messages that arrive after recording but before a channel's marker (m2 on C31) become that channel's state. With the bank example: 450 + 350 + 180 + 20 = ₹1000.

G7. Bully election message flow with 5 processes.



Explanation: P2 detects the failure; higher processes reply OK and take over; P4, the highest alive, gets no reply from P5 and announces itself.

G8. Hold-back queue for ordered multicast.



Explanation: - A received message is not given to the application immediately. It waits in the hold-back queue until the ordering guarantee is met. - **FIFO:** the next sequence number from that sender has arrived. - **Causal:** all causally earlier messages (vector timestamps) have been delivered. - **Total:** the message's agreed sequence number is next. - Then it moves to the delivery queue and is delivered.

H. COMPARE / DIFFERENTIATE ANSWERS (full tables)

H1. Client–server vs peer-to-peer

Basis	Client–server	Peer-to-peer
Roles	Fixed: clients request, server serves	Every peer is both client and server
Control	Centralised	Decentralised
Single point of failure	Yes (server)	No
Scalability	Limited by server capacity	Improves as peers join
Cost	Expensive servers	Low (uses peers' resources)
Security and management	Easy, central	Difficult
Data availability	Depends on server	Depends on peers being online
Examples	Web, e-mail, banking	BitTorrent, blockchain

H2. Message passing vs shared memory

Basis	Message passing	Shared memory
Method	send/receive	Read/write common memory
Works across network	Yes	No (only via DSM)
Speed	Slower	Faster
Synchronisation	Implicit	Explicit locks/semaphores
Scalability	High	Low
Isolation	Good	Poor
Problems	Lost messages, formats	Race conditions
Examples	Sockets, MPI, RMI	Threads, shm, DSM

H3. RMI vs RPC

Basis	RPC	RMI
Paradigm	Procedural	Object-oriented
Invokes	Remote procedure	Method of a remote object
Parameters	Plain data, by value	Objects and remote references
Language	Language-neutral (IDL)	Java only (Java RMI)
Stubs	Client and server stubs	Stub and skeleton
Naming	Portmapper/binder	RMI registry
Garbage collection	No	Distributed GC
Overhead	Lower	Higher
Example	Sun RPC, gRPC	Java RMI

H4. Synchronous vs asynchronous distributed system

Basis	Synchronous	Asynchronous
Process execution time	Known bound	No bound
Message delay	Known bound	No bound
Clock drift	Known bound	No bound
Failure detection	Reliable with timeouts	Not reliable
Algorithms	Easier (e.g. f + 1 round consensus)	Harder (FLP impossibility)
Examples	Real-time embedded/control systems	The Internet

H5. Logical vs physical time

Basis	Physical time	Logical time
Meaning	Real time (UTC)	Order of events
Source	Hardware/atomic clocks	Counters updated on events
Synchronisation needed	Yes	No
Drift and skew	Affected	Not affected
Answers	"What time is it?"	"Which event came first?"
Overhead	Periodic sync messages	Timestamp in each message
Algorithms	Cristian, Berkeley, NTP	Lamport, vector clocks
Uses	Timestamps, expiry, scheduling	Mutual exclusion, causal ordering, debugging

H6. Lamport vs vector clocks

Basis	Lamport clock	Vector clock
Size	One integer	Array of N integers
Receive rule	L = max(L, t) + 1	V[j] = max(V[j], t[j]) for all j, then V[i]++
Property	$a \rightarrow b \Rightarrow L(a) < L(b)$	$a \rightarrow b \Leftrightarrow V(a) < V(b)$
Detects concurrency	No	Yes
Message overhead	Small (O(1))	Larger (O(N))
Total ordering	Easy (add process id)	Not directly
Uses	Mutual exclusion, total ordering	Causal multicast, debugging, conflict detection

H7. Bully vs Ring

Basis	Bully	Ring
Topology	Any-to-any	Logical ring
Knowledge	All IDs	Successor only

Basis	Bully	Ring
Timing	Synchronous (timeouts)	Asynchronous is fine
Messages	ELECTION, OK, COORDINATOR	ELECTION, ELECTED
Best case	$N - 2$	$2N$
Worst case	$O(N^2)$	$3N - 1$
Winner	Highest alive ID	Highest alive ID
Weakness	Message storm, timeout errors	Ring break, slow

H8. Cristian vs Berkeley

Basis	Cristian	Berkeley
Type	External (to UTC)	Internal (within group)
Server role	Passive time server answers	Active master polls slaves
Result	Client sets Ts + RTT/2	Everyone gets an adjustment to the average
UTC receiver needed	Yes	No
Faulty clocks	Server fault affects all	Outliers ignored
Failure	Server is a single point of failure	Master failure → elect new master
Suited for	Clients syncing with a time server	LAN group of machines

H9. Token-based vs non-token-based mutual exclusion

Basis	Token-based	Non-token (permission-based)
Entry rule	Process holding the unique token enters	Process collects permission (replies)
Examples	Token ring, Suzuki–Kasami	Ricart–Agrawala, Lamport, Maekawa
Messages	1 to N	$2(N-1)$, $3(N-1)$, $\approx 3\sqrt{N}$
Safety comes from	Only one token exists	Timestamp ordering / voting
Main problem	Token loss, regeneration	A crashed process never replies
Starvation	Avoided by circulation order	Avoided by timestamps

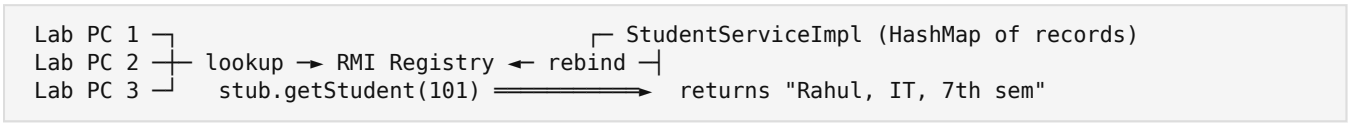
H10. Crash vs Byzantine failures

Basis	Crash failure	Byzantine failure
Behaviour	Process stops	Arbitrary, possibly malicious
Wrong values sent	No	Yes, possibly different to different processes
Detection	Timeouts (synchronous)	Very hard
Processes needed to tolerate f	$N \geq f + 1$ (sync flooding), $2f + 1$ (quorum protocols)	$N \geq 3f + 1$
Rounds (synchronous)	$f + 1$	$f + 1$
Example	Power failure	Hacked node, memory corruption

I. CASE-STUDY ANSWERS (complete)

I1. Student Information Service using Java RMI.

Design: a central server holds student records; lab computers (clients) call `getStudent(roll)` remotely.



Code:

```
// StudentService.java
import java.rmi.*;
public interface StudentService extends Remote {
    String getStudent(int roll) throws RemoteException;
}
// StudentServiceImpl.java
import java.rmi.*; import java.rmi.server.*; import java.util.*;
public class StudentServiceImpl extends UnicastRemoteObject implements StudentService {
    private final Map<Integer, String> db = new HashMap<>();
    public StudentServiceImpl() throws RemoteException {
        db.put(101, "Rahul, IT, Sem VII");
        db.put(102, "Priya, IT, Sem VII");
    }
    public String getStudent(int roll) { return db.getOrDefault(roll, "Not found"); }
}
// StudentServer.java
import java.rmi.registry.*;
public class StudentServer {
    public static void main(String[] a) throws Exception {
        LocateRegistry.createRegistry(1099).rebind("StudentService", new StudentServiceImpl());
        System.out.println("Student server ready");
    }
}
// StudentClient.java
import java.rmi.registry.*;
public class StudentClient {
    public static void main(String[] a) throws Exception {
        StudentService s = (StudentService) LocateRegistry.getRegistry("localhost", 1099)
                                                           .lookup("StudentService");
        System.out.println(s.getStudent(101));
    }
}
```

Output: `Student server ready` (server) and `Rahul, IT, Sem VII` (client). **Execution steps:** start the server (registry + rebind) → the client looks up and gets the stub → it calls `getStudent` → the stub marshals the call → the server executes it → the result is returned. **Benefits:** location transparency; one central copy of the data; clients stay simple. **Improvement:** replicate the server or add a database for reliability.

I2. Consistent total of three bank branches (P1 = 500, P2 = 300, P3 = 200).

Problem: transfers are in progress; reading each branch at a different time may count money twice or miss money in transit. **Solution:** use the **Chandy–Lamport snapshot**: 1. P1 records its balance and sends MARKERs to P2 and P3, then records its incoming channels. 2. Each branch records its balance on its first marker, sends markers, and records its other incoming channels. 3. Money arriving on a channel after recording but before that channel's marker is recorded as **in transit**. 4. Total = Σ recorded balances + Σ in-transit amounts. **Example:** P1

sends ₹50 to P2 before the snapshot; P3 sends ₹20 to P1 during it. Recorded: P1 = 450, P2 = 350, P3 = 180, C31 = 20 → **₹1000** ✓. **Requirements:** FIFO, reliable channels. The banking application keeps running during the snapshot.

I3. Five replicated database servers; one may crash. Agree on commit or abort.

Approach (synchronous, f = 1 crash) – flooding consensus with f + 1 = 2 rounds: 1. Each server proposes its vote (commit or abort) and multicasts it to the others. 2. Round 1: suppose S5 crashes after sending its vote to S1 and S2 only. 3. Round 2: every server multicasts any votes it learned in round 1, so S3 and S4 get S5's vote. 4. All correct servers (S1–S4) now hold the same set of votes. 5. Decision rule (same for all): commit only if all votes are commit, otherwise abort (atomic commit rule), or use a majority. 6. **Agreement:** same set + same rule → same decision. **Validity:** the decision is one of the proposed votes. **Termination:** decided after 2 rounds.

Challenges: - **Crash failures:** partial multicast leads to different views, which is why the extra round is needed. - **Communication delays:** in an asynchronous network a slow server looks crashed; wrong timeouts break agreement (FLP). - **Message overhead:** $O(N^2)$ messages per round. - **Byzantine case:** if the faulty server can lie, $N \geq 3f + 1 = 4$ is needed (5 ✓), and majority voting on relayed values is used.

Practical systems: Paxos or Raft (a majority of 3 out of 5 is enough to decide).

I4. File server shared by 6 clients (mutual exclusion scheme).

Scheme	Messages per update (N = 6)	Failure risk
Central lock server	3	Lock server crash
Token ring	1–5 (+ continuous circulation)	Token loss
Ricart–Agrawala	2 × 5 = 10	Any client crash blocks everyone
Lamport	3 × 5 = 15	Same, plus needs FIFO
Recommended: central lock server (the file server itself can act as coordinator).		
- Fewest messages (3).		
- Simple and fair (FIFO queue).		
- The file server is already a single point of dependency, so the coordinator adds no new risk.		
- If it crashes, run the **Bully algorithm** to elect a new lock manager.		

I5. 7 servers; coordinator P7 crashes. Elect a new one with Bully and Ring.

Bully (P3 detects):

```
P3 —ELECTION→ P4, P5, P6, P7✗        P4, P5, P6 —OK→ P3
P4 —ELECTION→ P5, P6, P7✗            P5, P6 —OK→ P4
P5 —ELECTION→ P6, P7✗                P6 —OK→ P5
P6 —ELECTION→ P7✗ (timeout)
P6 —COORDINATOR→ P1, P2, P3, P4, P5  ⇒ P6 elected
```

Ring (P1→P2→…→P6→P1, P7 skipped; P3 starts):

```
P3: ELECTION(3) → P4 (replace 4) → P5 (replace 5) → P6 (replace 6) → P1 (fwd 6)
→ P2 (fwd 6) → P3 (fwd 6) → P4 → P5 → P6 receives own id ⇒ P6 elected
P6: ELECTED(6) → P1 → P2 → P3 → P4 → P5 → P6
```

Both elect P6, the highest alive process. Bully used 21 messages here (4+3+3+2+2+1+1 ELECTION/OK + 5 COORDINATOR); Ring used 9 ELECTION + 6 ELECTED = 15.

EXTRA PRACTICE QUESTIONS + TOP-30 LIST

> ### 🎯 Claude Predicted Practice Questions
>
> **These are predicted/practice questions based on syllabus and emphasis, not leaked or guaranteed questions.** They are *not* copied from your assignments. Each has a pointer (→) to the section that contains the answer. Numerical answers are given at the end.

PART A — 30 SHORT QUESTIONS (1–2 marks)

Claude Predicted Practice Question — each one.

1. Why is "no shared memory" a defining property of a distributed system? → 01 §1
2. Give two examples of heterogeneity in a distributed system and how middleware handles it. → 01 §2
3. What is location transparency? Give an example. → 01 §2
4. Why is a server called a single point of failure? → 01 §3.1
5. What is a DHT in P2P? (one line) → 01 §3.2
6. Name two real-world grid computing projects. → 01 §3.3
7. List three characteristics of cloud computing. → 01 §3.4
8. What is horizontal scalability? → 01 §4
9. What is meant by "openness" of a distributed system? → 01 §2
10. What is jitter and where does it matter? → 01 §5.3
11. Why can't timeouts detect failures reliably in an asynchronous system? → 01 §5.3
12. Give one threat to a channel in the security model. → 01 §5.3
13. Which device joins two different networks? → 01 §6
14. Why is UDP used for video streaming? → 01 §6
15. What is a socket? → 01 §7.1
16. What is a race condition in shared memory? → 01 §7.2
17. Why can pointers not be passed in RPC? → 01 §10
18. What is an IDL? → 01 §10
19. What is the difference between `bind` and `rebind` in Java RMI? → 01 §12.7
20. What does `UnicastRemoteObject` do? → 01 §12.3
21. What is a topic-based subscription? → 01 §11
22. What is a logical clock? → 02 §5
23. What is a "concurrent event"? Give an example. → 02 §4
24. Why must the receive rule of Lamport clock use `max`? → 02 §5
25. What is UTC? → 02 §7
26. What is a stable predicate? Give an example. → 02 §10
27. What are the three types of events in a process? → 02 §3
28. Why is starvation a problem in mutual exclusion? → 02 §12
29. What is the difference between delivering and receiving a multicast message? → 02 §14

30. State the FLP impossibility result in one line. → 02 §15

PART B — 20 MEDIUM QUESTIONS (3–4 marks)

Claude Predicted Practice Question — each one.

1. Explain the three major challenges caused by the absence of a global clock in a distributed system. → 02 §1, §7
2. "A distributed system is more than a set of networked computers." Justify with reference to transparency and coordination. → 01 §1–2
3. How is cloud computing different from grid computing? Explain with examples. → 01 §3.5
4. Explain the role of middleware in masking heterogeneity. → 01 §5.2
5. Explain caching and proxy servers as architectural elements. → 01 §5.2
6. Explain mobile code and mobile agents with advantages and security concerns. → 01 §5.2
7. Explain the failure model and the notion of masking failures. → 01 §5.3
8. Describe how a synchronous system differs from an asynchronous one in designing failure detectors. → 01 §5.3
9. Why is the request–reply protocol central to RPC and RMI? Explain marshalling and semantics. → 01 §7, §9
10. Explain the three invocation semantics and state which is used by Java RMI and why idempotent operations matter. → 01 §9
11. Why are remote exceptions necessary in RMI but not for local calls? → 01 §8
12. Explain event-based (publish–subscribe) architecture and its advantages over polling. → 01 §11
13. Distinguish between logical clocks and vector clocks using an example where Lamport clocks fail to detect concurrency. → 02 §5–6
14. Explain how skew is bounded by drift rate and how often clocks must be synchronised. → 02 §7
15. Explain Berkeley algorithm with an example of three machines. → 02 §7
16. Why is a consistent cut needed for recording a global state? Give an example of an inconsistent cut. → 02 §9
17. What is "Possibly Φ" and "Definitely Φ" in distributed debugging? → 02 §10
18. Explain why Ricart–Agrawala guarantees mutual exclusion and freedom from starvation. → 02 §12
19. What happens in the Ring election algorithm if two processes start elections at the same time? → 02 §13
20. Explain causal ordering vs total ordering in multicast with an example each. → 02 §14

PART C — 15 LONG-ANSWER QUESTIONS (7–10 marks)

Claude Predicted Practice Question — each one.

1. Define distributed system. Discuss its characteristics, advantages and disadvantages, and the main design challenges. → 01 §1–4
2. Discuss in detail the architectural models of distributed systems and explain how middleware supports them. → 01 §5.2
3. Explain the interaction, failure and security models. Which assumptions does each make about the network? → 01 §5.3
4. Explain the TCP/IP protocol stack and how internetworking supports distributed applications. → 01 §6
5. Explain interprocess communication using sockets (UDP and TCP), marshalling and the request–reply protocol. → 01 §7
6. Explain RPC in detail: components, steps, failure semantics and issues. Compare with RMI. → 01 §10
7. Explain the complete architecture and working of Java RMI with a student-database example. → 01 §12
8. Explain events and notifications with the publish–subscribe model, roles, subscription types and applications. → 01 §11
9. Discuss physical clock synchronisation algorithms (Cristian, Berkeley, NTP) with formulas and examples. → 02 §7
10. Explain happened-before, Lamport clocks and vector clocks; work out an example with 3 processes and 8 events. → 02 §4–6
11. Explain global states, cuts, consistency and the Chandy–Lamport algorithm; prove that the recorded state is consistent. → 02 §9
12. Discuss distributed debugging: predicates, lattice of global states, possibly/definitely, and monitor process design. → 02 §10
13. Compare all mutual exclusion algorithms on messages, delay and failure tolerance. → 02 §12
14. Discuss multicast communication: reliable multicast properties, FIFO/causal/total order and implementation. → 02 §14
15. Explain consensus, Byzantine generals and interactive consistency; state bounds on number of faulty processes. → 02 §15

PART D — 10 APPLICATION / CASE-STUDY QUESTIONS

Claude Predicted Practice Question — each one.

1. **Online ticket booking:** many users book the last seat simultaneously. Which distributed-system characteristics and which mutual-exclusion mechanism apply? → 01 §2, 02 §12
2. **YouTube-like service:** identify architectural elements (proxy/caches, multiple servers, CDN) and fundamental-model issues (latency, failures). → 01 §5
3. **Chat application:** choose message passing vs shared memory, TCP vs UDP; justify. → 01 §6–7
4. **University ERP using Java RMI:** which interfaces and objects are remote? How does the client find the service? What invocation semantics? → 01 §12
5. **Stock-price alerts to 10,000 traders:** design using publish–subscribe; discuss reliability and ordering. → 01 §11, 02 §14

6. **Distributed logging across servers:** how would you order events from servers with drifting clocks? Compare NTP with logical clocks. → 02 §5–8
7. **Bank with branches:** design a method to compute total money at a consistent instant while transfers continue. → 02 §9
8. **Cluster with a master node** that crashes every few days: choose an election algorithm and justify; discuss the effect of a flapping high-ID node. → 02 §13
9. **Replicated database with 7 replicas, 2 may be faulty:** can consensus be reached (crash vs Byzantine)? How many rounds? → 02 §15
10. **Group chat with total ordering:** ensure every member sees messages in the same order; describe the sequencer method and its weakness. → 02 §14

PART E — 10 NUMERICAL / PROBLEM-BASED QUESTIONS

Claude Predicted Practice Question — each one. (Answers below.)

N1. P1: a, b(send→P2), c. P2: d, e(recv from P1), f(send→P3). P3: g, h(recv from P2). All clocks start at 0. Give Lamport timestamps. N2. P2's clock reads 8; it receives a message with timestamp 8. Then P2 sends a message. What timestamps result? N3. Vector clocks (3 processes): P1 sends message with V = (2,0,0). P2 currently has V = (0,3,1). What is P2's clock after receiving? Does the send happen-before an event of P2 with V = (0,3,1)? N4. Cristian: T0 = 09:15:00.000, T1 = 09:15:00.400, server time = 09:15:07.100, minimum one-way delay = 0.05 s. Find the corrected time and accuracy. N5. Berkeley: master 10:00, slave A 09:58, slave B 10:07, slave C 10:03 (master included). Compute adjustments. N6. Two clocks each drift at most 5×10^{-6} s/s. After 10 minutes since sync, what is the maximum skew? N7. Compare number of messages for 10 processes: central server, Ricart–Agrawala, Lamport ME, ring (best/worst). N8. Bully algorithm: 8 processes, coordinator P8 crashes, process P7 detects. How many messages? What if P1 detects (count roughly)? N9. Ring election with N = 10, initiator is the anticlockwise neighbour of the highest-id process. Total messages? Best case? N10. Chandy–Lamport: balances P1 = 400, P2 = 350, P3 = 250. Before snapshot: P1 sends 30 to P2. P1 starts snapshot after sending. P3 sends 10 to P1 before receiving marker. FIFO channels. Find recorded local states and channel states, verify total.

Answers to numericals

N1. a=1, b=2 (msg ts 2); P2: d=1, e=max(1,2)+1=3, f=4 (ts 4); P3: g=1, h=max(1,4)+1=5. c (P1) = 3. **N2.** Receive: max(8,8)+1 = **9**; send next: **10**. **N3.** Receive: P2 first takes max((0,3,1), (2,0,0)) = (2,3,1), then own increment → **(2,4,1)**. Send event with (2,0,0) < (0,3,1)? Compare: 2 > 0 in first component ⇒ not ≤ ⇒ **not comparable ⇒ concurrent**, so no happened-before. **N4.** RTT = 0.4 s → time = 7.100 + 0.2 = **09:15:07.300**. Accuracy = ±(0.2 − 0.05) = **±0.15 s**. **N5.** Offsets: master 0, A −2, B +7, C +3. Avg = (0 −2 +7 +3)/4 = **+2 min** → target 10:02. Adjustments: master +2; A +4; B −5; C −1. **N6.** Max skew = $2\rho\Delta t = 2 \times 5\times10^{-6} \times 600$ = **6 ms**. **N7.** N = 10: central = 3; RA = 2×9 = **18**; Lamport = 3×9 = **27**; ring = 1 to 9 messages per entry (plus continuous token circulation). **N8.** P7 detects: ELECTION to P8 (1); P8 silent; P7 wins after timeout; COORDINATOR to P1–P6 = 6 ⇒ **N−2 = 6 coordinator messages + 1 election = 7**. If P1 detects: ELECTION to P2–P8 (7), OK replies from P2–P7 (6), and each of P2–P7 starts its own election to higher ids… total is on the order of **N^2 ≈ 60+** (roughly 7+6+

(6+5+...+1 elections + replies) + 7 coordinator). **N9.** Worst case = 3N−1 = **29** messages (9 election to reach highest + 10 to circle back + 10 elected). Best case = 2N = **20**. **N10.** Sequence: P1 sends 30 to P2 (P1 = 370, m in C12). P1 records **S1 = 370**, sends markers on C12, C13, records C21, C31. P3 sends 10 to P1 (P3 = 240, message on C31 before its marker). P2 receives 30 (P2 = 380) **then** marker on C12 → records **S2 = 380**, C12 = {}. P3 gets marker → records **S3 = 240**, C13 = {}. P1 receives 10 from P3 while recording → **C31 = {10}**, then marker. Total = 370 + 380 + 240 + 10 = **1000** = 400 + 350 + 250 ✓.

🔥 TOP 30 QUESTIONS TO PREPARE FIRST

(Ranking is my judgement from the official syllabus + the emphasis in your two BPIT assignments. It is not a guarantee.) Priority: 🔥🔥🔥 very high, 🔥🔥 high, 🔥 medium. Marks = expected marks at typical BPIT/GGSIPU-style weightage.

#	Question	Unit	Topic	Exp. marks	Priority	Why important
1	Design a Java RMI application (calculator/student service): architecture, interface, server, client, registry, code, output	I	Java RMI	5–10	🔥🔥🔥	Assignment case study + official "Case study – Java RMI"
2	Explain distributed objects and RMI (remote object, interface, stub/proxy, steps)	I	Distributed objects / RMI	4–8	🔥🔥🔥	A1-3b; syllabus
3	RMI vs RPC	I	RMI/RPC	3–5	🔥🔥🔥	Standard comparison; both in syllabus
4	Message passing vs shared memory	I	IPC	3–5	🔥🔥🔥	A1-3a explicit comparison
5	Compare client–server, P2P, grid, cloud	I	Examples of DS	4–5	🔥🔥🔥	A1-1b + syllabus lists the four
6	Explain system models: architectural & fundamental (diagrams)	I	System models	5–10	🔥🔥🔥	A1-2a
7	Define DS and explain characteristics with examples	I	Characteristics	2–5	🔥🔥🔥	A1-1a; base for compulsory question
8	Networking & internetworking: role of protocols	I	Networking	3–5	🔥🔥	A1-2b
9	Explain RPC (stubs, steps, semantics)	I	RPC	3–5	🔥🔥	Syllabus topic; needed for RMI vs RPC
10	Events and notifications (publish–subscribe)	I	Events	3–5	🔥	Syllabus topic, not in assignment → short/medium likely

#	Question	Unit	Topic	Exp. marks	Priority	Why important
11	Advantages of distributed systems	I	Advantages	1–3	🔥	Syllabus; easy marks
12	Lamport logical clock: rules + timestamp numerical	II	Logical clocks	3–5	🔥🔥🔥	A2-1b; numerical topic
13	Happened-before relation with example	II	Events/order	2–3	🔥🔥🔥	A2-1b; foundation
14	Vector clocks vs Lamport	II	Logical time	3–4	🔥	Natural follow-up; tough-paper question
15	Need for physical clock sync; skew, drift, delay	II	Physical clocks	3–4	🔥🔥	A2-1c
16	Cristian & Berkeley algorithms (with numericals)	II	Physical clocks	3–5	🔥🔥	Standard numericals; supports A2-1c
17	Logical vs physical time table	II	Time	2–3	🔥🔥	Standard compare question
18	Events and process states; role in synchronisation	II	Sync intro	2	🔥🔥	A2-1a
19	Global state: components and importance of consistency	II	Global state	3–4	🔥🔥🔥	A2-2a
20	Distributed snapshot (Chandy–Lamport) with P1, P2, P3	II	Global state	4–8	🔥🔥🔥	A2-2b case study
21	Consistent vs inconsistent cut (diagram)	II	Global state	2–3	🔥🔥	Ties to 19–20
22	Distributed debugging	II	Debugging	2–4	🔥	Syllabus; not in assignment
23	Requirements of mutual exclusion + any two algorithms	II	Mutual exclusion	4–8	🔥🔥🔥	A2-3a
24	Ricart–Agrawala algorithm with example & message complexity	II	Mutual exclusion	4–6	🔥🔥🔥	Most standard permission-based algorithm
25	Election: Bully algorithm with example	II	Election	4–6	🔥🔥🔥	A2-3b
26	Ring election algorithm with example	II	Election	4–6	🔥🔥🔥	A2-3b
27	Bully vs Ring comparison (messages, assumptions)	II	Election	3–5	🔥🔥🔥	A2-3b asks it explicitly
28	Consensus: definition, agreement, validity, termination, example	II	Consensus	3–5	🔥🔥🔥	A2-4a
29	Consensus with 5 processes, 1 may fail; failures & delays	II	Consensus	4–8	🔥🔥🔥	A2-4b case study

#	Question	Unit	Topic	Exp. marks	Priority	Why important
30	Multicast communication: reliable, FIFO/causal/total ordering	II	Multicast	3–5	🔥	Syllabus; likely short/medium

Study order suggestion: 1–7 (Unit-I core) → 12, 13, 19, 20 (Unit-II core numericals/diagram) → 23–29 (algorithms) → 15–18 → the 🔥 items.

THREE MODEL MID-SEM PAPERS (Unit-I + Unit-II ONLY)

> **These are practice papers made by Claude from the syllabus and assignment emphasis. They are *not* leaked papers and there is no guarantee any question will appear.** Marks (30 each) and time (about 90 minutes) are assumptions — ask your faculty for the real pattern and scale accordingly. **Attempt each paper on paper, timed, before opening the answer key.**

PAPER 1 — NORMAL DIFFICULTY

Course: CIE-407T Distributed Systems & Cloud Computing · **Syllabus:** Unit-I & II · **Max marks:** 30 · **Time:** 90 min

Q1. Answer any FIVE (2 marks each) — Short-answer / conceptual [10] - (a) Define a distributed system. Give two of its characteristics. - (b) What is marshalling? What is the role of a stub? - (c) State the three rules of the happened-before relation. - (d) What is a critical section? State the safety requirement of distributed mutual exclusion. - (e) Differentiate omission failure and arbitrary failure. - (f) What is the role of the RMI registry?

Q2. Comparison [5] Compare **message passing and shared memory** as IPC mechanisms (working, speed, synchronisation, scalability, applications).

Q3. Numerical [5] Three processes start with Lamport clocks = 0. P1: a (internal), b (send m1 → P2), c (internal), d (receive m3 from P3). P2: e (receive m1), f (internal), g (send m2 → P3). P3: h (internal), i (internal), j (receive m2), k (send m3 → P1). (i) Find the Lamport timestamp of every event. (ii) Are events b and h concurrent? Justify.

Q4. Diagram-based [5] Draw and explain the architecture of Java RMI. Show remote interface, stub, registry, server and client with the steps of execution.

Q5. Application / Case study [5] A college's five lab servers S1–S5 use S5 (highest id) as coordinator. S5 crashes and S2 notices. Using the **Bully algorithm**, show the messages exchanged and the new coordinator. State the best and worst-case message count for N processes.

ANSWER KEY — PAPER 1

Q1. (a) A system whose components on networked computers communicate and coordinate **only by message passing**. Characteristics: concurrency; no global clock; independent failures. (b) **Marshalling** = converting arguments/results to a flat byte format for transmission. **Stub** = client-side proxy that marshals the request, sends it, and unmarshals the reply, hiding remote communication. (c) HB1: events in the same process are ordered by

occurrence. HB2: send(m) → receive(m). HB3: transitivity (a→b, b→c ⇒ a→c). (d) **Critical section** = code accessing a shared resource. Safety: at most **one** process in the CS at a time. (e) Omission: process crashes or message lost (fails to act). Arbitrary/Byzantine: behaves incorrectly – wrong or contradictory values, corrupts messages; hardest to handle. (f) The RMI registry is a naming service (port 1099) mapping names to stubs; the server `rebind`s, the client `lookup`s.

Q2. Definition of each (message passing = explicit send/receive; shared memory = common region read/written with locks) + table: | Point | Message passing | Shared memory | |---|---|---| | Location | same or different machines | same machine (or DSM) | | Speed | slower (copy, network) | faster | | Synchronisation | implicit in blocking send/receive | explicit locks/semaphores | | Scalability | high | low | | Isolation/security | better | poor | | Applications | client-server, chat, MPI | multithreading, OS shared segments | Conclusion: DS uses message passing since nodes have no common physical memory. (Diagram of both: 1 mark.)

Q3. | Event | Rule | L | |---|---|---| | a | 0+1 | 1 | | b (send m1, ts=2) | 1+1 | 2 | | e (recv m1) | max(0,2)+1 | 3 | | f | 3+1 | 4 | | g (send m2, ts=5) | 4+1 | 5 | | h | 0+1 | 1 | | i | 1+1 | 2 | | j (recv m2 ts=5) | max(2,5)+1 | 6 | | k (send m3 ts=7) | 6+1 | 7 | | c | 2+1 | 3 | | d (recv m3 ts=7) | max(3,7)+1 | 8 | (ii) b → e → f → g → j → k → d is a causal chain, but there's **no message path from b to h or h to b** (h is before j only). So **b ‖ h** (concurrent) even though L(h)=1 < L(b)=2: Lamport clocks cannot show concurrency.

Q4. Marking: diagram (2), components (1.5), steps (1.5). Diagram: Server (remote object) ←→ RMI Registry (1099) ←→ Client (stub). Steps: (1) define remote interface extends Remote; (2) implementation extends UnicastRemoteObject; (3) server creates object and `rebind("Name",obj)`; (4) client `lookup("Name")` → stub; (5) client calls method on stub; (6) stub marshals & sends; (7) server unmarshals, invokes real object; (8) result returned, unmarshalled. (See `01 §12`.)

Q5. S2 → ELECTION to S3, S4, S5 (3). S3, S4 → OK (2). S5 crashed (silent). S3 → ELECTION to S4, S5 (2); S4 → OK to S3 (1). S4 → ELECTION to S5 (1) → timeout, no reply. S4 announces **COORDINATOR(S4)** to S1, S2, S3 (3). **New coordinator = S4.** Total = 12 messages. Best case: second-highest process detects → **N−2** coordinator messages; worst case: lowest-id process detects → **O(N²)**.

PAPER 2 — MODERATE DIFFICULTY

Max marks: 30 · **Time:** 90 min

Q1. Answer any FOUR (2 marks each) [8] - (a) What is middleware? Why is it needed? - (b) Distinguish synchronous and asynchronous distributed systems. - (c) Write the three invocation semantics of RPC/RMI and name the one used by Java RMI. - (d) Define clock skew and clock drift. - (e) What is a consistent cut? - (f) List the properties of reliable multicast.

Q2. Diagram-based / Conceptual [5] Explain **architectural models** of distributed systems. Draw diagrams for (i) software layers with middleware, (ii) client–server with multiple servers, (iii) proxy/cache.

Q3. Numerical [5] (a) *Cristian's algorithm:* client sends request at T0 = 14:00:00.000, receives reply at T1 = 14:00:01.000; server time in reply = 14:00:20.500. Compute the client's new time and the error bound if the minimum one-way delay is 0.1 s. (b) *Berkeley algorithm:* master clock 5:00, slave A 4:52, slave B 5:14. Compute each machine's adjustment.

Q4. Comparison [4] Compare **logical time and physical time** (any 6 points) and mention one algorithm for each.

Q5. Case study [8] Three bank servers P1, P2, P3 initially hold ₹600, ₹400, ₹200 (total ₹1200). P1 sends ₹100 to P2 (m1); P2 sends ₹50 to P3 (m2). P3 initiates a snapshot after m2 was sent but before it arrives. Using the **Chandy–Lamport algorithm** (FIFO channels) show the markers, the recorded local states and channel states, and verify that the total is ₹1200. State the assumptions of the algorithm.

ANSWER KEY — PAPER 2

Q1. (a) Software layer between OS and applications masking heterogeneity (hardware, OS, network, languages) and giving a uniform programming model (RMI, RPC, CORBA). (b) Synchronous: known bounds on execution time, message delay, drift → timeouts detect failures. Asynchronous: no bounds (Internet) → slow vs crashed indistinguishable. (c) Maybe, at-least-once, at-most-once; Java RMI = **at-most-once**. (d) Skew = difference between two clocks' readings at the same instant. Drift = rate at which a clock deviates from a perfect clock. (e) A cut in which for every receive event inside the cut its send event is also inside (no orphan message). (f) Integrity, validity, agreement.

Q2. (Refer `01 §5.2`.) Answer skeleton: definition (architectural = structure/placement of components); (i) layers diagram: Applications → Middleware → OS → Hardware (platform = OS + HW); (ii) multiple servers: clients → server 1/2/3 cooperating (partition/replicate); (iii) proxy: client → proxy/cache → server (reduces latency & load); add P2P, mobile code, mobile agents, thin clients in one line each; conclusion. Marks: 3 diagrams (2.5) + explanation (2.5).

Q3. (a) RTT = 1.000 s; new time = 20.500 + 0.500 = **14:00:21.000**. Accuracy = ±(RTT/2 − min) = ±(0.5 − 0.1) = **±0.4 s**. (b) Offsets: master 0, A −8, B +14 (minutes). Average = (0 − 8 + 14)/3 = **+2 min** → target time 5:02. Adjustments: **master +2, A +10, B −12** minutes.

Q4. Table (`02 §8`): meaning; based on; purpose; sync needed; affected by drift; example; relation to real time; detects concurrency. Algorithms: physical → Cristian/Berkeley/NTP; logical → Lamport clock (or vector clock).

Q5. 1. P1 sends m1 (₹100): P1 = 500. P2 sends m2 (₹50): P2 = 350. 2. P3 (initiator): records **S3 = 200**, sends MARKERs on C31, C32; starts recording C13, C23. 3. m2 arrives at P3 (before P2's marker on C23) → recorded into channel C23. 4. P1 receives first MARKER (on C31): records **S1 = 500**, C31 = ∅, sends MARKERs on C12 (after m1) and C13; records C21. 5. P2 receives P3's MARKER (C32) [before m1 arrives]: first marker → records **S2 = 350**, C32 = ∅, sends MARKERs on C21, C23; starts recording C12. 6. m1 arrives at P2 while recording C12 → then P1's MARKER on C12 → **C12 = {₹100}**. 7. P3 receives P2's MARKER on C23 → **C23 = {₹50}** (m2 arrived while recording). P3 receives P1's marker on C13 → C13 = ∅. P1 receives P2's marker on C21 → C21 = ∅. **Recorded state:** S1 = 500, S2 = 350, S3 = 200, C12 = {100}, C23 = {50}. Sum = 500+350+200+100+50 = **₹1200** ✓. Assumptions: FIFO, reliable,

unidirectional channels; no process/channel failure during snapshot; strongly connected network; any process may initiate. (Diagram + message flow: marks included.)

PAPER 3 — TOUGH / CONCEPTUAL

Max marks: 30 · **Time:** 90 min

Q1. Answer any FOUR – reasoning questions (2 marks each) [8] - (a) Lamport clocks satisfy a→b ⇒ L(a)<L(b). Why can we not conclude a→b from L(a)<L(b)? Give a small example. - (b) Why must channels be FIFO in the Chandy–Lamport algorithm? What may go wrong otherwise? - (c) Why does the Bully algorithm assume a synchronous system, whereas the Ring algorithm does not? - (d) Why can Byzantine agreement with one traitor not be reached among three generals? - (e) Why must a clock never be set backwards during synchronisation? What is done instead? - (f) Why is total-ordered multicast harder to implement than FIFO-ordered multicast?

Q2. Analysis [4] "An asynchronous system with crash failures cannot guarantee consensus." Explain this statement, and describe two practical techniques used to work around it.

Q3. Numerical / concurrency detection [5] Use **vector clocks** for 3 processes (initially (0,0,0); increment own component before every event): P1: a (internal), b (send m1 → P2), c (receive m3 from P3). P2: d (receive m1), e (send m2 → P3). P3: f (internal), g (receive m2), h (send m3 → P1). (i) Compute the vector timestamp of every event. (ii) Show that a → h. (iii) Show that f and b are concurrent. (iv) Compute Lamport timestamps for the same events and identify a pair where Lamport clocks mislead.

Q4. Application / Comparison [5] Six servers share a log file that is updated a few times per hour. Compare **central-server, token-ring, Ricart–Agrawala and Lamport's** mutual-exclusion algorithms for N = 6 on message count, synchronisation delay and failure behaviour. Recommend one with justification.

Q5. Case study [8] A replicated database has 7 processes that must agree on a commit/abort decision. (a) If up to 2 processes may **crash** (synchronous system), how many rounds are needed and how does the flooding algorithm reach consensus? (b) If up to 2 processes may be **Byzantine**, is consensus possible with 7 processes? With 6? Why? (c) State the three conditions of consensus and the effect of unbounded message delay.

ANSWER KEY — PAPER 3

Q1. (a) The clock condition works only one way. Example: P1 executes local event x (L=1); P2 executes event y (L=2) after a previous local event; no message between them → L(x)<L(y) but x ∥ y. Only vector clocks capture both directions. (b) The marker separates messages sent **before** the sender recorded its state from those **after**. With non-FIFO channels a message sent after the marker could overtake it and be counted as "before", or the reverse, producing an inconsistent state (lost or double-counted money). (c) Bully depends on **timeouts** to decide that a higher-ID process is dead (no OK ⇒ declare self coordinator) – reliable only if delays are bounded (synchronous). Ring never waits for absence of a reply; it passes an ID around and a process wins only when its own ID returns, so it does not need timing bounds (only a repaired

ring). (d) With 3 generals and 1 traitor a loyal lieutenant cannot tell whether the commander or the other lieutenant is lying (symmetric information) — Byzantine agreement needs N ≥ 3f+1 = 4. (e) Setting the clock backwards would make time repeat and break ordering (e.g., file timestamps, `make`, logs). Instead the clock is **slowed down** (lower tick rate) until it catches up. (f) FIFO needs only per-sender sequence numbers; total order requires all processes to **agree on one global order** — needs a sequencer (SPOF/bottleneck) or an agreement protocol (ISIS) ⇒ effectively consensus.

Q2. FLP (Fischer–Lynch–Paterson 1985): in a purely asynchronous system, no deterministic algorithm can guarantee termination if even one process may crash, because a slow process cannot be distinguished from a crashed one; waiting forever violates termination, deciding early risks disagreement. Workarounds: (i) **failure detectors / timeouts** (partial synchrony, e.g., Paxos/Raft leader election with timeouts); (ii) **randomisation** (probabilistic termination); also (iii) assume synchrony bounds.

Q3. (i) | Event | Vector | |---|---| | a | (1,0,0) | | b (send m1) | (2,0,0) | | d (recv m1) | max((0,0,0), (2,0,0)) = (2,0,0) → own+1 = **(2,1,0)** | | e (send m2) | **(2,2,0)** | | f | **(0,0,1)** | | g (recv m2) | max((0,0,1),(2,2,0)) = (2,2,1) → +1 = **(2,2,2)** | | h (send m3) | **(2,2,3)** | | c (recv m3) | max((2,0,0),(2,2,3)) = (2,2,3) → +1 = **(3,2,3)** | (ii) V(a) = (1,0,0) < V(h) = (2,2,3) (every component ≤, one strictly <) ⇒ **a → h**. (iii) V(f) = (0,0,1), V(b) = (2,0,0): f ≰ b (third component 1 > 0) and b ≰ f (first 2 > 0) ⇒ **f ‖ b**. (iv) Lamport: a=1, b=2; d=max(0,2)+1=3, e=4; f=1, g=max(1,4)+1=5, h=6; c=max(2,6)+1=7. Note L(f)=1 < L(b)=2, but f ‖ b ⇒ Lamport misleads (suggests f "before" b).

Q4. (N=6) | Algorithm | Msgs/entry | Sync delay | Failure behaviour | |---|---|---|---| | Central server | 3 | 2 msg times | Server crash stops all (SPOF); needs election | | Token ring | 1–5 (+ constant circulation) | 1–5 | Token loss / node crash breaks ring | | Ricart–Agrawala | 2(N−1) = 10 | 1 | Any crash ⇒ no reply ⇒ block | | Lamport | 3(N−1) = 15 | 1 | Same, plus needs FIFO | **Recommendation:** since updates are rare (few per hour), the **central server** is best (3 messages, simple, fair); a dedicated coordinator (elected via Bully if it fails) gives fault tolerance. Ring wastes bandwidth circulating a token when nobody wants the CS; RA/Lamport are unnecessarily heavy.

Q5. (a) f = 2 crashes ⇒ **f + 1 = 3 rounds**. Each process keeps a set of values; in each round multicasts the values not yet sent, adds received values; after round 3 decides deterministically (e.g., min or majority) of its set. Since among 3 rounds at least one has no crash, all correct processes have identical sets ⇒ agreement. (b) Byzantine needs **N ≥ 3f + 1**: f = 2 ⇒ N ≥ 7. **7 processes: possible** (with f+1 = 3 rounds of message exchange, oral messages OM(2)). **6 processes: impossible** (unsigned messages) — traitors can create indistinguishable views. (c) Termination (every correct process eventually decides), Agreement (same value), Validity/Integrity (decided value was proposed; if all propose v, decision = v). Unbounded delay ⇒ system asynchronous ⇒ FLP: cannot distinguish slow from crashed; must use timeouts/randomisation, accepting possible non-termination in rare runs or weaker guarantees.

Self-evaluation table

Paper	Marks /30	Weak topics to revise
1		

Paper	Marks /30	Weak topics to revise
2		
3		

ONE-DAY REVISION SHEET · VIVA QUESTIONS · ANSWER-WRITING STRATEGY

PART 1 — ⏱️ ONE-DAY REVISION SHEET (1–2 hours)

1. Key definitions (one line each)

- **Distributed system:** networked components that communicate & coordinate **only by messages**.
- **Middleware:** layer masking heterogeneity (RMI, RPC, CORBA).
- **Marshalling:** data → byte stream. **Stub/Proxy:** client-side representative. **Skeleton/Dispatcher:** server-side.
- **RMI:** call method of remote object. **RPC:** call remote procedure.
- **Happened-before →:** same process order + send→receive + transitive.
- **Global state:** process states + channel states. **Consistent cut:** no orphan messages.
- **Mutual exclusion:** one process in CS at a time. **Election:** choose one coordinator (highest id).
- **Multicast:** one → group. **Consensus:** all agree on one proposed value.

2. Characteristics (Unit-I)

Concurrency · **No global clock** · **Independent failures** (core 3) + heterogeneity, openness, scalability, transparency, fault tolerance, security, resource sharing.

3. Client–Server vs P2P vs Grid vs Cloud

	C-S	P2P	Grid	Cloud
Idea	server serves clients	all equal	many orgs share compute	rent services
Control	central	none	virtual org/scheduler	provider
SPOF	server	no	scheduler	no (redundant)
Eg	Web	BitTorrent	SETI@home, CERN	AWS, Azure
Billing	–	free	project	pay-per-use

4. System models

- **Architectural:** layers+middleware, client-server, multiple servers, proxy/cache, P2P, mobile code, mobile agents, thin client.
- **Fundamental: Interaction** (latency, bandwidth, jitter, drift; sync/async) · **Failure** (omission, arbitrary, timing) · **Security** (threats, crypto, authentication).

5. Message passing vs Shared memory

MP	SM
send/receive, works across network	common memory, same machine (DSM emulation)
slower, scalable, implicit sync	faster, needs locks, poor scalability

6. RMI architecture (draw this!)

```
Client → PROXY(stub) → comm+remote ref module ═net═ comm module → DISPATCHER → SKELETON → SERVANT
```

Semantics: maybe / at-least-once / **at-most-once (Java RMI)**.

7. Java RMI (recall)

Interface `extends Remote` + `throws RemoteException` → Impl `extends UnicastRemoteObject` → Server: `createRegistry(1099)`, `rebind("name", obj)` → Client: `getRegistry`, `lookup("name")`, call.

8. RPC vs RMI

RPC	RMI
procedures, C-like, IDL	objects/methods, Java
no object passing	objects & remote references
client stub / server stub	stub / skeleton, GC, registry

9. Events & notifications

Publisher → **Event service** → Subscribers; asynchronous, decoupled (topic/content based).

10. Lamport clock — RULES

```
event:   L = L + 1
send:    L = L + 1; attach L
receive: L = max(L, t) + 1
a → b  ⇒  L(a) < L(b)   (converse FALSE)
```

Vector clock: V[i]++; on receive V=max(V,t) then V[i]++. a→b ⇔ V(a)<V(b).

11. Physical clocks

- **Skew** = difference now. **Drift** = rate ($\rho \approx 10^{-6} \Rightarrow$ 86.4 ms/day). Resync interval $\leq \delta/2\rho$.
- **Cristian:** new = Ts + RTT/2 (± (RTT/2 − min)). **Berkeley:** master averages, sends adjustments. **NTP:** strata; $\theta = ((t1-t0)+(t2-t3))/2$.

12. Logical vs Physical

Logical: order, no sync, no drift, counters. Physical: real time, needs sync, affected by drift/skew.

13. Global state / snapshot (Chandy–Lamport)

Initiator: record state → send MARKER on all outgoing → record incoming. Receive first MARKER: record state, that channel = ∅, send markers, record other incoming. Later marker: channel = messages received since recording. FIFO channels. Example 450+350+180+20 = 1000.

14. Distributed debugging

Monitor + vector timestamps → consistent global states → check predicate. **Possibly Φ** (some path) / **Definitely Φ** (all paths). Stable vs non-stable.

15. Mutual exclusion

ME1 safety · ME2 liveness · ME3 ordering. | Alg | Msgs | |---|---| | Central | 3 | | Ring | 1..N−1 | | **Ricart–Agrawala** | **2(N−1)** | | Lamport | 3(N−1) | | Maekawa | ~3√N | RA: WANTED → multicast REQUEST(T,i) → wait N−1 replies → HELD. On request: if HELD or (WANTED & my ts smaller) **queue**, else **reply**. Exit → reply queued.

16. Election

- **Bully:** ELECTION → higher ids; OK; highest alive → COORDINATOR. Sync system, all IDs known. **N−2 best, O(N²) worst.**
- **Ring:** ELECTION(id) circulates, larger id substituted; own id returns → winner; ELECTED circulates. Async OK. **2N best, 3N−1 worst.**

17. Multicast

Reliable: integrity, validity, agreement. Ordering: FIFO (seq numbers), causal (vector clocks), total (sequencer/ISIS).

18. Consensus

Termination · Agreement · Validity(Integrity). Crash: f+1 rounds. Byzantine: N ≥ 3f+1. **FLP:** async + 1 crash ⇒ no guaranteed deterministic consensus. Related: Byzantine generals, interactive consistency, total-order multicast.

19. Keywords to sprinkle in answers

transparency · middleware · marshalling · proxy · dispatcher · at-most-once · latency · jitter · omission/Byzantine · happened-before · concurrent · causality · orphan message · marker · consistent cut · safety/liveness · timestamp · token · coordinator · quorum · agreement/validity/termination.

20. Last-hour checklist

☐ Draw RMI diagram ☐ Write Lamport rules & one example ☐ Write RA pseudo-code ☐ Draw Bully 5-process example ☐ Draw Chandy–Lamport with P1,P2,P3 ☐ Recite consensus 3

conditions ☐ Recite 4-way comparison ☐ Recite MP vs SM ☐ Recite Cristian & Berkeley formulas.

PART 2 — 🎤 30 VIVA / ORAL QUESTIONS

Unit-I 1. **What is a distributed system?** Independent computers connected by a network that coordinate only through messages and appear to the user as one system. 2. **Why do we need distributed systems?** For resource sharing, scalability, reliability and performance which one computer cannot give alone. 3. **What is transparency?** Hiding the distribution — user doesn't know where resources are, how many copies exist or if failures happened. 4. **Difference between client–server and P2P?** In client–server roles are fixed and server is central; in P2P every node is both client and server. 5. **How is grid different from cloud?** Grid pools computers of many organisations for large computations (batch, sharing); cloud rents on-demand services from a provider with pay-per-use. 6. **What is middleware?** Software between OS and application that hides differences of hardware/OS/network (e.g., RMI, CORBA). 7. **Why do we need system models?** They state assumptions clearly so we can design and prove algorithms without depending on a particular system. 8. **What is the difference between architectural and fundamental models?** Architectural = structure/placement of components; fundamental = abstract properties (interaction, failure, security). 9. **What is the difference between synchronous and asynchronous systems?** Synchronous has known bounds on execution/message delay/drift; asynchronous has none, so timeouts can't reliably detect crashes. 10. **What is an internetwork?** Network of networks connected by routers (the Internet). 11. **TCP vs UDP?** TCP is reliable, ordered, connection-oriented; UDP is connectionless, faster but unreliable. 12. **What is IPC and its two types?** Way for processes to exchange data: message passing and shared memory. 13. **Why does DS prefer message passing?** No common physical memory between machines; messages work over networks and give isolation. 14. **What is a remote object?** An object whose methods can be invoked from another process through a remote reference. 15. **How does RMI work?** Client calls the stub, which marshals the call and sends it; server skeleton/dispatcher unmarshals, calls the real object and sends result back. 16. **What is the difference between RPC and RMI?** RPC calls procedures (procedural, no objects); RMI calls methods on remote objects and can pass objects/references. 17. **What is an RMI registry?** A naming service (port 1099) where servers bind names to stubs and clients look them up. 18. **Why does every remote method throw `RemoteException`?** Because network or server failure can occur and the caller must handle it. 19. **What are events and notifications?** Event = something of interest; notification = message telling subscribers about it via an event service (publish–subscribe).

Unit-II 20. **Why do we need synchronization?** No global clock and no shared memory, so processes need methods to agree on order, resource access, leader and decisions. 21. **What is the happened-before relation?** Causal ordering: same-process order, send→receive, and transitivity; events without a path are concurrent. 22. **How does a Lamport clock work?** Increment before every event; send timestamp with message; on receive set clock = max(local, message ts)+1. 23. **Can Lamport clocks detect concurrency?** No; vector clocks can. 24. **Difference between clock skew and drift?** Skew is the difference between two clocks now; drift is the rate at which a clock deviates from real time. 25. **How does Cristian's algorithm set the time?** Time = server time + RTT/2. 26. **What is a global state /**

consistent cut? Local states of all processes + messages in transit; consistent if no message is received without being sent inside the cut. 27. **How does the snapshot algorithm work?** Initiator records its state and sends markers; others record on the first marker; messages arriving between recording and marker form channel state. 28. **State the requirements for mutual exclusion.** Safety (one in CS), liveness (no deadlock/starvation), ordering (happened-before fairness). 29. **How does the Bully algorithm elect a leader?** A process that detects failure sends ELECTION to higher IDs; if none answers it becomes coordinator; otherwise the higher process continues; highest alive ID wins. 30. **What is consensus and its conditions?** All correct processes decide the same value; requires termination, agreement and validity.

Bonus quick ones: Which ring election message count is worst? 3N−1. Ricart–Agrawala messages? 2(N−1). Byzantine tolerance bound? N ≥ 3f+1. Rounds for f crashes? f+1.

PART 3 — ✍️ HOW TO WRITE 5-MARK AND 10-MARK ANSWERS

3.1 The universal skeleton

Step	What to write	Time
1	**Definition** (1–2 lines, boxed/underlined keyword)	1 min
2	**Concept in simple words** (2–3 lines)	1 min
3	**Diagram** (neat, labelled, numbered arrows, caption)	2–3 min
4	**Working / steps** (numbered points; algorithm/pseudocode if asked)	3–5 min
5	**Example** (small numeric or real-life)	2 min
6	**Advantages / disadvantages** (3+3 short points)	1 min
7	**Conclusion** (1 line linking to distributed systems)	30 s

5-mark answer = ~1 page: definition (1) + diagram (1.5) + working (1.5) + example/adv-disadv (1). **10-mark answer** = ~3 pages: definition + explanation + 2 diagrams + detailed steps + example + comparison table + adv/disadv + conclusion.

Tips 1. Underline keywords (transparency, marshalling, happened-before…). 2. Always draw diagrams with a ruler/pencil and **label arrows with message names**. 3. Use tables for comparisons (minimum 6 rows). 4. For algorithms: purpose → assumptions → steps → example → messages → pros/cons. 5. For numericals: **show the rule, then a table of calculations, then the final boxed answer**. 6. Write "Ans:" and box the final result. 7. Don't leave any question blank – attempt definition + diagram at least. 8. For Java code, write the 4 files with brief comments and the sample output.

3.2 SAMPLE 5-MARK ANSWER

Q. Explain Lamport's logical clock with rules and an example. [5]

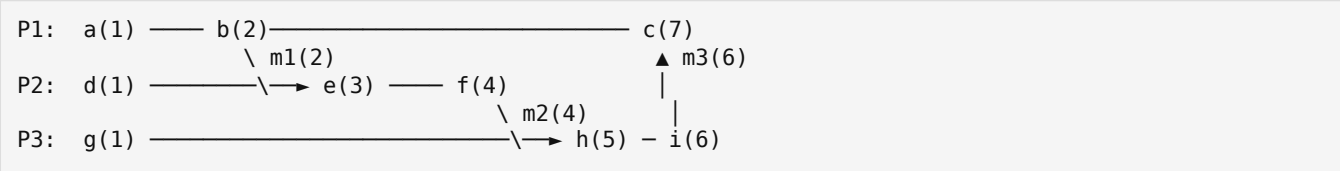
Definition: A *Lamport logical clock* is a software counter kept at each process that assigns timestamps to events such that **if a → b then L(a) < L(b)**. It orders events by causality without using physical clocks.

Why needed: In a distributed system there is **no global clock** and physical clocks drift, so events on different computers cannot be ordered by real time.

Happened-before (→): (1) events in same process ordered; (2) send(m) → receive(m); (3) transitivity. Events not related are **concurrent**.

Rules - LC1: Before each event, `L = L + 1`. - LC2: When sending, attach current `L` to the message. - LC3: On receiving (m, t): `L = max(L, t) + 1`.

Diagram (timeline)



Working: e receives m1 (t = 2) so L = max(1, 2)+1 = 3; h receives m2 (t = 4): max(1,4)+1 = 5; c receives m3 (t = 6): max(2,6)+1 = 7.

Advantages: simple (one integer); preserves causality; no clock sync. **Limitations:** if L(a) < L(b) we cannot say a → b (a and b may be concurrent, e.g., a(1) and d(1)); cannot detect concurrency – vector clocks are needed.

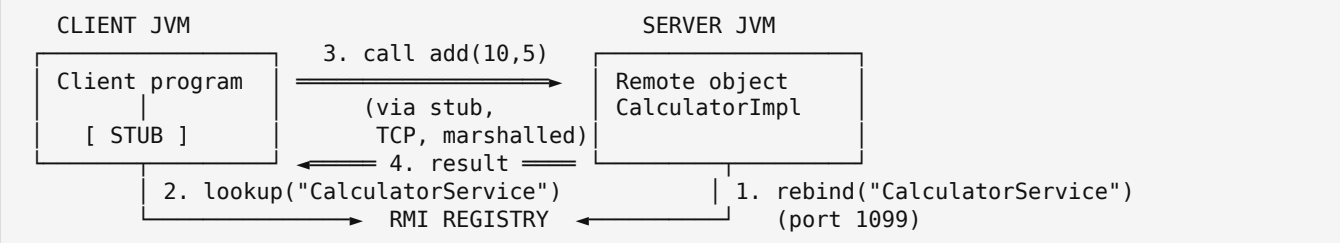
Conclusion: Lamport clocks give a consistent ordering of events in distributed systems and form the basis for total ordering in algorithms like Ricart–Agrawala mutual exclusion.

3.3 SAMPLE 10-MARK ANSWER

Q. Design a simple Java RMI application (Calculator). Explain architecture, remote interface, server, client, registry and execution steps with code and output. [10]

1. Introduction (1 mark). *Java RMI (Remote Method Invocation)* lets an object in one JVM invoke methods of an object in another JVM (possibly on another machine) as if it were local. It follows the distributed-object model, hides communication through **stubs**, and gives **at-most-once** semantics.

2. Architecture diagram (2 marks)



3. Components (2 marks) | Component | Role in the calculator | |---|---| | Remote interface `Calculator` | extends `Remote`; declares add/subtract/multiply/divide with `throws RemoteException`

| | Remote object `CalculatorImpl` | implements the interface, extends `UnicastRemoteObject` (exports the object) | | Server | creates the object, starts the registry, `rebind`s the name | | Registry | naming service; maps "CalculatorService" → stub | | Stub | client-side proxy that marshals the call | | Client | looks up the stub and invokes methods |

4. Code (3 marks)

```
// Calculator.java
import java.rmi.*;
public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
}
// CalculatorImpl.java
import java.rmi.*; import java.rmi.server.*;
public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    public CalculatorImpl() throws RemoteException { super(); }
    public int add(int a, int b) { return a + b; }
    public int multiply(int a, int b) { return a * b; }
}
// CalculatorServer.java
import java.rmi.registry.*;
public class CalculatorServer {
    public static void main(String[] args) throws Exception {
        Registry r = LocateRegistry.createRegistry(1099);
        r.rebind("CalculatorService", new CalculatorImpl());
        System.out.println("Server ready");
    }
}
// CalculatorClient.java
import java.rmi.registry.*;
public class CalculatorClient {
    public static void main(String[] args) throws Exception {
        Registry r = LocateRegistry.getRegistry("localhost", 1099);
        Calculator c = (Calculator) r.lookup("CalculatorService");
        System.out.println("10 + 5 = " + c.add(10, 5));
        System.out.println("10 * 5 = " + c.multiply(10, 5));
    }
}
```

5. Execution steps (1.5 marks) 1. `javac *.java` 2. `java CalculatorServer` (registry starts, object registered) 3. `java CalculatorClient` 4. Client contacts registry → gets stub → calls `add(10,5)` → stub marshals → server executes → result unmarshalled → printed.

6. Sample output (0.5 mark)

```
Server ready                    (server terminal)
10 + 5 = 15                     (client terminal)
10 * 5 = 50
```

Advantages: simple, OO, automatic stubs/GC, object passing. **Limitations:** Java-only, slower than local calls, firewall issues. **Conclusion:** Java RMI implements the distributed-object model by hiding network communication behind proxies, making distributed programming close to normal object-oriented programming.

3.4 Common mistakes to avoid

Mistake	Fix
Writing Unit-III/IV (HDFS, IaaS) in Unit-I/II answers	Stay inside Unit-I/II; mention cloud only as an *example*
Saying L(a)<L(b) ⇒ a→b	Only a→b ⇒ L(a)<L(b)
Confusing skew and drift	Skew = difference; drift = rate
Bully "lowest id wins"	**Highest** id wins in both Bully and Ring
Forgetting `throws RemoteException`	Always include; interface must extend `Remote`
Diagrams without labels	Label every arrow with message name/step number
Ignoring assumptions of algorithms	FIFO for snapshot; synchronous for Bully; reliable delivery for RA
Writing only theory for numericals	Show rule → table → boxed answer

---

You now have: notes (01, 02) → priorities (03) → questions (04, 05) → timed papers (06) → last-day sheet (this file). **Good luck for the mid-sem!**